# openvaf-r — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

August 2026

# Contents

# openvaf-r — Full Change Report

Every modification applied to the OpenVAF-Reloaded compiler in this project, with its reason. The baseline is the pristine OpenVAF-Reloaded source tree (as vendored in the project's `original/` snapshot); the current state is the tree committed in this repository under `OpenVAF-master-20260610/`. Each entry links the enhancement write-up in `enhancements_doc/` that carries the full engineering detail and the verifying example suite. The companion document ngspice_changes_full-report.md covers the simulator side.

> Maintenance note: this report is updated whenever an enhancement touches compiler sources. The per-enhancement index at the end tells you the last change it covers.

Scope summary. ~120 modified source files across the compiler pipeline, two new files (`hir/src/elaborate.rs` — the elaboration pass — and `hir_def/src/item_tree/diagnostics.rs`), plus three classes handled wholesale: the `test_data/` fixtures and snapshots (95 files — new test inputs and snapshots regenerated whenever a feature legitimately changed descriptor contents), the `integration_tests/*/*.osdi` reference objects (regenerated at each ABI bump), and the removal of the AGPL `va-cask` submodule. The compiler builds warning-free; every change below was regression-gated by the example-suite arsenal, the integration suite, and the VA_TEST industry-model corpus.

---

## 1. Lexer — **`openvaf/lexer/`** (**`lib.rs`**, **`cursor.rs`**)

- Based integer literals (`8'hFF`, `'sb101`): the lexer had only a commented sketch; underscores crashed it. Both the sized and unsized forms now lex, requiring a digit after the base (a silent-0 trap otherwise) (E-46).
- Don't-care digits `x/X/z/Z/?` accepted in the binary/octal/hex digit runs for `casex`/`casez` items (E-78).
- Escaped identifiers (`\my-net!`) lexed per the LRM (E-46).
- **`// comment at EOF`** hang fixed: with no trailing newline the line-comment loop spun forever on the EOF character — the only EOF-unsafe loop in the lexer (E-35).

## 2. Tokens and grammar — **`openvaf/tokens/`**, **`openvaf/parser/`**, **`veriloga.ungram`**, **`sourcegen/`**

`tokens/src/parser/generated.rs` gains the keywords and node kinds each language feature needed, threaded through `parser/src/grammar/*` (`stmts.rs`, `expressions.rs`, `items.rs`, `items/module.rs`), the grammar source of truth `syntax/veriloga.ungram`, and the `sourcegen/` generators (`ast/src.rs`, `hir_builtins.rs`, `mir_instructions.rs` — the templates the generated token/AST/builtin/instruction tables come from):

- `do … while` post-test loops (E-19);
- `paramset`/`endparamset` blocks (E-21);
- `{…}` concatenation and `{n{…}}` replication as real operators, with `'{…}` remaining the typed aggregate (E-34);
- array-literal expressions actually parsed (they were fully scaffolded but unreachable) (E-4, E-14);
- vectored (bus) net declarations and bit-selects (E-3), multi-index bit-selects for N-D arrays (E-15), and the LRM name-then-range declaration order (E-18);
- module instantiation items (E-5), `generate for`/`genvar` (E-8), and `generate if`/`generate case` with optional block labels (previously mis-parsed into a broken `GENERATE_FOR`; E-67);
- `defparam` — a reserved word with no grammar rule (E-58);
- event OR lists `@(cross(…) or timer(…))` via a new `or` keyword and looped event grammar (E-59);
- `casex`/`casez` sharing the case rule, the keyword token carrying the flavor (E-78);

- `@(initial_step)`/`@(final_step)` with analysis-phase lists (E-7, E-53);
- indirect branch assignment `V(x): V(y) == expr;` (E-2);
- the <<</>>> arithmetic-shift tokens (plus a pre-existing lexer bug fix) (E-6);
- hierarchical parameter override targets (`#(.blk.p(4))`): the extra `.segments` are consumed so the parse stays clean instead of cascading -- elaboration then rejects the block-scoped override with a targeted diagnostic (E-87);
- hierarchical branch reference tails (`.branch(a,b)` / `.branch(<p>)`) swallowed into the path node so the enclosing item's CST stays whole for the elaboration rewrite (a keyword in path position used to shred the item, leaving the hole scanner truncated text) (E-86);
- part-selects in instance connections (`inst (out[3:2], in)`): the bit-select bracket accepts an optional `: expr`, with the colon token in the CST distinguishing a range from multi-dimensional indexing — no new node kind (E-85);
- module-level **`generate for`**/**`if`**/**`case`** without the optional **`generate`**/**`endgenerate`** keywords: `module_items` handled a `generate … endgenerate` region but had no bare `FOR_KW/IF_KW/CASE_KW` arm, so a keyword-less loop at module scope fell to error recovery — `unexpected token 'for'`, or a silent drop of the loop when a following `analog` block let recovery resync. New arms parse it into the same `GENERATE_FOR/IF/CASE` nodes the nested/`generate`-wrapped forms produce (E-96);
- panic-free direction parsing: `port_decl`/`func_arg` asserted the next token was a direction (`bump_ts`), which any non-Verilog text reaching a module-head port list turned into a compiler crash; both now emit a diagnostic with one token of forced progress (E-84);
- operator precedence corrected against LRM Table 4-2: `%` bound tighter than `*`/`/` (so `6*7%4` evaluated to 18, not 2), and xnor split from xor (E-38);
- derived-nature parents emitted the wrong node kind (`NAME_REF` where the AST wanted `Path`), leaving the entire inheritance machinery unreachable (E-39).
- a parenthesised COMMA LIST is no longer an expression (E-423): `paren_expr` carried a tuple-parsing loop over from rust-analyzer -- its Rust test comment (`const A: (i64, i64) = (1, #[cfg(test)] 2);`) was still in it -- so `(a, b, c)` parsed happily, `hir_def` lowering kept only the FIRST child (`ParenExpr(e) => collect_opt_expr(e.expr())`), and every later element was discarded before the HIR existed. Not merely a wrong answer: an undeclared name, a wrong argument count or a type error hiding in a dropped element was never reported. A `,` written where a `+` was meant, in a parenthesised sum split across lines, silently dropped a whole term ( measured: -5 mA -> -1 mA). Reported as its own `CommaExpr` error rather than `UnexpectedToken`, for the reason E-387 gave `ExprTooDeep` its own variant -- the source is not malformed, it just means something the author did not write. Also: a TRAILING COMMA in a call argument list (`max(1, 2,)`) ended `arg_list`'s loop on the `)` and was accepted as two arguments; a comma must now be followed by an argument.

3. Preprocessor — **`openvaf/preprocessor/`** (+ **`basedb`** diagnostics)

- Ten compiler directives that previously hard-failed as undefined macros: `` `default_discipline ``, `` `celldefine ``/`` `endcelldefine ``, `` `unconnected_drive ``/`` `nounconnected_drive ``, `` `timescale ``, `` `line ``, `` `pragma ``, `` `undefineall ``, `` `default_nettype `` (E-6).
- **`` `default_transition ``** parsed and threaded to `transition()` lowering (E-47).
- Recursive macro expansion crashed the compiler (stack overflow, direct and mutual): the `MacroRecursion` diagnostic existed but was never emitted, and its report renderer was a `todo!()`. The guard pushes around the macro *body* expansion only — an entry-scoped guard false-positives on the legal `QUAD(x) = TWICE(TWICE(x))` nesting (E-65).

4. Syntax / AST — **`openvaf/syntax/`** (**`ast.rs`**, **`generated/nodes.rs`**, **`expr_ext.rs`**, **`node_ext.rs`**, **`name.rs`**, **`validation.rs`**, **`error.rs`**)

- Typed AST nodes for every new construct above (generated from the ungrammar).

- Literal decoding (`expr_ext.rs`): based-int parsing with size/sign handling (E-46) and the mask-aware variant returning `(value, x_mask, z_mask)` for casex/casez (E-78); a compiler crash on large bare-integer-shaped literals fixed by falling back to a float constant (E-4).
- String unescaping rewritten as a single pass: the sequential `str::replace` chain corrupted overlapping escapes (`\\n`) and octal `\ddd` was missing; all consumers route through one function (E-48).
- **`Name::resolve`** ate the last character of escaped identifiers (the committed `std.va` snapshot had baked the bug in as `logi`) (E-46).
- more than one **`default`** arm in a case statement rejected (IEEE 1364-2005 9.5): accepted in silence before. The behaviour was never wrong -- the first `default` runs -- which is why it is worth saying: the later arm is unreachable code that looks like it does something. Checked here, where the `default` TOKENS are, so the report points at the offending arm and back at the one that wins; a duplicate case ITEM is legal in Verilog and stays accepted (E-421).
- a real literal that overflows to infinity, and a based literal with a ZERO size, are rejected (E-425): `f64::from_str` returns an INFINITY rather than an error for an overflowing exponent and `StdRealNumber::value` is `parse().unwrap()`, so `1e309` compiled clean and the model returned INF -- the same mistake E-396 rejects in a data file and E-422 rejects in `abstol`. Only the LITERAL: arithmetic overflow (`1e308*10.0`) is a runtime property and stays out, as does underflow and `inf` as a range bound. Separately, `parse_based_int_masked`'s `.clamp(1, 32)` silently RAISED a zero size to one bit, so `0'd5` evaluated to 1; the upper half of that clamp is left alone because E-46 documents truncation to the 32-bit `integer` as intended (`4'hFF` is 15).

### 5. Elaboration — **`openvaf/hir/src/elaborate.rs`** (new file)

The compile-time flattening pass: every stage downstream of the parser is architected around exactly one flat module per artifact, so hierarchy is resolved by recursively inlining instantiated modules — alpha-renamed per instance, ports bound to the caller's nets, parameters bound to overrides — into an ordinary flat module *before* the rest of the pipeline runs. Built for module instantiation (E-5) and grown into the home of every structural feature:

- `generate for`/`genvar` unrolling (E-8), nested loops, anonymous blocks, `generate if`/`case` with elaboration- time constant folding, and the genvar-substitution fix (genvars in ordinary expressions were routed through identifier renaming, which re-escaped `1e3*(i+1)` into a broken escaped identifier; they now become literal-value holes) (E-67);
- implicit nets from undeclared instance connections, discipline derived from the connected port, prefixed per instance so no cross-instance shorts (E-41);
- hierarchical names `u1.u2.node` and `$root` via an instance-chain map and a hole scanner (E-49);
- **`defparam`** resolved through the same chain rewrite, with the LRM-mandated precedence over instance `#()` overrides (E-58);
- net concatenation in port connections expanded bit-by-bit onto vectored ports (E-59);
- bus slicing onto instance arrays and bus ports (E-5); paramset twin-module rendering (E-21); net initializers preserved through re-rendering (E-45); escaped identifiers re-rendered correctly (E-46);
- undefined instantiation targets are a hard error — they used to be silently dropped from the rendered output, turning a typo'd module name into an invisible open circuit; discipline-named misparses (`electrical out[0:2];` parses as an instantiation) and paramsets dropped over unresolvable targets get tailored messages (E-84);
- name-then-range net/port declarations (`input in[0:2]`, `electrical out[0:2]`, LRM 3.6/3.7): a textual pre-pass rewrites the `<head> <name>[range]` form to the range-then-name form (Enhancement-3), reusing all bus/port machinery; the `(`-after-range instance rule keeps instance arrays untouched (E-89). Extended to multi-name lists (`input a[0:1], b[0:3], c;`), split into one range-then-name declaration per name with per-name widths; a multi-dimensional name is left untouched (unsupported in both orders) (E-91);

- parameter-dependent declaration widths (`electrical [0:N-1] out;`, `real w[0:N-1];`): a textual pre-pass folds a declaration range whose bounds reference a parameter to a literal range, using the module's constant-integer parameter defaults (a fixpoint resolves one default through another; `from`/`exclude` constraints are skipped). Only declaration ranges (with a `:`) are folded, not bit-selects; the shared constant-integer evaluator gained an optional parameter map (empty map = the E-88 literal-only behavior). The width is fixed at the default -- a structural parameter, since OSDI has one node count per module (E-67/88 decision) (E-91). A parameter that shaped a width is then frozen to a **`localparam`** (`freeze_width_parameters`): a multi-parameter declaration is split so only the structural names freeze, and a range constraint is dropped from a frozen name. This keeps structure and behaviour consistent under a netlist override -- without it, overriding a width parameter that also bounds a runtime loop left the frozen array size behind, a silent out-of-bounds (E-92);
- the legacy **`generate <id> (start, end [, incr])`** statement (obsolete Verilog-A 1.0 analog-block loop-unroll, LRM Annex C.4) unrolled by a textual pre-pass: the index substitutes to a literal per iteration (with bit-select bracket folding, since a bus bit-select needs a literal index), constant bounds only -- a parameter bound cannot shape the unroll (E-67 scope decision) (E-88);
- **`` `__FILE__ ``**/ **`` `__LINE__ ``** expanded by a textual pre-pass run before the other passes (preprocessor tokens are (kind, span) pairs into existing source and cannot carry synthesized literals): `` `__FILE__ `` becomes the root file's basename (machine-portable provenance, the E-58 rule), `` `__LINE__ `` the exact 1-based line; string/comment occurrences are skipped and replacements are inline so later line numbers stay true (E-85);
- part-select actuals sliced onto bus ports in `bind_port` (`base[msb:lsb]`, constant bounds; positional and named forms; width-1 slices onto scalar ports degrade to the bit-select) with the same ascending bit-order convention as full-bus slicing (E-85);
- block-scoped parameter override rejected with a targeted message (a named instance override `#(.blk.p(4))` naming a hierarchical target is collected and bailed like the unknown-module errors; block-scoped parameters are local to their block and take their value from their initializer, which may reference the module's parameters) (E-87);
- hierarchical branch probes (E-86): the top module's absolute chain map rides into every inlined child so SIBLING bodies resolve `V(top.a1.b)`/`$root...` references; unnamed-branch forms expand to the flattened node pair; port-branch probes `I(inst.branch(<p>))` read a 0V ammeter synthesized at flattening (which also fixes child-declared `branch (<p>)` port branches, broken after inlining); hierarchy-bound monitor modules are omitted from standalone output; `ground`/net-type declarations in inlined children keep their keyword;
- **`$port_connected`** resolved at flattening time to a literal `(1)`/`(0)` per instance — after inlining, an open port is just a synthesized local net, so the builtin used to fail validation in exactly the unconnected case it exists to detect; top-level modules keep the native OSDI connected-flag path (E-84).

## 6. Definition layer — **`openvaf/hir_def/`** (+ **`openvaf/hir/`** mirrors)

`body.rs`/`body/lower.rs`/`body/pretty.rs`, `expr.rs`, `item_tree*`, `nameres*`, `builtin.rs`, `data.rs`, `types.rs`, `db.rs`, plus the new `item_tree/diagnostics.rs` and the `hir` façade (`body.rs`, `lib.rs`, `db.rs`, `diagnostics.rs`):

- statement/expression collection for every new construct: `do…while` (E-19), `repeat` and `disable` (E-9), event OR lists (E-59), concat/ replication (E-34), array aggregates and whole-array assignment (E-14), `CaseKind` + per-item don't-care masks with a stray-literal list (E-78);
- N-D array machinery: per-dimension size lists, multi-index bit-selects, per-element parameter expansion (E-15); declaration initializers on (N-D) arrays via a `Var::array_index` leaf split (E-43);
- **`localparam`** made genuinely non-overridable (it behaved exactly like `parameter`) with derived localparams still tracking inputs (E-9); paramset lowering as twin modules whose bound params

are localparams with override expressions (E-21);
- name resolution: `electrical ground gnd;` ordering (E-9), nature-attribute access (`net.potential.abstol` — the third "scaffolded-but-unwired resolver boundary" found) (E-45), derived-nature resolution (E-39);
- **`builtin.rs`** (the system-function registry): the noise-table pair (E-9), the full `$random`/`$dist_*`/`$rdist_*` family (E-10), file I/O and string functions (E-11), the last fallback group (`$simprobe`, aliases, plusargs) — after which `is_unsupported()` is empty (E-12), `$simparam$str` (E-25), `$realtime` (E-59), `$table_model` (E-16);
- new diagnostics file: wrong-arity calls produced invalid HIR and crashed downstream — now a proper diagnostic (functions and parameters both) (E-43);
- multiple `analog` blocks collect into `entry_stmts` in source order — the as-if-concatenated LRM semantics, by construction (E-60);
- bus-port node ordering (`item_tree/lower.rs`): a non-ANSI header bus port (`module m(in, y); input [0:2] in;`) had its first bit merged into the header placeholder but the remaining bits appended after later ports, scrambling OSDI terminal order whenever the bus was not the last port. A pre-scan of body port widths now expands the bus into contiguous bits in header order at placeholder-creation time (E-90).
- **`exclude`** constraints validated, as **`from`** already was (E-421): E-399 rejects a `from` range no value can satisfy and never looked at `exclude`, so the identical defect spelled the other way went through. Two shapes, both silent: an `exclude` that covers the whole `from` range leaves the parameter UNSETTABLE (every netlist value aborts with "Parameter x is out of bounds!", only the default reads -- exactly what E-399 reports for `from [3:1]`), and an INVERTED `exclude [3:1]` excludes NOTHING, so every value the declaration appears to forbid is accepted. A small interval sweep over literal bounds decides the cover; endpoint inclusivity is the whole difficulty, since `from [0:10] exclude (0:10)` leaves exactly the two endpoints settable and must still compile.

## 7. Types and validation — **`openvaf/hir_ty/`**

`inference.rs` (+ `inference/fmt_parser.rs`), `builtin.rs` + `builtin/generated.rs` (signature tables), `lower.rs`, `types.rs`, `validation.rs`, `validation/body.rs`, `diagnostics.rs`:

- signature tables — a recurring defect source: `ANALYSIS` made variadic (E-30); `$table_model` given shape-synthesized varargs signatures for any dimension (a generic-varargs fallthrough *truncated* multi-arity signature lists — varargs builtins with signature lists need their own inference arm) (E-40); `TRANSITION`'s table was one argument short (3-argument calls crashed, 4-argument worked by accident) (E-47); `transition()` input typed Real, a `$dist` typo fixed (E-49); `SIMPARAM_STR` returned Real instead of String (E-25);
- format-string inference (`fmt_parser.rs`): terminates on every conversion character and reports it, so `%5d`/`%-8s`/`%08x` type their arguments correctly — previously only real conversions accepted flags/width (E-71);
- array inference: element-wise array `case`, array-literal function arguments (which previously bound nothing and silently returned 0), integer arrays typed Real, output-literal args accepted silently — all fixed (E-33); whole-array function arguments/outputs/returns (E-18, E-20, E-23); untyped function args default to Real instead of an ICE masquerading as an initializer bug (E-43);
- **`Type::base_type`** infinite loop on any array-type check (it looped on `self` instead of the cursor) — hung the compiler on simple type mismatches (E-14);
- string handling: ternary over strings (SELECT + string binary ops appended to the operator tables) (E-37);
- validation: loop bodies get their own context so the analog-operator restriction reports "loops" (LRM 4.5.1), not "conditions" (E-70); call-graph cycles among analog functions are clean errors naming the cycle instead of a recursive-inliner stack overflow (E-59); casex/casez restrictions (stray don't-care literals, `x` digits under `casez`, non-integer discriminants) (E-78); `domain discrete` with natures rejected per LRM 3.6.2.2 (E-50); parameter defaults exempt from their own

ranges (the CMC "feature disabled" idiom — stock rejected diode_cmc, BSIM-CMG, PSP-HV and the HiSIM family at setup) while given values stay fully validated (E-56);
- part-selects outside port connections diagnosed (the E-78 stray-list pattern: body lowering collects them, validation reports a dedicated error naming the supported connection form) (E-85);
- contributions to port branches diagnosed (`ContributeToPortFlow`, with the declaration site labeled): `I(pb) <+ …` on a `branch (<p>) pb;` slipped through the write path unvalidated and panicked during lowering (E-84);
- contributions to an all-**`ground`** branch diagnosed (`ContributeToGround`): `V(gnd) <+ …` / `V(gnd, gnd) <+ …` reduce both endpoints to the fixed 0 reference (no unknown) and hit an `unreachable!()` in `lower_contribute_unnamed_branch` — an ICE. The write path now checks the branch's `is_gnd` nodes and reports a clean error; a real node-to-ground branch and a `V(gnd)` probe are untouched (E-97).
- `**` typed by its operands, not unconditionally real: `Power` was the only arithmetic operator pinned to `REAL_BIN_OP`, so two integer operands were promoted to float. IEEE 1364-2005 §5.1.5 makes the result real only if either operand is real; it now takes the two-candidate `NUMERIC_BIN_OP` like `+ - * / %`, and the lowering grew the matching integer arm — a signature change alone is a wrong-code bug, the trap E-376 recorded (E-420).
- four more constant arguments that were accepted and then degenerate, continuing the E-396/E-399/E-405 line and folding literals only: an identically-zero `laplace_*`/`zi_*` coefficient denominator (the transfer function is num/0, which failed the operating point with a "timestep too small" naming neither model nor call — an all-zero *root* list means poles at the origin and is untouched); a zero or negative `zi_*` sampling period (the filter returned its input unchanged); a `last_crossing` direction outside $\{-1, 0, +1\}$ (silently read as 0); a `$discontinuity` degree below $-1$ ($-1$ itself is the LRM's limiting marker and stays legal). `ac_stim` also joined `analysis()` on the existing `unknown_analysis_name` lint (L021), with a note of its own — an unmatchable name leaves the stimulus permanently inactive rather than producing a dead branch (E-420).
- **`$simparam`**/**`$simparam$str`** names checked (new `unknown_simparam` lint, L025): the third of a family and the only one left unchecked, and the only one whose bad name is FATAL -- `analysis()` (L021), `$limit` (L020) and `ac_stim` (L021) all warn and then degrade benignly, while an unresolvable `$simparam` sets `EVAL_RET_FLAG_FATAL` and aborts the analysis. The name set is ngspice's (`osdiload.c`), not the LRM's: the LRM's `minr`/`imelt`/`shrink`/`imax`/`rthresh` are served by nothing here, and for `$simparam$str` the two sets do not intersect at all, so an LRM-derived check would have warned on the names that work. The non-fatal `$simparam(name, default)` form is deliberately not warned and is what the diagnostic points at (E-421).
- every **`nature`**/**`discipline`** reference to another nature is now resolved and reported (E-422): `parent`, `ddt_nature`, `idt_nature`, `potential` and `flow` are the same reference resolved by the same function, and a name that failed to resolve produced THREE different outcomes -- `parent` crashed the compiler, `ddt_nature`/`idt_nature` were silently discarded, and a discipline's `potential`/`flow` was reported much later against the MODEL BODY. `verify_nature`/`verify_discipline` resolve all five at the declaration now. Also here: a nature inheritance CYCLE ( silently broken by salsa's recovery, leaving the nature as its own base nature and so changing discipline compatibility), and `abstol`, unvalidated both as a value (zero, negative, a literal overflowing to infinity) and as an expression (anything that did not fold to a real was silently DISCARDED, leaving no abstol at all).
- a noise or **`ac_stim`** source inside a RUN-TIME LOOP is now reported (E-424): it used to register NO SOURCE AT ALL and contribute exactly nothing -- `onoise_total` bit-identical to a model with no noise, `ac_stim` from mag 500 to 0 -- silent even under `-E all`. Every other member of the family (`ddt`, `idt`, `absdelay`, `transition`, `laplace_*`) was ALREADY rejected in that position (LRM 4.5.1); the noise builtins are not in `is_analog_operator()` so they fell past the check and were discarded further down. A new `is_small_signal_source()` predicate keeps the restriction LOOP-ONLY -- noise inside an `if`/`case` is legitimate and still works -- and `generate` still unrolls to one source per iteration. Also here: `$finish`/`$stop` accepted any diagnostic level, where IEEE 1364-2005 17.1.2 defines only 0, 1 and 2.

- table data files are checked against the grammar the READERS use (E-425): the only shape check used to be global token-count PARITY with non-numeric tokens silently discarded, so a corrupt row that dropped TWO tokens passed and one that dropped ONE was caught -- pure luck. `table_file_is_usable` now takes the CALL'S dimensionality (carried in the diagnostic, computed exactly as `lower_table_model` computes it) and applies the matching grammar: one pair per line for `ndim == 1` and every `noise_table` file, an exact self-describing shape for `ndim >= 2`. The dimensionality had to come from the call because `2 3 / 4 5 / 6 7` is a valid 1-D table whose leading numbers also read as a 2-D header.

## 8. Lowering — **`openvaf/hir_lower/`**

`expr.rs`, `stmt.rs`, `ctx.rs`, `callbacks.rs`, `parameters.rs`, `state.rs`, `fmt.rs`, `lib.rs`:

- integer `**` per IEEE 1364-2005 Table 5-6 (`lower_int_pow`): with two integer operands `2 ** -1` computed 0.5 in floating point and rounded *away from zero* to 1, where the standard says 0. Written branchless, and the exponent handed to the float `pow` is clamped non-negative — `pow(0, -1)` is infinity and `llvm.lround` of an infinity into an i32 is undefined, which is what made `0 ** -1` return 2147483647 (E-420).
- analog operators: `laplace_nd/np/zd/zp` via exact state-space realization (E-4) with complex pole/zero pairs per the LRM's (re, im) convention (E-31); `zi_*` via bilinear transform (E-6); `slew`/`transition` as a saturating tracking loop (E-6) with the LRM negative `max_neg_rate` convention honored (the sign defect made `slew` ignore its input and ramp away) (E-61) and a DC-singularity clamp via an `EnableIntegration` identity (E-47); `idtmod`'s modulo wrap moved off the reactive residual (it diverged) and an argument- index bug fixed (E-27); `idt` initial conditions survive into transient (E-28); the `idt` assert/reset forms with smooth reset dynamics (E-52); `limexp` kept stateless as a documented decision (E-13);
- **`$table_model`**: 1-D piecewise-linear lowered to differentiable MIR so autodiff yields the Jacobian for free (E-16); N-D multilinear as recursive 1-D (E-17, E-40); natural cubic splines with the precomputed moment matrix (E-22);
- events: `cross`/`above`/`timer` with persistent previous-value slots (E-8); OR lists folded with a select-based bool-or (a raw `ior` ICEs const-eval) (E-59); final-step gating and phase-list AND-ing (E-53);
- variable persistence: genuine per-instance `hidden_state` storage behind a two-pass MIR build (E-7), typed slots so integer state stopped crashing instruction selection (E-32);
- callbacks (`callbacks.rs`, `fmt.rs`): the `$display` family with the full `[flags][width][.precision]` surface and `%h`/`%b`/`%r` translation (E-71); file I/O and string formatting through a generalized `PrintDst` sink (E-11); the deterministic seed-and-salt RNG lowering (E-10); `$simparam$str` (E-25); simulation-control return flags (E-55); `$discontinuity` (E-24);
- operator fixes: unary ~ emitted arithmetic negate; the constant folder sign-extended >> where the runtime was unsigned — const and runtime disagreed (E-37);
- whole-array coercion (`expr.rs`, `stmt.rs`): inference records a coercion of a whole array as a cast on the array *expression*, but `lower_array_elems_impl` decomposes the array and lowers each element itself, so `lower_expr`'s `needs_cast()` never saw it — the cast was silently dead, and every new array-consuming context re-inherited the same crash (an integer element reaching a float MIR op, which const-eval has no case for: *"invalid operation fdiv Int(1) Float(..)"*). Patched per-call-site four times — the `case` discriminant (E-33), `laplace_*`/`zi_*` integer-literal then integer array-variable coefficients, and an integer `case` item against a real discriminant — before being fixed at the chokepoint, which now honours the recorded cast for every consumer (E-214). The explicit `coerce_real` flag remains for consumers whose element type is fixed by the language rather than by an inferred cast (a coefficient vector is real per LRM 9.19, and inference records no cast for it);
- masked casex/casez comparisons (two `iands` ahead of the existing integer equality) (E-78); array function-argument writeback (E-20) and array returns via a function-named var-array (E-23);

- named port branches (`branch (<p>) pb;` — LRM 3.7.2): a flow probe of a PortFlow-kind branch routes through the same `CurrentKind::Port` param as a direct `I(<p>)`, so E-29's defining equation covers both spellings; probing one used to hit `unreachable!()` (E-84);
- literal **`if`** conditions lower only the taken branch — a dead analog operator (`transition()` under `if ((0))`, the exact shape the `$port_connected` rewrite produces) survived const-folding as a detached-but-interned op whose state setup read optimized-away values and aborted codegen (E-84);
- **`case`** fall-through block left unsealed: `max`/`min`/`abs` lower through `make_cond` to real control flow, so one in a `case` default arm moved the builder onto its own merge block — the seal at the end of the case landed there and the case's own fall-through block was never sealed ("block N is not sealed"). It is now sealed where it is created; the branch just emitted is its only predecessor (E-291).

### 9. MIR core and optimization — **`openvaf/mir*`**

`mir/` (`instructions.rs` + `instructions/generated.rs`, `dfg.rs` with `dfg/phis.rs` and `dfg/uses.rs`, `dominators.rs`, `flowgraph.rs` with `flowgraph/transversal.rs`, `layout.rs`, `cursor.rs`, `lib.rs`, `builder/generated.rs`), `mir_opt/` (`simplify_cfg.rs`, `simplify.rs`, `const_eval.rs`, `global_value_numbering.rs`), `mir_autodiff/` (`builder.rs`, `lib.rs`, `live_derivatives.rs`), `mir_llvm/` (`builder.rs`, `intrinsics.rs`), `mir_build/`, `mir_interpret/`:

- three pre-existing general CFG bugs, all found via event-counter verification: a dangling-reference bug in unreachable-block removal, a multi-exit post-dominance bug in the dominator-tree builder, and a block-merge bug that could corrupt which block the DAE builder treated as the exit (E-8);
- the post-dominator sink-root pathology that let dead-code elimination delete op-dependent `$fatal` calls (side-effecting callbacks under op-dependent control now stay in eval; control dependence computed by arm-reachability) (E-55);
- `split_tainted.rs` tolerates layout-detached branch instructions (a branch whose condition const-folded has no CFG effect to taint; it used to unwrap and panic) (E-84);
- const-eval agreement with runtime semantics for shifts and the bitwise-not fix (E-37);
- autodiff: noise operators process before any `ddt` (a shared ddt evaluated between two noise operators silently dropped the second source), and the reactive coupling of a `ddt`-shaped noise wave is registered so small-signal pruning keeps it (E-54);
- **`llvm.fabs.f64`** registered in the LLVM intrinsics table — it never had been, and the signed noise-power convention needed it (E-42);
- new opcodes/instructions supporting the operator work (barrier/ integration-identity forms; MIR deliberately has no `fabs`, so lowering composes it) (E-47, E-52);
- **`const_eval.rs`** — folding must match codegen or decline: `eval_binary` evaluated `5/0`, `5%0` and `i32::MIN/-1` *inside the compiler*, so a literal zero divisor killed it outright while a runtime one had always been accepted. It now returns `Option`, declines the undefined cases and out-of-range shifts, and folds `+`/`-`/`*` with wrapping arithmetic to match the emitted code (E-286);
- **`simplify_cfg.rs`** — a const-folded branch did not flag its change, so the sweep that collects orphaned blocks never re-ran and a phi kept an edge naming a value reachable only through the deleted edge — a broken-SSA function the release build carried forward, the MIR verifier being a `debug_assert!` (E-287);
- **`mir_llvm/intrinsics.rs`** — two declarations disagreed with the calls emitted: `hypot` was declared with one parameter and called with two (E-288), and the overloaded `llvm.ctlz` was registered without its `.i32` type suffix — it backs `$clog2` (E-289). Both produced modules the LLVM verifier rejects, and both were invisible in release for the same reason.
- branch-to-jump rewrites must retire the condition's use: a `Branch` carries one value operand, a `Jump` none, so overwriting the instruction in place leaves the condition's use record naming an operand that no longer exists. `simplify_bb`'s empty-exit-block rewrite and `dead_code_aggressive`'s dead-block rewrite both did; the two in `const_fold_terminator` already zapped/detached first (E-294).

10. DAE construction — **`openvaf/sim_back/`**

`dae.rs` + `dae/builder.rs`, `topology*` (incl. `lineralize.rs`, `small_signal_network.rs`), `noise.rs`, `context.rs`, `lib.rs`:

- implicit equations for indirect branch assignment (one new unknown
  - residual per statement) (E-2) and the absdelay synthetic two-node form (E-1) — with equation indices kept stable by mapping dead/collapsed unknowns to zero-contribution placeholders instead of renumbering;
- port-flow probes **`I(<port>)`**: a TODO stub returning 0 became a synthesized DAE unknown mirroring the node's KCL residual (E-29);
- probe-only branches: probing a never-contributed branch read 0 and an open circuit — a 0 V-source pass materializes them, enabling ideal ammeters and flow-only signal-flow disciplines (E-36);
- noise factors (`noise.rs`): equation-path noise was silently lost (never attached to the DAE); factors generalize to `re + jω·im` with no extra matrix unknowns (E-54); same-named source grouping metadata (E-42);
- `idt` reset-mode charge dynamics and conditional step bounding (E-52).
- voltage-source branches feeding internal nodes were open circuits at DC: the small-signal (noise/`ac_stim`) pruner keyed node registration on the branch's LIVE voltage unknown, so a pure `V(a, f) <+ expr` (V(a,f) never read) registered nothing and the node's conduction silently moved to the AC-only residual; any voltage-capable branch now disqualifies its nodes (E-86);
- probed **`V(x,y) <+ 0`** branches were node-collapsed away, making `I(branch)` read zero; hint pairs whose branch current is a DAE unknown are suppressed (the unprobed collapse idiom is untouched) and `NodeCollapse::hint` tolerates suppressed pairs; pinned by the permanent `vsrc_internal_node` snapshot test. Behavior change: a collapsible branch whose current the model references (MVSG_CMC's access resistances carry noise on `flow(d,drc)`) stays a real 0V source — electrically identical, two extra matrix rows (E-86);
- **`lineralize.rs`** — one analog operator nested directly inside another (`ddt(ddt(x))`): an operator materialized as an implicit equation deletes its own instruction, but a later linear contribution holds its dimension values *outside* the data-flow graph where `replace_uses` cannot reach them — and with direct nesting that stored dimension IS the deleted result. Pending entries are now retargeted onto the implicit unknown the inner operator became, which is also the correct second-derivative formulation (E-293);
- **`small_signal_network.rs`** — pruning is best-effort: the linearity classifier and the dimension replay can disagree, and the pass then indexed a `val_map` key that was never inserted ("no entry found for key"). It now gives up on that value — resolving its placeholder so no invalid value survives — instead of crashing (E-292).
- terminal shorts (E-401): `V(a,b) <+ 0` lowers to a node-COLLAPSE request, and a simulator cannot honour one whose endpoints are both terminals -- it does not own those unknowns. The potential arm of `build_branch` built no equation for a trivial contribution either, so the branch connected nothing: a silent open circuit where the model wrote a short (the LRM's `parares` degenerated to an open instead of a wire). `collapse_is_ignored` now detects that case -- every non-ground endpoint is a port -- and the branch gets a real 0 V source, in the constant-potential arm AND in the switch arm, which is the one `parares` takes (its condition is parameter-dependent, so `is_voltage_src` is a runtime value while `op_dependent` is false, and the branch had been dismissed as "just node collapsing"). Terminal-to-INTERNAL collapse is deliberately untouched: it works, it is exact, and it is what every real compact model uses. The branches are recorded

in `Builder::terminal_shorts` and handed out of `DaeSystem::new` through an out-parameter rather than stored on `DaeSystem`, whose `Debug` output is a checked-in snapshot in several tests.

- **`diagnostics.rs`** (new) — the DAE build got a diagnostic channel: a branch contributed as BOTH a potential and a flow source keeps only the last contribution and silently discards the other, and nothing said so. `build_branch` now reports it (`discarded_contribution`, L022) through the same `ConsoleSink` `collect_modules` already uses, threaded `openvaf::compile` → `osdi::compile` → `CompiledModule::new` → `DaeSystem::new`. The evidence is split across two stages: the DAE build knows the branch is not a switch (a real switch branch has a runtime `is_voltage_src`), while the HIR still holds the discarded statement -- `hir_lower`'s `contribute_value` zeroes the opposite place as it writes, so `BranchInfo` reports a discarded contribution as trivially zero, exactly like one never written. `context.rs` records `unconditional_branch_kind` from the UNOPTIMIZED MIR, because constant propagation can also make `is_voltage_src` constant by folding a configuration constant, and the surviving arm of a correct switch branch is not a discarded contribution (E-400).

## 11. OSDI code generation — **`openvaf/osdi/`**

`lib.rs`, `metadata.rs` + `metadata/osdi_0_4.rs`, `inst_data.rs`, `eval.rs`, `load.rs`, `setup.rs`, `compilation_unit.rs`, `ndatable.rs`, `stdlib.c`, `build.rs`, reference headers:

- descriptor emission for every ABI evolution (see the ngspice report's ABI table): the absdelay/last_crossing info arrays (E-1, E-6), `OsdiNode.nodeset` (E-45), the `ac_stim` source array + `load_ac_stim` (E-51), stride-2 signed noise pairs in `load_noise` (E-54) — the version tuple lives in `osdi/src/lib.rs`;
- **`metadata.rs`**: the `PARA_FLAG_FIXED` parameter-descriptor flag (a free bit of the `flags` field, additive — no layout/ABI change) set on every `localparam`, so the simulator can warn when a netlist tries to set a non-settable parameter (a `localparam`, or a structural width parameter frozen by Enhancement-92) (E-93);
- **`eval.rs`**: final-step flag gating (E-53); simulation-control return flags (E-55); the `ac_stim` large-signal value (was an `unreachable!()` panic) (E-26);
- **`inst_data.rs`**: absdelay slot layout (E-1); hidden-state slots typed by the variable (`ty_f64` was hardcoded — integer state fed f64 loads into integer MIR ops and aborted instruction selection) (E-32);
- **`compilation_unit.rs`** (print codegen): the `%b` segfault — the binary-formatted string was remembered for `free()` but never pushed to the `snprintf` argument list, so the matching `%s` consumed a garbage pointer (E-71); the print-callback machinery with its shadowed-`fun` and volatile-table IPO traps (E-11);
- **`stdlib.c`**: the file-descriptor table behind `$fopen`/`$fdisplay`/... (E-11); the splitmix64 `osdi_rng_*` runtime (E-10); a `simparam_str` loop/return bug (E-25);
- **`ndatable.rs`**: `ddt`/`idt` natures resolved through `resolve_nature_index` instead of panicking on derived natures (E-39);
- **`setup.rs`**: range validation with default-exemption (E-56); a 64-line abandoned unsafe experiment (`VoidAbortCallback`) deleted in the warning cleanup (E-66);
- **`build.rs`**: the copied-target stdlib compilation trap fixed (E-10);
- **`inst_data.rs`**: **`$temperature`** read as an operator ARGUMENT used the wrong struct-GEP type — the field type (`double`) instead of the instance-data struct — so LLVM computed the offset as a flat `5*sizeof(double)` rather than `offsetof(instance, temperature)`. The shipped compiler died with SIGSEGV on `ac_stim("ac", $temperature, 0)`; the same wrong type was on the operating-point-variable read path (`nth_opvar_ptr`) (E-290).
- **`resolve_nature_ref`** no longer panics on an unresolved name (E-422): three `.unwrap()`s on the nature/discipline name maps turned a one-character typo in a `parent` nature into a compiler crash report -- whether or not the nature was ever used, so one bad declaration in an included header killed every model that included it. The names are diagnosed in `hir_ty` now, so a bad reference

never arrives; this returns `NATREF_NONE` if one ever does again. The tell was five lines below the panic: `resolve_nature_index`, added by E-39 for the `ddt_nature`/`idt_nature` references, was already written as `unwrap_or(u32::MAX)` -- one side hardened, the other left to crash.

## 12. Driver, libraries, and infrastructure

- `openvaf-driver/src/main.rs`: hard errors now exit non-zero — the error arm printed the failure but fell through to a success exit, so every elaboration failure looked like a successful compile to shell scripts (a quirk first noticed during E-58's work) (E-84);
- `openvaf-driver/src/crash_report.rs`, `linker/src/lib.rs`, `lib/bforest/` (`map.rs`, `pool.rs`, `set.rs`), `lib/typed_indexmap/` (`set.rs`): the zero-warning cleanup (44 → 0 across 13 crates: lifetime annotations, dead code, nightly cfgs, an unused LLVM-prefix probe) (E-66);
- `basedb/` (`ast_id_map.rs`, `lints.rs`, `lib.rs`, `diagnostics/preprocessor_error.rs`, `diagnostics/syntax_error.rs`): AST-id plumbing for elaboration-created items (the `AstId::from_erased` placeholder used by array returns; E-23), preprocessor/syntax diagnostic rendering for the new errors;
- `.llvm-version` pins the LLVM 18 toolchain expectation; the `hir` and `preprocessor` `Cargo.toml`s carry the dependency additions their new code needed.

## 13. The test suite — **`openvaf/openvaf/tests/`**, **`lib/mini_harness/`**, **`test_data/`**, **`integration_tests/`**

The fork's own integration suite over real compact models had never run in this project: its OSDI loader was frozen at ABI 0.4, its optional VACASK legs panicked on a never-initialized submodule, and the tests hide behind `RUN_DEV_TESTS=1`. Fixed test-side only (E-68):

- `tests/load/osdi_0_4.rs` + `load/mod.rs`: loader structs synced to ABI 0.7; `mock_sim/mod.rs`: stride-2 noise convention; `mini_harness`: missing directories skip with a note instead of panicking;
- VACASK removed entirely (`.gitmodules`, the submodule, 34 harness legs and their stale snapshots): AGPL-3.0 cannot be vendored here;
- `test_data/` (95 files as a class): new fixtures for new features and snapshots regenerated whenever descriptor contents legitimately changed — every regeneration reviewed (the diffs are the project's own features: `flow(<port>)` unknowns, probe-only branches, implicit-equation nodes); `integration_tests/*/*.osdi` regenerated at each ABI bump.

---

- Two correctness blind spots closed with mutation-tested guards (E-295, verification-only): the full 4x4 multi-terminal conductance AND capacitance matrices (the autodiff suite was 2-terminal plus one off-diagonal, so the KCL-derived source row and the zero body row were untested), and per-instance parameter-slot readback across 13 interleaved model/instance parameters (the guard for the E-290 offset class) (E-295).

## 14. Per-enhancement index (compiler-touching enhancements)

E-1 *sim_back, osdi*

`absdelay()` synthetic-node DAE + descriptor arrays

E-2 *parser→hir_lower, sim_back*

indirect branch assignment (implicit equations)

E-3 *parser, hir_def*

vectored (bus) nets with bit-selects

E-4 *hir_lower, syntax, parser*

`laplace_*` state-space + array-literal parsing

E-5 *elaborate.rs (new), parser*

module instantiation via compile-time flattening

E-6 *preprocessor, parser, hir_lower, osdi*

directives, shifts, slew/transition, zi_*, last_crossing

E-7 *hir_lower (state), osdi*

`@(initial_step)` gating + variable persistence

E-8 *parser, elaborate, hir_lower, mir/mir_opt*

generate for + events; three general CFG bug fixes

E-9 *hir_def, hir_lower, sim_back*

noise tables; localparam/ground/strings; repeat/disable

E-10 *hir_lower, osdi stdlib*

`\$random`/`\$dist_*` deterministic RNG

E-11 *hir_lower, osdi*

file I/O + string functions (PrintDst)

E-12 *hir_def, hir_lower*

last unsupported builtins as LRM fallbacks

E-13 *hir_lower*

`limexp` kept stateless (documented decision)

E-14 *hir_def, hir_ty, hir_lower*

array literals/params/dynamic indexing; base_type hang

E-15 *hir_def, hir_ty, hir_lower*

N-D arrays

E-16/17/22/40 *hir_ty, hir_lower*

`\$table_model`: 1-D, N-D multilinear, cubic spline, varargs sigs

E-18/20/23/33 *hir_ty, hir_lower, basedb*

arrays in analog functions: in/out/return/literals + array case

E-19 *tokens→hir_lower*

`do … while`

E-21/44 *parser, elaborate, hir_def, sim_back*

paramset + hidden system parameters

E-24/55 *hir_lower, mir, osdi*

`\$discontinuity`; `\$finish`/`\$stop`/`\$fatal` honored

E-25 *hir_ty, osdi stdlib*

`\$simparam\$str`

E-26/51 *osdi, sim_back*

`ac_stim`: crash fix, then full AC-RHS injection

E-27/28/52 *hir_lower, sim_back*

the `idt` family: modulo, IC, assert/reset

E-29/36 *sim_back*

port-flow probes; probe-only branches

E-30 *hir_ty, hir_lower*

variadic `analysis()`

E-31 *hir_lower*

complex poles/zeros in root forms

E-32 *osdi inst_data*

integer persistent state (crash fix)

E-34 *tokens→hir_lower*

`{…}` concat + `{n{…}}` replication

E-35 *lexer*

EOF comment hang

E-37/38 *hir_lower, mir_opt, parser*

operator + precedence audits (~, >> fold, % level)

E-39 *parser, hir_ty, osdi*

derived natures unwired at the node-kind boundary

E-41/49/58/59 *elaborate*

implicit nets; hierarchical names; defparam; port concat + OR lists

E-42/54 *sim_back noise, mir_autodiff, mir_llvm, osdi*

correlated + node-free complex noise factors

E-43 *hir_def (+diagnostics), hir_ty*

initializers completed; arity diagnostics

E-45 *osdi, hir_def, elaborate*

nodesets + nature-attribute access (ABI 0.5)

E-46/48 *lexer, syntax*

escaped ids, based literals, single-pass unescaper

E-47 *preprocessor, hir_ty, hir_lower*

`` `default_transition `` + transition fixes

E-50 *hir_ty validation*

domain-binding validation

E-53 *hir_lower, osdi eval*

`@(final_step)` + phase lists

E-56 *hir_ty, osdi setup*

parameter defaults exempt from ranges

E-59 *tokens→hir_lower, hir_ty*

`\$realtime`; OR lists; recursion diagnostics

E-61 *hir_lower*

`slew` sign fix (operator-args audit)

E-65 *preprocessor*

macro-recursion guard

E-66 *13 crates*

zero-warning cleanup (44 → 0)

E-67 *tokens, parser, syntax, elaborate*

generate audit: genvar fix, nesting, if/case

E-68 *tests, mini_harness*

integration suite enabled; VACASK removed

E-70 *hir_ty validation*

loop-context diagnostics

E-71 *hir_ty fmt, hir_lower fmt, osdi*

display-format surface + `%b` segfault

E-78 *lexer→hir_lower, hir_ty*

`casex`/`casez` don't-care masks

E-84 *parser, elaborate, hir_ty, hir_lower, mir_opt, driver*

LRM example sweep: 6 defect fixes (port-branch panic, parser robustness, silent undefined modules, `\$port_connected` on open ports, dead-op codegen, exit codes)

E-85 *parser, elaborate, hir_def, hir_ty*

`` `__FILE__ ``/ `` `__LINE__ `` + connection part-selects (the last two sweep findings)

E-86 *parser, elaborate, sim_back*

hierarchical branch probes + 2 DAE fixes (V-source-to-internal open circuit, collapse-of-probed-branch)

E-87 *parser, elaborate*

block-scoped parameters (validated) + clean diagnostic for the illegal `#(.blk.p())` override

E-88 *elaborate*

legacy `generate <id> (start,end)` analog-block loop-unroll (textual pre-pass)

E-89 *elaborate*

name-then-range net/port decls (textual normalize) + Annex E SPICE-primitives example library

E-90 *hir_def (item_tree)*

multi-bit input bus port bit reads: pre-scan body widths so a non-last bus port's bits stay contiguous in header/terminal order

E-91 *elaborate*

multi-name name-then-range declarations + parameter-dependent declaration widths (folded from the parameter default)

E-92 *elaborate*

freeze structural (width) parameters to `localparam` (split multi-parameter decls) so a netlist override cannot desync the frozen width from behaviour

E-93 *osdi (metadata)*

flag `localparam` OSDI parameters non-settable (`PARA_FLAG_FIXED`, additive) so ngspice can warn on a netlist override (ngspice side too)

E-96 *parser (module grammar)*

parse a module-level `generate for/if/case` without the optional `generate`/`endgenerate` keywords

E-97 *hir_ty (validation)*

clean diagnostic (was an ICE) for a contribution to an all-`ground` branch (`V(gnd) <+ …`)

E-101 *hir_ty (builtin), mir_opt / mir_interpret / mir_llvm*

`\$clog2`: accept one argument (`INT_MATH_1`, was a 2-arg signature) and compute `ceil(log2 n)` = `bit_width(n-1)` in all three backends (was `floor(log2 n)+1`, wrong on exact powers of two)

E-102 *parser (items), syntax (ungrammar + ast), hir_def (item_tree)*

name-then-range array-valued parameters (`parameter real c[0:2]`); `parameter()` accepts post-name `[range]`, `lower_param` resolves dims per name

E-103 *mir_llvm (intrinsics)*

register the `llvm.ceil.f64` intrinsic (was missing; `ceil()` of a non-constant argument crashed codegen -- `floor` was registered)

E-104 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr)*

add `\$rtoi` (real->int, truncate toward zero via `ficast((x<0)?ceil:floor)`) and `\$itor` (int->real) conversion builtins

E-105 *hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)*

`\$sscanf`/`\$fscanf` honour the format conversion base -- parse the format string, dispatch integer fields to base-specific scanners (`%h`/`%x` hex, `%o` octal, `%b` binary)

E-106 *hir_ty (types, inference), hir (signatures), hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)*

string relational comparison (</<=/>/>=) via a lexicographic `osdi_strcmp` callback (`RELATIONAL_COMPARISON` adds the STR signature; `a<op>b` lowers to `strcmp(a,b)<op>0`)

E-107 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)*

add `\$fgetc(fd)` single-character read as a `FileOp::Getc` -> `osdi_fgetc` (completes the file I/O family)

E-108 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)*

add `\$ungetc(c, fd)` one-character pushback as a 2-arg `FileOp::Ungetc` -> `osdi_ungetc` ( companion to `\$fgetc`)

E-109 *hir_lower (callbacks), osdi (load)*

correct `noise_table` (piecewise-linear in `f`) and `noise_table_log` (log-log, `P=10^lerp(log10 p, log10 f)`) interpolation to LRM 4.6.4.3/4.6.4.4 -- both were nonconformant (lin-log, and a mis-keyed log-freq input)

E-147 *hir_ty (validation/body)*

fix exponential-time compilation of nested `?:`. Found by a robustness campaign (117 production CMC models + ~50 adversarial inputs + 4000 mutation-fuzz iterations). The body validator's `validate_expr` ends by recursing into children via `walk_child_exprs`; arms that validate their own operands `return` first (`Call`, `Path`), but the `Select` (ternary) arm did NOT -- it validated `cond/then_val/else_val` via `validate_condition` and then fell through, validating `then_val/else_val` a SECOND time, so a chain of N nested `?:` was validated 2^N times. Depth ~30 ( reachable via macros) hung the compiler; `sample`-profiling confirmed an unbounded `validate_expr`↔`validate_condition` recursion with doubling call count. Fix: add `return;` to the `Select` arm (the operands are already fully validated). O(2^N)→O(N): depth 40 hang→0.09 s, depth 2000→1.6 s. Behaviour-preserving: all 117 models give the IDENTICAL verdict before/ after (0 flips, head-to-head), BSIM4 (12.6k lines) still ~2.3 s, and hir_ty/hir/hir_lower/sim_back tests pass with no snapshot changes. Campaign also confirmed 0 panics/segfaults across 4000 fuzz iterations; remaining lower-severity findings (parser stack overflow on ~4k-32k-deep nesting, `include` self-recursion, huge array dimension) are documented follow-ups. Verified 7/7 (nested_ cond_examples: depth 20/40/80/160 all <0.2 s, linear growth, correct piecewise value in ngspice)

E-148 *parser (parser, grammar/expressions), preprocessor (diagnostics, processor), basedb (diagnostics/ preprocessor_error), hir_def (item_tree, item_tree/diagnostics, item_tree/lower), hir (elaborate)*

compiler hardening: closes the three lower-severity robustness findings from E-147 (pathological input → crash/hang) by turning each into a clean bounded diagnostic. (1) Parser expression-depth limit: a shared `Parser::expr_depth` counter, incremented on each `atom_expr` (recursion) and per operator in the `expr_bp` loop (operator-chain / tree depth), bounds total expression-tree depth at 1000 and recovers to the next statement boundary when exceeded — so `----…x` / `x+1+1+…` / `((…))` / `sin(sin(…))` / `1?…:0` no longer overflow the recursive-descent parser OR a later recursive tree traversal (`expr_bp` split into a save/restore wrapper + `expr_bp_inner`; `atom_expr` into an inc/check/dec wrapper + `atom_expr_inner`; reuses the existing `err_recover`/`unexpected_tokens_msg` recovery). (2) **`` `include ``** recursion cap: an `include_depth` counter on the preprocessor's `Processor` caps nesting at 64, emitting a new `PreprocessorDiagnostic::IncludeRecursionLimit` (rendered in basedb) instead of overflowing the stack on a self-including file — mirrors the E-65 macro-recursion guard. (3) Array element cap: a shared `array_elem_count` helper (product of `|msb−lsb|+1`, capped at 2^20 ≈ 1.05M) + a new `ItemTreeDiagnostic::ArrayTooLarge`, applied at EVERY expansion site — variable arrays, parameter arrays, net/port buses, array function returns (all in `hir_def` item-tree `lower.rs`), and instance arrays in BOTH the item-tree lowering and the `hir` elaboration pass (which re-expands `dev s[0:N]()` into rendered text) — so `real x[0:100000000]` (and param/bus/ instance equivalents) degrade to a single scalar with an error instead of exhausting memory. Behaviour-preserving: all 117 production CMC models give the IDENTICAL verdict before/ after (0 flips, head-to-head), BSIM4 still ~2.3s; parser/preprocessor/hir_def/hir_ty/sim_back unit

tests pass with no snapshot changes; 0 build warnings. Verified 17/17 (robustness_examples: 5 deep-expression shapes + self-include + 4 huge-array kinds all error in <1s no crash/hang with expected diagnostics; valid nested-ternary-30 / parens-100 / 100-term-sum / small arrays still compile). Fully resolves the E-147 campaign's remaining findings

E-185 *mir_autodiff (builder.rs)*

autodiff audit: hypot & atan2 derivative fixes. A deep audit compared the AC small-signal conductance dI/dV (built by the mir_autodiff automatic differentiation that feeds AC/convergence/noise/pz) of a battery of nonlinear laws I=f(V) against the ANALYTIC f'(V), and caught two builtins right in VALUE (DC correct) but wrong in DERIVATIVE -- the accidental-correctness pattern, first on the compiler side. (1) hypot(x,y): the autodiff rule computed (x'+y')/(2*hypot) -- the sqrt(x) pattern x'/(2*sqrt) misapplied to a 2-arg function -- instead of the correct (x*x'+y*y')/hypot; 28% off at V=0.7, y=0.5, only accidentally correct at x=0.5. Fixed by splitting hypot from sqrt in inst_cache (cache holds hypot(x,y), not 2*hypot) and a correct chain rule with the usual zero/one folding. (2) atan2(x,y): two bugs in the cached factors feeding the shared Pow|Atan2 chain rule (x'*c0 + y'*c1)*c2 -- the common factor c2 was (x^2+y^2) where the rule MULTIPLIES, so it needed the reciprocal 1/(x^2+y^2); and c1 (the y' factor) was +x where d/du atan2 = (x'*y - y'*x)/(x^2+y^2) SUBTRACTS, so it needed -x. Wrong magnitude AND sign. Fixed: c2 = 1/(x^2+y^2), c1 = -x. The fix is at the autodiff-rule level so it applies to resistive currents, reactive charges (verified via AC susceptance of ddt(hypot)), higher-order derivatives, and ddx. verify_vafautodiff: 9 checks (both arg orders, sqrt-form equivalence, the V=0.5 accidental-correctness point, atan2 sign + atan2(V,V)->0 cancellation, reactive susceptance) + a 15-builtin regression battery confirming the rest were already correct. Regression 150/150.

E-187 *mir_opt (simplify.rs)*

math-identity simplifier: invalid inverse-function cancellations. The algebraic simplifier (simplify_unary_op) rewrites f(g(x)) -> x for function-inverse pairs. Six of these were applied unconditionally even though the cancellation is only valid when f is a true left inverse of g over ALL of g's range -- so they returned the raw inner x, WRONG for ordinary finite inputs, corrupting the DC value itself (a more severe class than the derivative-only autodiff bugs). (1) asin(sin(x)) != x for |x|>pi/2 (asin(sin 3)=pi-3=0.1416). (2) acos(cos(x)) != x outside [0,pi] (acos(cos 4)=2pi-4). (3) atan(tan(x)) != x for |x|>pi/2 -- and this is a legitimate angle-WRAP idiom the optimizer silently defeated (atan(tan 2)=2-pi). (4) acosh(cosh(x)) != x for x<0 (cosh even -> |x|). (5,6) sqrt(x*x) *and sqrt(x**2) are |x|, not x (sqrt((-3)^2)=3, returned -3). The const-fold path (eval_unary) evaluates each op numerically and was always correct, so constant spot-checks passed -- the bug lived only in the symbolic cancellation on runtime values. Fix: Asin/Acos/Atan/Acosh decline to simplify (return None); the Sqrt arm returns None (MIR has no fabs, so the sqrt stays and computes |x|). Kept the cancellations valid over the whole real line (tan(atan), ln(exp), asinh(sinh), atanh(tanh), sinh(asinh), cosh(acosh), log10(pow(10,.))). Rewrites unsound only for Inf/NaN (x-x->0, x*0->0, sin(asin(x)), exp(ln(x))) left as standard finite-math.* Removed the now-unused F_TWO import. mir/mir_opt/mir_autodiff unit tests unchanged (13/7/19). New example mathident_examples/verify_mathident.py: 12 DC-value checks. Regression 151/151.

E-186 *mir_autodiff (builder.rs + lib.rs)*

autodiff audit: real-modulo (%) derivative fix. The same AC-conductance referee (E-185) caught a third builtin right in VALUE, wrong in DERIVATIVE. Verilog-A real modulo lowers to the Frem opcode; x % c = x - floor(x/c)*c is a slope-1 sawtooth in x, so d/dx(x % c) = 1 away from the wrap points -- yet Frem was grouped with the genuinely-constant opcodes (floor, ceil, $clog2, integer/bitwise ops, comparisons) and forced to derivative 0 in TWO places: (1) the live-derivative gate zero_derivative() in lib.rs, which decides which SSA values even depend on the differentiation variable -- with Frem listed, a modulo result was declared independent of the unknown so its derivative was never even requested; and (2) the inst_derivative chain rule in builder.rs (the => return no-edge arm). Because the gate came first the AC/Jacobian contribution of any modulo term was identically zero: a .dc sweep*

*of I=1e-3*(V % 1.0) had slope exactly 1e-3 while the AC conductance read 0. Fixed by removing Frem from both zero-derivative groups and giving it the correct rule d/du(x % c) = x' - floor(x/c)*c' -- folding to just the dividend derivative x' for the common constant divisor (c'=0), and emitting the -floor(x/c)*c' term only when the divisor itself carries a derivative. floor/ceil/$clog2/integer ops correctly stay in the zero group. Applies everywhere the derivative is taken -- resistive I, reactive Q, ddx, higher orders. verify_vafautodiff extended to 14 checks (adds 5 modulo cases: two constant divisors, prefactor scaling, chain-rule d/dV (V%1)^2 = 2*(V%1), and floor/ceil staying 0), a separate 3-terminal probe confirming the variable-divisor branch, cross-checked via the ddx value path; the 19 mir_autodiff unit tests are unchanged and pass. Regression 150/150.

E-213 *syntax (lib.rs, parsing/tree_builder.rs), lexer (lib.rs), parser (grammar/paths.rs), preprocessor ( parser.rs), examples/vafcrash_examples/**

crash hardening: four compiler panics. Fuzzing openvaf-r with malformed Verilog-A found four distinct panics: instead of printing a diagnostic the compiler aborted with "OpenVAF encountered a problem and has crashed!" (exit 101) and directed the user to file a bug report -- for what is often just a typo. All are reachable from ordinary source mistakes; the headline case is a module merely missing its `endmodule`, one of the most common Verilog-A editing errors. BUG 1 -- EOF SPAN (tree_builder.rs): a parse error at end of file built its span as TextRange::at(self.text_pos, ); text_pos is already the end of the source, so adding the previous token's length produced a span running PAST the end of the file -- 6..12 for the 6-byte input "module". Mapping it back to its file then failed assert!(range.end() <= self.range.end()) in FileSpan::with_subrange (preprocessor/sourcemap.rs), crashing the compiler WHILE TRYING TO PRINT THE ERROR ("subrange 6..12 -> 6..12 must fit into the total range 0..6"). FIX: an empty range at EOF -- exactly what the adjacent expected_at field already does. Also find_ctx_range (syntax/lib.rs) maps a position through half-open [start,end) ranges that never cover the EOF position itself (pos == last_range.end() compares Less against every range); it .expect()ed and panicked, now clamps. Triggers: missing endmodule; bare `module`; module header then EOF; unclosed `analog begin`; unterminated string. BUG 2 -- REAL LITERAL (lexer/lib.rs): e/E was consumed as an exponent marker without checking an exponent follows (eat_float_exponent()'s return, which reports whether a digit was seen, was discarded), so 1e became a Float token whose text does not parse as f64 and panicked in ast::StdRealNumber::value()'s src.parse().unwrap(). FIX: an `e` only joins the number when a digit (optionally signed) follows; a bare 1e lexes as 1 + identifier e and surfaces as an ordinary parse error -- the approach based_literal_body already takes for a malformed 8'squark. Valid exponents (1.5e3, 2e-3, 1E6) untouched. Triggers: 1e, 1e+, 99e, 1.5e, parameter real p=2e. BUG 3 -- PREPROCESSOR EOF (preprocessor/parser.rs): previous_range() (self.full_tokens[pos], reached while building the "expected an identifier" diagnostic) and followed_by_bracket_without_space( ) (self.relevant_tokens[self.pos + 1u32]; "index out of bounds: the len is 2 but the index is 2") indexed the token list directly, out of bounds one past the last token -- a bare "`define`" ending the file. FIX: both use `.get()` with a sensible fallback, mirroring the `.get( ).map_or()` idiom `current_range()` already used. BUG 4 -- PATH() ASSERT (parser/grammar/paths.rs): path() opened with `assert!(p.at_ts(PATH_SEGMENT_TS))`, a precondition several callers do not check and plausible input violates: aliasparam x = 5;(a literal where a parameter name belongs),aliasparam x = ;, I(<1>)/I(<>)( port_flow),discipline d 1 = 2;. FIX: drop the assert -- the next line, `p.expect_ts( PATH_SEGMENT_TS)`, already emits "expected identifier" and returns false, so the error is reported and an empty path node completed (nature's parser guarded its own call site this way in E-39; this makes the helper safe for every caller). This is the openvaf-r counterpart to E-212 (the same campaign against ngspice) and a direct continuation of E-148: E-148 hardened against pathological DEPTH (inputs too big), E-213 covers inputs that stop too early or are malformed. No change to any accepted program -- every fix is on a path that previously aborted the compiler; generated OSDI for every existing model is identical. Verify (examples/vafcrash, 25 checks): every repro yields a clean error instead

`of a crash; valid code unchanged (resistor; real exponents; aliasparam bound to a real parameter;` define with args). NOTE: openvaf-r installs a CUSTOM panic hook printing its own message and exiting 101 rather than dying on a signal or printing the usual Rust "panicked at" -- a crash check looking only for those two (as E-148's suite does) scores these panics as ordinary errors, so the new suite treats exit 101 and the hook message as a crash. Toolchain tests unchanged (lexer 8, preprocessor 6, basedb 9, sim_back 26, hir 15, hir_lower 4). 25 checks. Regression 173/173 (new example folder).

E-214 *hir_lower (expr.rs, stmt.rs), examples/arraycast_examples/**

whole-array type coercion: a recurring crash class, fixed at its root. An integer Value reaching a float MIR op (feq/fmul/fsub/fdiv) panics mir_opt::const_eval::eval_binary, which has no (Int, Float) case -- "invalid operation fdiv Int(1) Float(..)", exit 101, "please open an issue". (Unfolded, the same defect instead reaches LLVM as `i32 = fadd .., ConstantFP:f64` and aborts with "LLVM ERROR: Cannot select" -- one bug, two faces.) FOUR INSTANCES, each patched at its own call site as found: the `case` discriminant over an integer array (E-33, element type hardcoded real); laplace_/zi_ integer-LITERAL coefficients (b77266ec); laplace_/zi_ integer ARRAY-VARIABLE coefficients (c55812d6 -- the previous fix had reasoned "a whole-array variable reference is already real by its declaration", which is false for an `integer` array); and an integer `case` ITEM against a REAL array discriminant (8d0ab057 -- the opcode is chosen from the discriminant, Feq, but the item's elements lower to i32). ROOT CAUSE: inference DOES record the coercion (expect( ) -> casts.insert(item, Array{Real})), but it lands on the array EXPRESSION, and lower_array_ elems_impl decomposes the array and lowers each element itself -- so lower_expr's needs_cast( ) never saw it and the cast was silently DEAD. Every new array-consuming context therefore re-inherited the trap. FIX: lower_array_elems_impl -- the one place every whole-array consumer passes through -- now honours a recorded cast to a real target (coerce_real |= matches!(needs_ cast(expr), Some((_, dst)) if *dst.base_type() == Type::Real)), making inference's intent effective for all consumers instead of requiring each call site to ask. Proven in isolation: with the case-site flag disabled the structural guard alone fixes every case repro. lower_case additionally keeps its own discr_op == Opcode::Feq coercion -- choosing the opcode from the discriminant makes matching operand types that function's invariant, independent of inference's bookkeeping. NO MISCOMPILE: the integer spelling of a filter is bit-identical to the '{1.0} spelling (worst AC diff exactly 0 dB over 13 points) and matches the analytic response to 3e-8 dB; an integer `case` item still selects its arm. MUTATION-TESTED: reverting the fixes makes all four repros crash (exit 101) again, so the guard is not vacuous. Also folds in two verification guards hardened during the same hunt -- vafautodiff (8->16 checks: cross-derivatives with both arguments live, the blind spot that hid E-185's hypot bug, plus a 44-point battery) and operator_examples (+14 const-vs-runtime checks: every check there used literal operands, exercising only the folder -- the side E-37's >> bug happened to be on). New examples/arraycast_examples (23 checks). Regression 174/174.

E-215 *hir_ty (builtin.rs), hir_lower (callbacks.rs, expr.rs), osdi (compilation_unit.rs, stdlib.c), examples/ plusargs_examples/**

$test$plusargs / $value$plusargs productionized. E-12 had gated these as mechanism-unavailable fallbacks ($test always FALSE, $value never wrote its output). E-215 serves them through the simparam channel (ngspice publishes each command-line +name[=value] as namespaced simparams -- see the ngspice report). COMPILER side: $test$plusargs("name") lowers to simparam("$test$plusargs$name",0)!=0; $value$plusargs("name=%fmt",var) builds the key from the compile-time literal and reads by TARGET TYPE -- $valnum for int/real (ficast for an int target), and a NEW non-fatal simparam_str_opt for a string target. simparam_str (E-25) FATALS on an unknown name, which a plusarg probe must not, so simparam_str_opt (stdlib.c + the SimParamStrOpt callback + compilation_unit wiring) returns the default instead. hir_ty: VALUE_PLUSARGS's 2nd arg became an OUTPUT target -- three signatures Var(Integer)/Var( Real)/Var(String) (mirroring $random's seed / $fgets's buffer) instead of the read-only Val( String) that made extraction impossible; TEST_PLUSARGS's arg is now Literal(String). DESIGN

NOTE: reading the value through the op-dependent simparam channels DELIBERATELY avoids the $sscanf scanner, whose scanf_begin/scan_* thread a hidden module-global cursor with no explicit data dependency -- the setup/eval partitioner can hoist a bias-independent scan_* into instance setup while its scanf_begin stays in eval, so the scan reads a NULL cursor and segfaults (observed, then designed out). $value$plusargs matches only the name=value form per the LRM. No OSDI ABI change; old .osdi unaffected. Verify (examples/plusargs, 12 checks both solvers). Regression 175/175 (new example folder).

E-219 *preprocessor (grammar.rs), basedb (diagnostics/sink.rs), examples/robustness_examples/**

preprocessor macro-argument hang + diagnostic-flood cap. Re-running the openvaf-r robustness campaign against the shipped binary (see the robustness report) found a FIFTH hang path the E-147/E-148 guards did not cover, at a ~2% mutation-fuzz rate. FINDING A -- INFINITE LOOP: a backtick token followed by `(` (`name() is a macro call, and the preprocessor collects its parenthesised argument list token by token in parse_macro_call -> parse_macro_token. The SAME non-advancing pattern exists in the` define PARAMETER list loop (parse_define). Neither collector had a forward-progress guarantee: when a non-Macro compiler directive (`include,` ifdef, `endif,` undef, ...) appeared inside the argument list, parse_macro_token pushed an UnexpectedToken error and RETURNED WITHOUT CONSUMING the token, so the caller's collection loop re-examined the same directive token forever -- an unbounded diagnostics-vector growth that pins the CPU (a `sample` of a hung compile named the loop exactly: process_token -> parse_macro_call -> parse_macro_token -> RawVec::grow_one -> realloc). It is trivially reached: injecting `(` anywhere near an `include (or any macro token) splits the directive so the rest of the file is scanned as a macro argument. (define did` NOT trigger it -- compiler_directive() has no `define arm, so it falls through to Macro and is consumed as a bogus macro name, which happens to advance; only the RECOGNISED directives hit the non-advancing branch.) A stray delimiter in a` define parameter list that is neither an identifier, ) nor , (e.g. a / or ” in a corrupted define) is likewise consumed by none of the loop's expect/eat calls, so parse_define spins on it. FIX (grammar.rs): guarantee forward progress -- the stray-directive branch of parse_macro_token consumes the offending token after reporting it (p.bump); and BOTH collection loops (macro-call args and `define params) gained a backstop that records the source offset and bails with a clean UnexpectedEof if an iteration consumes nothing, so no non-advancing token, present or future, can spin them. No valid model puts a directive inside a macro call's parentheses, so valid input is unaffected. FINDING B -- DIAGNOSTIC FLOOD: with A fixed, deeply nested NON-macro garbage (e.g. sin( x3000 in a declaration) no longer looped but still took ~40s -- it produces thousands of parse errors and codespan_reporting builds a full source-annotated report for EVERY one; rendering ~3000 diagnostics (each extracting and laying out its surrounding source) is the entire cost (it persists with stderr redirected to /dev/null -- it is the BUILDING, not the writing). A bounded-but-40-second rejection is still a DoS vector. FIX (sink.rs): cap the number of diagnostics the console sink RENDERS at 128; beyond that only the counters advance and a one-line 'further diagnostics suppressed' note prints. summary() still reports the true error total and the exit code is unchanged -- the standard 'too many errors' behaviour of rustc/clang, which does not affect any input with <=128 diagnostics (every real model and every ordinary mistake). RESULT:` name( + stray `include: infinite -> clean error <0.01s; real model + injected (': >90s -> <0.05s;` sin( x3000 in a declaration: ~40s -> <1s; a few real errors: unchanged, all shown. Continuation of E-148 (E-148 hardened the parser expr-depth / include recursion / array expansion; E-219 covers the two paths E-148 did not touch -- the preprocessor macro-arg collector and the diagnostic sink). Verify: examples/robustness_examples gains eight argument-collection cases -- five macro-call (`m( then include/ifdef/endif/undef, and one with 4000 leading '('`) and three define parameter-list (M(a,/,b), M(a”b), M(a;b)) -- plus a valid macro-call-with-nested-parens regression

(26/26); the production corpus (VA_TEST/compile_all.py) still compiles 92/92 standalone models to the identical verdict; the campaign fuzzer's ~2% hang rate (3000 iterations) drops to 0 hangs, 0 crashes. Two files (openvaf/preprocessor/src/grammar.rs, openvaf/basedb/src/diagnostics/sink.rs). No change to any valid program's output beyond the >128-diagnostic suppression note.

E-220 *parser (parser.rs), syntax (lib.rs), preprocessor (grammar.rs, sourcemap.rs), basedb (diagnostics.rs), hir_ty (diagnostics.rs, inference.rs, validation.rs, validation/body.rs), examples/vafcrash2_examples/**

crash hardening round 2: ten compiler panics -> clean errors. A second robustness pass, continuing E-213 and E-219. Re-running the mutation fuzzer against the shipped compiler with DIVERSE compact-model seeds (BSIM/HICUM/PSP/HiSIM/VBIC/MEXTRAM/EKV) and new mutation strategies (keyword, attribute (* *), and bracket injection alongside byte/truncate/delimiter/deep-nest) found ~5% of mutated inputs still CRASHED the compiler (exit 101, 'OpenVAF encountered a problem and has crashed!') rather than reporting an error. Every one is the E-213 pattern -- a panic/assert/unwrap/index a valid model never reaches but malformed input does (all caught by the E-213 panic hook, so never memory-unsafe; the bug is the crash UX + missing diagnostic). TEN root causes, all fixed: (1) parser/parser.rs -- the parser can spin in error recovery; a step counter assert!(steps<=10M,'seems stuck') panicked. It now signals EOF at the limit so parsing winds down and reports its errors. (2) basedb/diagnostics.rs -- a diagnostic built from an empty node list hit to_unified_span_list([])=>unimplemented!(); it renders label-less, anchored to the new SourceMap::root_file(). (3) preprocessor/grammar.rs -- TextRange::new(start, end) with start>end (a macro-argument/`define`/include span at EOF) tripped a text-size assert; five sites clamp end>=start. (4) hir_ty/diagnostics.rs + validation.rs -- 28 sites did expr_map_back[e].unwrap()/stmt_map_back[s].unwrap(); a SYNTHESIZED expression/statement has no source-map-back entry, so reporting a type error on it panicked. They resolve the span through a fallback (empty range) via one expr_range helper. (5) hir_ty/validation/body.rs -- 11 sites did expr_types[arg].unwrap_node()/unwrap_branch()/unwrap_port_flow(), which unreachable!() when a builtin (e.g. $port_connected) or nature access gets a wrong-typed argument inference did not reject; they bail cleanly (match {Ty::X(id)=>id,_=>return}). (6) syntax/lib.rs -- to_ctx_span mapping a span across source contexts could yield start>end in TextRange::new; clamp. (7) preprocessor/sourcemap.rs -- FileSpan::with_subrange asserted the subrange fit its parent (E-213 fixed one trigger at its source; this makes the mapping itself total); clamp into the parent. (8) preprocessor/grammar.rs -- stripping include quotes with path[1..len-1] panicked on a malformed/unterminated string literal (a lone quote); saturating_sub + get make it total. (9) hir_ty/inference.rs -- resolving a call whose arguments match NO overload left the candidate list empty after the semantic-retain and candidates[0] indexed out of bounds; it falls back to the pre-filter set ( mirroring the existing restore after the exact-match retain). (10) hir_ty/validation/body.rs -- the builtin-validation arms index args[0..2] assuming the call has as many arguments as the builtin requires; a call with too few (e.g. $simparam(), $port_connected()) indexed out of bounds. ONE entry guard skips builtin validation when args.len() < BuiltinInfo::from(call).min_args -- inference already reports the ArgCntMismatch. Method: the fuzzer classifies OK/clean-error/CRASH/HANG; each crash was triaged from its crash log to the innermost openvaf frame, grouped, and the panic read at the source; fixes were applied one cause at a time, each verified against the recorded crash corpus, then a fresh fuzz re-checked convergence (each pass exposed the next-rarest site -- a classic fuzzing tail -- until the rate hit zero). Result: the ~390-input recorded crash corpus all clean-errors; a fresh 12000-iteration fuzz on the fixed compiler is 0 crashes / 0 hangs (down from ~5%); the 92/92 standalone production models compile to the identical verdict; parser/syntax/preprocessor/basedb/hir_ty/hir/sim_back unit suites pass. Verify (examples/vafcrash2): 19 checks with hand-crafted inputs targeting each cause, MUTATION-TESTED (reverting the fixes makes the guarded inputs crash again, so the suite is not vacuous). No OSDI/ABI change; generated .osdi for every existing model is identical. Eight files. Regression 179/179 (new example folder).

E-230 *hir_def (item_tree/lower.rs), hir_ty (validation.rs), syntax (ast/expr_ext.rs), examples/vafcrash3_examples/**

crash hardening round 3: three compiler panics -> clean errors. A third robustness pass (following E-213/E-219/E-220). The production corpus recompiled (92/92 standalone, identical verdicts) plus a fresh ~19,500-iteration mutation-fuzz over diverse compact-model seeds found THREE more distinct panic root causes, all the E-213 class (caught by the panic hook, memory-safe, but a crash instead of a diagnostic). (1) hir_def/item_tree/lower.rs -- a `begin :` sequential block with the scope colon but a MISSING/invalid name identifier has `block_scope().is_some()` yet 'name = block_scope().and_then(

E-261 *mir_autodiff (builder.rs), examples/vafsqrtguard_examples/**

autodiff: guard the **`sqrt()`** derivative singularity. `d/dx sqrt(x) = 1/(2*sqrt(x))` is `+inf` at ngspice's default `x=0` DC initial guess; openvaf-r emitted that raw `+inf` into the Jacobian, which produced a `nan` and made the operating point fail outright -- a conductor `I=K*sqrt(V)` never found ANY DC op (dynamic/true gmin, source, and pseudo-transient stepping all died, node = `nan`). The tell was an internal inconsistency: `Pow` carried an explicit `base==0 -> derivative:=0` guard "to ensure numerical stability" while `Sqrt` had none, so the mathematically identical `pow(V,0.5)` and `V**0.5` converged and ngspice's own behavioral B-source `sqrt(v(n))` converged, but `sqrt(V)` NaN-failed. (The `Pow` guard is itself incomplete -- a block split that only protects a bare terminal `pow`/`sqrt`; `2.0*pow(V,0.5)` fails on the unmodified compiler because the downstream multiply consumes the raw derivative.) FIX (`inst_cache`): change the `Sqrt` derivative cache from `2*sqrt(x)` to `2*sqrt(x + a)` (`a = 1e-18`) -- the exact derivative of the smoothly regularized `sqrt(x+a)`, so the emitted derivative is `x'/(2*sqrt(x+a))`. It is FINITE at `x=0` (`1/(2*sqrt(a)) ~ 5e8`, a large but bounded conductance -> small controlled Newton steps that creep out of the singularity, exactly like the B-source); EXACT for `x>0` (the nudge is INSIDE the root, so the perturbation is `~a/(2x)`, below the ULP -- no finite-bias derivative changes, higher-order derivatives and the internal `sqrt(1-x^2)` of asin/acos/asinh/acosh/atanh included); and COMPOSABLE (being a plain value it propagates through downstream operators `K*sqrt(x)`, `1/(1+sqrt(x))`, `exp(-sqrt(x))`, unlike the block-split guard). Two alternatives were rejected: a block-split `Sqrt` arm mirroring `Pow` (does not compose -- scaled `sqrt` still fails), and an additive `2*sqrt(x)+delta` (perturbs the derivative `~delta/(2*sqrt(x)) ~ 1e-9`, breaking the bit-exact higher-order asin/acos autodiff unit tests) -- the `sqrt(x+a)` form is exact to the ULP AND convergent, so NO test tolerance was loosened. `pow(x,fractional)` is unchanged (its base derivative `(y/x)*x^y` is an `inf*0` form in the shared `Pow`/`Atan2` chain rule with no clean branchless regularization; `sqrt()` is the standard spelling). Verify (examples/vafsqrtguard, both solvers): bare and strongly-scaled `K*sqrt(V)` (K=1,2,5) find their true KCL op -- not `nan` -- and match the equivalent B-source to ~1e-5; the guarded derivative `K/(2*sqrt(V))` is exact for `V>0` (~1e-6); composed `G0/(1+sqrt(V))` converges; `pow(V,0.5)` now agrees with `sqrt(V)`. openvaf-r's own autodiff suite (incl. numeric third-order asin/acos exactness at `10*eps`) and the OSDI/sim_back MIR snapshots pass unchanged in value. Two source files.

E-262 *mir_autodiff (builder.rs), examples/vafsqrtguard_examples/**

autodiff: guard the **`pow(x,y)`** base-derivative singularity (the E-261 sqrt fix, applied to pow). For `0<y<1`, `d/dx pow(x,y) = y*x^(y-1)` is `+inf` at the `x=0` DC initial guess -- the same singularity as sqrt (pow(x,0.5) IS sqrt(x)). openvaf-r builds it via the chain rule shared by Pow and Atan2: `d(pow) = (x'*cache[0] + y'*cache[1])*cache[2]` with `cache[0]=y/x`, `cache[2]=x^y`, so the base term `x'*(y/x)*x^y` is `+inf*0 = NaN` at `x=0` (the `y/x` factor is `+inf`, the `x^y` factor is `0`) -- it NaN-poisons the Jacobian and the operating point fails. Pow carried a PARTIAL guard (a block split forcing the derivative to 0 when `base==0`), which is why a BARE terminal `pow(V,0.5)` converged -- but a block split cannot compose: a downstream instruction (`2*pow(V,0.5)`, `1/(1+pow(V,0.5))`) consumes the RAW derivative computed inside the conditional block, so scaled/combined fractional pow still NaN-failed on the unmodified compiler. FIX (`inst_cache`, mirroring E-261's

sqrt): cache the derivative of the a-shifted `pow(x+a, y)` (`a = 1e-18`) while the VALUE `res = x^y` is unchanged -- `cache[0] = y/(x+a)`, `cache[1] = ln(x+a)`, `cache[2] = (x+a)^y`. The shared chain rule then yields base term `x'*y*(x+a)^(y-1)` and exponent term `y'*ln(x+a)*(x+a)^y`, both FINITE at `x=0` (`y*a^(y-1)` is large but bounded -> a controlled Newton step out of the singularity, like the sqrt guard and ngspice's B-source), EXACT for `x>0` (the nudge is inside the power, perturbation `~a/x` below the ULP), and PLAIN VALUES so they compose through downstream operators. With a composing branchless guard in place the old block-split Pow guard is removed. Atan2 -- which shares the chain-rule ARM but has its own cache -- is untouched and verified unchanged; `y>=1` is a no-op (no singularity); the solution-dependent exponent term is now finite at `x=0` too (`ln(x+a)` not `ln(0)=-inf`). `ln(x)`/`1/x` are left as-is (their VALUE, not just the derivative, is +-inf at 0, so no derivative guard well-poses a model that evaluates them at 0). Verify (examples/vafsqrtguard [6]/[7], both solvers): bare and strongly-scaled `K*pow(V,Y)` (K=1,2,5 at Y=0.5,0.3,0.25) find their true KCL op -- not `nan` -- where the unmodified compiler NaN-failed the scaled/fractional cases; the guarded derivative `K*Y*V^(Y-1)` is exact for `V>0` (~1e-6). openvaf-r's autodiff suite (incl. atan2 and higher-order checks) and the OSDI/sim_back MIR snapshots pass unchanged in value; the many production models using pow (BSIM/PSP/...) are bit-unchanged for `V>0`. Completes the autodiff singular-derivative work (E-261 sqrt + E-262 pow). Two source files.

E-263 *sim_back (init.rs), hir_ty (inference.rs), hir (elaborate.rs), test_data/ui/ddx.log, examples/vafcrash4_examples/**

robustness fuzzing: three compiler panics -> clean errors (the 4th such pass, following E-213/E-219/E-220/E-230). Three fuzzing strategies against the committed compiler -- byte/token mutation of the whole .va corpus, grammar-aware STRUCTURED adversarial inputs, and VALID-but-pathological modules that compile through to the backend -- surfaced one distinct panic each (all the E-213 class: panic-hook-caught, memory-safe, but a crash instead of a diagnostic). (1) `sim_back/init.rs` -- deeply nested analog operators (`I<+absdelay(ddt(ddt(absdelay(..)))))`) produced a value tagged for the instance-setup cache whose mapping in the INIT function is a `Const` or has no definition at all (`ValueDef::Invalid`); `build_init_cache` assumed every cached value is a computed instruction result and called `unwrap_result()`/`unwrap_inst()`, and even once survived the OSDI LLVM backend read a `BuilderVal::Undef`. FIX: before allocating a cache slot, handle the non-instruction cases -- substitute a `Float`/`Int`/`Str`/`Bool` CONSTANT init value directly into the eval function (eval uses the init-time value; no runtime slot needed) and treat an `Invalid` mapping like the existing dead-value path (default to 0). Only a genuine instruction result takes the slot+optbarrier path, so no cache slot or codegen ever sees a constant/undefined value again. (2) `hir_ty/inference.rs` -- `ddx(V(p,n), 5)` (a `ddx` whose 2nd arg is not a potential/flow probe) crashed `hir_lower` at `value_def(unknown).unwrap_param()`. The type-checker HAD an "invalid ddx unknown" diagnostic, but its guard tested `expr` (the `ddx` call itself, ALWAYS an `Expr::Call`) instead of `unknown`, so the diagnostic was DEAD CODE and the malformed call slipped through to codegen. FIX: test `unknown` -- a non-call unknown (a literal, a plain variable) now raises the diagnostic and compilation stops before lowering. (This also newly (correctly) diagnoses the `ddx(1.0,1.0)`/`ddx(1.0,foo)` "random fuzz" cases the `ui/ddx.va` test already carried but that the dead code never rejected -- `ddx.log` updated to 8 errors.) (3) `hir/elaborate.rs` -- a malformed top-level module whose item TREE recorded an instantiation but whose parsed AST `module_items()` came back empty (parser error-recovery and item-tree construction disagreeing) crashed the E-5/E-49 hierarchy-flattening pass at `items.first().unwrap()`. FIX: an empty AST item list means nothing to flatten -- return the module text verbatim, the same no-op the pass already takes for a module with no instantiations. Behaviour-preserving for valid input: full dual-solver regression 214/214, `cargo test` unchanged except the intended `ddx.log` update (no MIR/OSDI snapshot changed -- no real model exercises the new paths), and a re-fuzz of the fixed compiler across all three strategies finds no surviving panics. Verify (examples/vafcrash4): a minimal reproducer per cause now compiles cleanly (nested analog ops) or clean-errors (ddx, malformed module), each having exited 101 on the shipped

binary. Three source files + one snapshot.

E-264 *hir (elaborate.rs), osdi (lib.rs), examples/vafhang_examples/**

large instance arrays: hierarchy-flatten O(N²)->O(N) + OSDI-codegen stack headroom for deep per-node fan-in. Two scalability/robustness defects on the same path -- compiling a module that instantiates a large array of a sub-module. (1) FLATTEN O(N²)->O(N) (`hir/elaborate.rs`): the E-5/E-49/E-86 pass renders one textual copy of a sub-module per instance with a per-instance name prefix and rewrites hierarchical name references, running N times for `leaf u[0:N]`. Three inner steps were themselves O(N), making it O(N²) -- doubling N QUADRUPLED time (2k≈1.8s, 8k≈30s, 16k≈100s), so a big array looked like a hang: hierarchical-name resolution (`find_instance_path_holes`) re-scanned every instance prefix per token via `.keys().any(..)`; it was called per port binding, per instance (multiplying by N again); and each per-instance scope `clone()`d the whole E-86 absolute-reference map (O(N) entries). FIX (behaviour-preserving): precompute an ANCESTOR SET once for O(1) "is this chain a hierarchical prefix?" tests (no per-token scan); a DOT-FREE EARLY-OUT (every hierarchical ref contains a `.`, so the common no-dot token skips resolution entirely); and share the absolute-reference map by reference -- a small `Rc<AbsPrefixes>` (map + its precomputed ancestor set) is `Rc::cloned` (O(1)) into each scope instead of deep-copied. Now O(N): 16 001 instances elaborate+compile in ~1s (was ~100s), 32 001 in ~2s; the elaborated text and every resulting `.osdi` are IDENTICAL (only faster). (2) CODEGEN STACK (`osdi/lib.rs`): when many instances contribute to the SAME node and don't collapse (distinct per-instance parameter), the residual is a chain of thousands of accumulated contributions that OpenVAF's recursive OSDI codegen + autodiff walk; that recursion ran on a rayon codegen worker whose default stack is only a few MB, so past ~8 000 contributions it aborted with `thread ... has overflowed its stack` / SIGABRT -- a crash on large-but-valid input. FIX: run OSDI codegen on a dedicated rayon pool built with a 256 MiB worker stack instead of the default global pool. A thread's stack size is a property of the thread, not the work -- it cannot change the emitted code (the `.osdi` is byte-equivalent modulo the linker's already-nondeterministic build id) -- it only lets the deep recursion complete; the 8 001-contribution design now compiles cleanly. Behaviour-preserving: full dual-solver regression passes and `cargo test` is unchanged (no MIR/OSDI snapshot moved -- the flatten fix only accelerates identical work; the stack fix is a thread property). Extreme instance counts remain bounded by the existing 1 M array/generate caps and downstream compile cost. Verify (examples/vafhang): check A ( always) -- 16 001- and 32 001-instance arrays compile in ~1s/~2s within generous absolute bounds a re-introduced O(N²) would blow; check B (`--slow`) -- the 8 001-contribution deep-fan-in design compiles cleanly (~38s) where the pre-fix binary SIGABRT'd. Two source files.

E-265 *hir_ty (inference.rs), examples/vaflaplace_examples/**

**`laplace_*`/`zi_*`** coefficient argument: compiler panic -> clean diagnostic (fifth robustness-campaign find, following E-213/-220/-230/-263). A num/den (pole/zero) argument must be a real coefficient array (LRM 9.19). `laplace_*` bypasses the generic argument checker (which would reject a bare array-variable reference before the operator's special case can accept it) and uses `infere_array_arg`, whose fallback returned the inferred type WITHOUT requiring a real value -- so a bare NET reference (`laplace_nd(1.0,1.0,p)`), a branch, or a string was typed but never rejected, reached `hir_lower`, and panicked (`resolve_path`: "invalid HIR: path .. was not resolved", exit 101) when the coefficient elements were lowered as values. Every ordinary value context (and the laplace INPUT arg) already rejects a net-as-value cleanly; only the coefficient arg was different. FIX: the check goes in `infere_laplace`, NOT in the shared `infere_array_arg` -- that helper is also used by E-33 `case` discriminants/items and E-34 concatenations, which legitimately carry integer/string values, and forcing real there made integer `case` PANIC (caught in regression before shipping). `infere_laplace` now requires each num/den type to be a real/integer scalar or array (`Ty::to_value()` is `None` for a net/branch reference -> rejected), emitting the normal "expected real value but found .." type-mismatch; the array-literal / array-variable / scalar shapes are unchanged. A second adjacent crash from the same fuzzing -- an empty

DIRECT denominator (`laplace_nd(V,1.0,'{})`, where the state-space realization computes `den.len()-1` and reads `den[n]`, underflowing/OOB) -- is rejected too (the operator kind is now passed to `infere_laplace` to tell a direct denominator from a `*_np`/`*_zp` pole list); an empty NUMERATOR (H=0) and an empty POLE list (denominator polynomial 1) stay legal. Behaviour-preserving: full dual-solver regression passes (incl. the laplace/zi filter models in the complex-pole and RF-convolution suites), `cargo test` UI snapshots unchanged, and a re-fuzz of 6000 random malformed `laplace_*`/`zi_*` calls finds no surviving panic. Verify (examples/vaflaplace, 15 checks): six malformed coefficient args (net-den/net-num/branch/string/net-in-zi_zp-roots/empty-direct-den) now clean-error; well-formed shapes (real/int array literals, scalar, array-var ref, zi_nd, empty numerator, empty pole list) still compile. One source file.

Enhancements not listed (57, 60, 62–64, 69, 72–77, 79–83, 94–95, 98–100) changed no compiler sources — they were validation suites, documentation, benchmark tooling, or ngspice-side work (see the ngspice report).

E-286 *mir_opt (const_eval.rs, simplify.rs), examples/vafcodegen_examples/**

constant-folding an integer division by zero killed the compiler. `q = 5 / 0;` exited openvaf-r with an internal error and produced no `.osdi`, while a *runtime* zero divisor had always been accepted (the generated code just performs the division) -- an inconsistency, not a policy. CAUSE: `eval_binary`'s integer arm evaluated every opcode directly (`func.dfg.iconst(lhs / rhs)`), so with `rhs == 0` the division happens INSIDE the compiler process and the compiler is what dies; `i32::MIN / -1` overflows the same way. The neighbouring arms had a quieter version of the same defect -- `Iadd`/`Isub`/`Imul` folded with checked arithmetic and the shifts folded by an unconstrained distance, so `2147483647 + 1` and `1 << 40` were folded in a way that does NOT match what the generated code computes (LLVM emits plain two's-complement wrapping arithmetic; a shift distance outside `0..32` is poison). This is the `const_eval == codegen` invariant: the folder must produce exactly what the runtime path would, or decline. FIX: `eval_binary` returns `Option<Value>` (the convention `eval_unary` already used) -- it declines for div/rem by zero, `i32::MIN/-1`, and shift distances outside `0..32`, leaving the instruction on exactly the runtime path a non-constant divisor takes; and folds add/sub/mul with `wrapping_*`. The single call site in `simplify.rs` forwards the `Option`. Verify (examples/vafcodegen): `constfold.va` exercises all five shapes in one module and compiles, where the pre-fix compiler exited 101. `cargo test` unchanged (no MIR/OSDI snapshot moved); full dual-solver regression passes. Two source files.

E-287 *mir_opt (simplify_cfg.rs), examples/vafcodegen_examples/**

a const-folded branch orphaned a block, leaving a stale phi edge (broken SSA). A noise operator in an `if` CONDITION is zero outside noise analysis, so the optimizer proves the branch constant and folds it -- producing a function that violates SSA. CAUSE: `const_fold_terminator` rewrites the branch to a jump and calls `remove_phi_edges(dead_dst, bb)`, which fixes the phis INSIDE the newly-unreachable successor -- but the phis that actually go stale are the ones in ITS successors, which keep an edge labelled with the orphaned block, naming a value only ever available through the edge just deleted (`v22 = phi [v31, block2], [v20, block3]` where block2 is now unreachable and v31 is defined in a block that reaches block3, not block2). `simplify_bb` IS the pass that collects predecessor-less blocks and prunes their successors' phi edges -- but only on a LATER sweep, and blocks are visited in layout order so the orphan is typically already behind the cursor. The sweep never ran again because this branch of `const_fold_terminator`, unlike its `then_dst == else_dst` sibling three lines above, never set `local_changed`; with no change flagged, `iteratively_simplify_cfg` stopped with the orphan in place. Because the MIR verifier is a `debug_assert!`, the release compiler carried the malformed function forward silently. FIX: set `local_changed` after folding, so the driver sweeps again -- monotone (a folded branch becomes a jump and cannot fold again), so termination is unaffected. Verify (examples/vafcodegen): `orphanblock.va` compiles and simulates (`I == V/1k`); an assertions-enabled compiler now accepts it where it reported `v31 doesn't dominate use (block11 !dom block2)`. This pass runs on

EVERY model, so the whole 248-model example corpus was recompiled release-vs-release: zero regressions. One source file, one statement.

E-288 *mir_llvm (intrinsics.rs), examples/vafcodegen_examples/**

**hypot** was declared with one parameter and called with two. `I(a,b) <+ hypot(V(a), V(b))` produced a module LLVM rejects ("Incorrect number of arguments passed to called function!"). CAUSE: `hypot` needs a special case in the intrinsics table (Windows spells it `_hypot`), and that case declared `&[t_f64]` while `builder.rs` emits a two-argument call. Every other binary entry -- `atan2` and `llvm.pow.f64`, both a few lines away -- is correct; `hypot` is the odd one out because it sits outside the `ifn!` macro block. Constant arguments fold before codegen, so only a runtime argument reached the bad declaration. It survived because the check that reports it (`llmod.verify_and_print()`) is a `debug_assert!` -- release builds never run the module verifier. FIX: declare it `&[t_f64, t_f64]`, matching the call and its neighbour `atan2`. Stated plainly: on arm64 macOS the malformed call still produced the RIGHT number (the extra argument lands in the register the callee reads anyway) -- this was invalid IR LLVM is licensed to miscompile under a different target, calling convention, or optimization pipeline, not a demonstrated wrong answer on this platform. Verify (examples/vafcodegen): `hypotclog2.va` uses runtime parameters and checks `hypot(3,4) == 5` exactly. One source file, one argument list.

E-289 *mir_llvm (intrinsics.rs, builder.rs), examples/vafcodegen_examples/**

**llvm.ctlz** was declared without its type suffix. `\$clog2(n)` with a runtime argument produced invalid IR ("Intrinsic name not mangled correctly for type arguments! Should be: llvm.ctlz.i32"). CAUSE: `llvm.ctlz` is an OVERLOADED LLVM intrinsic -- its name must carry the type it operates on -- but it was registered and looked up under the bare name. The neighbouring overloaded entries are all spelled correctly (`llvm.pow.f64`, `llvm.sqrt.f64`, `llvm.ceil.f64`, `llvm.lround.i32.f64`); `ctlz` is the only one missing its suffix. It backs `\$clog2` (which computes `bit_width(n-1)`), so every model calling `\$clog2` on a non-constant argument emitted invalid IR; as with E-288 the module verifier that reports it is a `debug_assert!`, so release builds shipped it. FOUND BY replaying the committed example corpus through an assertions-enabled compiler: `clog2_examples/clog2_demo.va` -- a model that had been shipping and simulating correctly -- was rejected. FIX: register and look it up as `llvm.ctlz.i32` (both files, including the "intrinsic not found" message). Verify (examples/vafcodegen): `hypotclog2.va` checks `\$clog2(100) == 7` with a runtime parameter; the pre-existing `clog2_examples` suite continues to pass and now also passes under an assertions-enabled compiler. Two source files.

E-290 *osdi (inst_data.rs), examples/vafcodegen_examples/**

**\$temperature** as an operator argument used the wrong struct-GEP type -- the shipped compiler SIGSEGV'd. `I(out) <+ ac_stim("ac", \$temperature, 0.0)` killed openvaf-r with exit 139 while optimizing the model, behind the malformed IR `getelementptr inbounds double, ptr %0, i32 0, i32 5` ("Invalid indices for GEP pointer type!"). CAUSE: `LLVMBuildStructGEP2`'s first argument is the AGGREGATE being indexed -- the instance-data struct -- and the `ParamKind::Temperature` arm passed the FIELD type (`cx.ty_double()`) instead. A two-index GEP is only meaningful on an aggregate, so the IR is invalid; and the offset it describes is a flat `5 * sizeof(double)` = 40 bytes rather than `offsetof(instance, temperature)`. `TEMPERATURE` is field 5 and fields 0..4 are the param-given bitfield, the two Jacobian pointer arrays, the node mapping and the collapse flags -- variable-length arrays whose combined size is essentially never 40 bytes -- so even where LLVM did not crash the load landed on unrelated bytes. Every sibling gets this right (`eval_output_slot_ptr`, `temperature_loc` both pass `self.ty`). The bug was in TWO places: `load_eval_output` (the noise / `ac_stim` argument path) and `nth_opvar_ptr` (the operating-point-variable read path). Only `\$temperature` read DIRECTLY as an operator argument takes this path -- a computed variable (`tk = \$temperature;`) lowers to an eval-output slot and was always correct, which is why ordinary models never tripped it. As with E-288/-289 the module verifier is a `debug_assert!`, so a release build had nothing to stop it be-

fore LLVM's optimizer hit the malformed GEP. FIX: pass `inst_data.ty` at both sites. Verify ( examples/vafcodegen): `tempacstim.va` compiles (pre-fix: exit 139) and through a 1 ohm load reads back the nominal 300.15 K. One source file, two call sites.

E-291 *hir_lower (stmt.rs), examples/vafcodegen_examples/**

**`max`**/**`min`**/**`abs`** in a **`case`** default arm left a block unsealed. `case (V(a,b)) 5.0: y = 11.0; default: y = max(3.0, 7.0); endcase` aborted the compile with "FunctionBuilder finalized, but block N is not sealed". The discriminator is sharp: `pow(2,3)` in the same arm is fine, `max` in an ITEM arm is fine, `max` outside a `case` is fine -- only a branch-lowering builtin in the DEFAULT arm failed. CAUSE: `max`/`min`/`abs` do not lower to a single instruction; they lower through `make_cond` to a real select with its own then/else/merge blocks (`pow` and friends emit one instruction and open no blocks). `lower_case` creates a fall-through block per case item and switches to it, but leaves it to be sealed by an `ensured_sealed()` -- at the top of the next iteration, or, for the LAST item, after the default arm's body is lowered. `ensured_sealed()` seals whatever block the builder is currently positioned in; when the default arm's body opens blocks of its own it leaves the builder on ITS merge block, so the seal lands there (already sealed, a no-op) and the case's fall-through block is never sealed at all. FIX: seal it where it is created -- the branch just emitted is its only predecessor, so all predecessors are known and immediate sealing is correct; the later `ensured_sealed()` calls check `is_sealed` first and become no-ops. Verify (examples/vafcodegen): `casemax.va` compiles AND still picks the right arm (`V=2` -> default `max(3,7)=7`, `V=5` -> item arm `11`); also checked for `min`, `abs`, nested combinations, an integer discriminant, and `casex`. One source file, one statement.

E-292 *sim_back (topology/small_signal_network.rs), examples/vafcodegen_examples/**

small-signal pruning indexed a key its own replay never inserted. A fuzzer input routing noise waves through `idt` into nested `laplace_nd` coefficient arrays aborted the compile with "no entry found for key". CAUSE: `prune_small_signal` moves a linear contribution into its own dimension "where possible" (its own doc comment) -- but whether it IS possible is decided by two different pieces of code: `collect_linear_contributes` classifies the contribution as linear, while the replay inside `create_dimension` is what actually BUILDS the per-dimension value and records it in `val_map`. The replay deliberately declines several shapes (an `fmul` whose BOTH operands depend on the dimension; any opcode falling through to its catch-all), so the two analyses can disagree -- and when they did, `self.val_map[&contribute]` panicked. FIX: treat the disagreement as what it is -- pruning is a best-effort optimization -- and give up on that value instead of crashing. The bail-out path must ALSO run `replace_uses(placeholder, val)`: `prune_small_signal` creates an invalid PLACEHOLDER value before the replay and resolves it at the end, so an early `continue` that skipped it would leave an invalid value in the function. The replay instructions that go unused are dead code the later DCE pass removes. Verify (examples/vafcodegen): `ssprune.va` (the reduced reproducer) compiles where the pre-fix compiler exited 101. Behaviour-preserving wherever the two analyses agree, which is every real model: the full 248-model example corpus compiles release-vs-release with zero regressions, including the noise and RF suites that exercise this pass most heavily. One source file.

E-293 *sim_back (topology/lineralize.rs), examples/vafcodegen_examples/**

one analog operator nested directly inside another. `I(a,b) <+ ddt(ddt(V(a,b)))` crashed the compile -- as did three deep, or split across a variable (`x = ddt(V); I <+ ddt(x);`) -- while putting anything at all in between (`ddt(2.0*ddt(V))`) always worked. CAUSE: `builid_analog_operators` materializes each classified operator; an `Evaluation::Equation` allocates an implicit unknown, calls `replace_uses(res, eq_val)` and then DELETES the operator's instruction, while an `Evaluation::Linear` adds a stored `dimension` value into the contribution's reactive part. Those dimension values live in the `Evaluation::Linear { contributes }` triples -- OUTSIDE the data-flow graph -- and `replace_uses` only rewrites operands held inside the DFG, so it cannot reach them. With direct nesting the stored dimension IS the inner operator's result,

so once the inner operator is processed the outer one holds a value whose defining instruction has been removed (`LINEAR inst1 uses dimension=v17 (def = Result(inst0,0))` where inst0 was just deleted); with an `fmul` in between the replay yields a fresh value (`Result(inst2)`, the multiply) and the situation never arose. Everything derived from the dangling value surfaced later as `invalid argument vN` when the instance-init function was validated. FIX: iterate the operator list by index so an operator can fix up the entries still pending behind it, and retarget any pending dimension naming this operator's result -- `eq_val`, the implicit unknown the inner operator became, is exactly what the outer operator's reactive contribution should use, so this states the correct second-derivative formulation rather than merely removing a dangling reference. The `Evaluation::Dead` arm carries the same hazard and is retargeted to `F_ZERO`; the reverse order is already safe (a `Linear` processed first inserts a genuine DFG use, which a later `Equation`'s `replace_uses` does reach). Verify (examples/vafcodegen): in AC a `ddt` is `j*omega`, so a second derivative is `(j*omega)^2 = -omega^2` -- ‘

E-294 *mir_opt (simplify_cfg.rs, dead_code_aggressive.rs), examples/vafcodegen_examples/**

a **`Branch`->`Jump`** rewrite left the condition in the use list. A `Branch` carries exactly one value operand (its condition); a `Jump` carries none, so every such rewrite must retire that operand's use-list entry. Two of the four sites simply overwrote the instruction (`insts[terminator] = Jump{. .}`), leaving the record linked into the condition value's list naming OPERAND 0 of an instruction that now has ZERO operands -- `use_to_value` then indexes an empty slice (”index out of bounds: the len is 0 but the index is 0”). Offenders: `simplify_bb`'s empty-exit-block rewrite (both arms), and `dead_code_aggressive`'s dead-block terminator rewrite (same defect, found by inspecting the class rather than from a failing model). The two rewrites in `const_fold_terminator` do it correctly -- one via `zap_inst`, one via `detach_operand` -- which is what made the omission legible as an inconsistency. REACHED BY a narrow shape: `\$fatal` exits the analog block so its arm becomes an EMPTY EXIT block, exactly what `simplify_bb` rewrites, and the condition must be a PARAMETER compare (with a node voltage the arm is not an empty exit block and the rewrite never fires; `\$finish` does not produce this shape either) -- one module in the whole corpus reached it. FIX: `zap_inst` the terminator before overwriting, at both sites. HONEST SEVERITY: the stale entry did NOT produce a wrong .osdi and no release build could be made to fail on it -- release never runs the MIR verifier (`debug_assert!`), and the passes that would trip over the record (`replace_uses`, `use_set_value`, whose bounds checks ARE active in release) happen never to touch that value again in any corpus model. A broken invariant with a latent release-crash hazard, not a demonstrated miscompilation -- and the last thing between the compiler and a clean assertions-enabled corpus run, the audit that produced E-286..293. A related latent case is deliberately left alone: `update_inst_uses` retires surplus use records with `truncate`, dropping them from the INSTRUCTION's list without detaching from the VALUE's -- same class, but unreachable today (every caller zaps first, and `attach_use`'s own debug_assert would have fired otherwise), so it is documented rather than patched speculatively. Verify (examples/vafcodegen): `staleuse.va` compiles and simulates; the authoritative check is the assertions-enabled corpus replay -- all 255 models clean, 0 latent, where `simctrl_demo.va` aborted. Release-vs-release 255/255, 0 regressions; cargo test 69/0. Two source files.

E-295 *examples/vafautodiff_examples/*, examples/vafcodegen_examples/

regression guards for the two correctness blind spots (verification-only, no source change). A correctness campaign (~150 oracle checks over parameter storage, multi-terminal Jacobian/capacitance matrices, noise, node collapsing, `\$mfactor`, `\$table_model`, temperature) found ZERO defects; this folds in only the two checks that were genuinely NEW coverage. NOT added because already covered: flicker-noise 1/f and the correlated-noise summation rule (`noise_examples + noisecorr_examples` assert both exactly) and 2-D `\$table_model` (mdtable already checks the DC surface AND both partials). [1] FULL MULTI-TERMINAL MATRICES (`vafautodiff`, 16->18 checks): the suite biased only 2-terminal devices and `[cross]` read ONE off-diagonal, so the entries openvaf does not obtain by differentiating a contribution --

the KCL-derived source row, and the identically-zero row/column of an untouched terminal -- were untested; the new `[matrix]` checks measure all 16 entries of BOTH `dI/dV` and `dQ/dV` (separate reactive code path) on a device whose two contributions are polynomials in three distinct branch voltages at a bias where every branch voltage differs. MUTATION-TESTED: dropping the second product-rule term in mir_autodiff (`d(uv)=u'v`) makes both `[matrix]` checks FAIL (1.6e-1 / 2.2e-1) while `[cross]`, `[regression]` and `[multipoint]` all still PASS -- the new guard has UNIQUE detection power, not just breadth. [2] PARAMETER SLOTS PER INSTANCE (`vafcodegen`, 17->19 checks): E-290 was a wrong struct-GEP offset, and in a 1-2 parameter model such an error lands on the right bytes by luck -- which every prior oracle test was. `paramslots.va` interleaves 13 model+instance parameters of mixed types with distinct non-round values, mirrors each through its own opvar, and 3 instances across 2 model cards give 39 readbacks covering defaults, model card, instance line, instance-overrides-model and cross-instance isolation. MUTATION-TESTED: the slot index is used by BOTH writer and reader so permuting it is a self-consistent renaming and unobservable -- only a reader/writer MISMATCH shows (which is what E-290 was); making `nth_opvar_ptr` read a different slot than eval wrote yields `mp0 = 7.0` (its neighbour `ip0`'s value) and the guard fails. HONEST LIMIT found by that mutation test: E-290 fixed two sites, and only `load_eval_output` is reachable (covered by the existing ac_stim check); `nth_opvar_ptr`'s ParamKind::Temperature arm is NOT reachable -- `ov = \$temperature` lowers to a computed eval-output slot, and a reachability marker compiled into that arm was hit ZERO times across all 326 corpus models -- so it cannot be covered by a runtime test and the suite does not claim to. Regression 237/237. Two example suites, one new model.

E-307 *openvaf/sim_back/src/topology/lineralize.rs*

A **`ddt`** with no contributions crashed the compiler. Found by grammar-based fuzzing aimed at the MIDDLE/BACK end (the parser was already hardened; this generator emits well-typed Verilog-A that compiles, so it reaches MIR/optimizer/autodiff/codegen), run against the assertions-enabled build. lineralize.rs assumed an analog operator reaching the linearizer with an empty contribution list could only be noise: `assert!(noise, "ddt should have been deadcode eliminated")`. False -- a `ddt` whose result never reaches a contribution survives DCE. And it was a PLAIN assert!, not debug_assert!, so the SHIPPED release crashed ("OpenVAF encountered a problem and has crashed!") on valid input. 5 independent seeds of 3000 hit the identical assert; delta-debug + ablation pinned the trigger to ddt + a current probe on a declared (probe-only) branch + if/else + case, in a module contributing nothing -- removing any one stops it. Fix: return `Evaluation::Dead` unconditionally (the branch already taken for noise; its consumer replaces the result with zero and retargets pending uses), so a ddt that feeds no device equation contributes zero. Verify ( examples/vafdeadop): reproducer compiles + .osdi loads + a CONTRIBUTING ddt still gives

E-308 *openvaf/mir_llvm/src/builder.rs*

Uninitialized read feeding a loop-carried phi crashed codegen. Second bug from the same grammar-based middle/back-end fuzz campaign as E-307 (seed 3230 of 8000). A variable read BEFORE a loop that is its only writer leaves the loop-carried phi with an incoming value no reachable block defines: MIR shows `v29 = phi [v18, block7], ...` where v18's defining phi was dropped by a pass on the dead path but the edge kept. Codegen's phi-completion loop then hit `BuilderVal::get()` on a still-`Undef` value -> `unreachable!("attempted to read undefined value")` at builder.rs:143 -- a plain unreachable, so the SHIPPED build crashed on valid Verilog-A. The MIR verifier misses it (its phi check only tests dominance when the def has a block; a detached def passes; and it is debug_assert! anyway). The trigger needs the module to contribute nothing keeping the value live (adding `I(p,n) <+ ra` fixes it). Fix, PROVABLY correct: build_func builds every reachable block before completing phis, so a phi input still Undef names a value NO reachable block defines (a dead path) -> lower it to `cx.const_undef` of the phi's type rather than panicking. Whole-class fix (any optimizer leaving a value that flows only into dead phi edges), not a per-pass chase. Verify (examples/vafuninitloop): reproducer

compiles + a LIVE loop-carried accumulator still reads back exactly N*g (proves undef touches only dead inputs); fails on the pre-fix compiler; 328-model corpus identical pass/fail old vs new; 8000-seed re-fuzz shows 0 of this and the E-307 crash. THIRD, rarer pre-existing ICE surfaced and documented not fixed: packed_option.rs:60 PackedOption::unwrap() on None, ~1/8000, old compiler crashes too.

E-309 *openvaf/mir_opt/src/global_value_numbering.rs*

GVN crashed on a user instruction in an unreachable block. Third and final crash from the same grammar-based middle/back-end fuzz campaign as E-307/E-308 (seed 6716, ~1/8000). When an instruction's congruence class changes, GVN re-queues its USERS via `inst_to_dfs[user].unwrap_unchecked()`. DFSMapping::populate numbers only instructions reachable through cfg_postorder, so a user in an UNREACHABLE block has no DFS id. `unwrap_unchecked` = `if cfg!(debug_assertions) { self.unwrap() } else { self.0 }`, so it panicked at packed_option.rs:60 under debug-assertions and in release returned the reserved sentinel that `touched_insts.insert` used as an OUT-OF-RANGE BitSet index -- the SHIPPED compiler crashed either way ("OpenVAF encountered a problem and has crashed!"). Fix: skip users with no DFS id -- an unnumbered user is in an unreachable block, not in the GVN work list (the solver only iterates dfs_to_inst), so re-queuing it is a no-op -- exactly as `get_rank` in the same file already tolerates the identical None (`if let Some(dfs_id) = inst_to_dfs[inst].expand()`). The two other unwrap_unchecked sites operate on the current instruction (always numbered) and are unchanged. Verify (examples/vafgvnunreach): reproducer compiles + a CSE-heavy model GVN actively optimises still computes exact I = 4*V*g + (V*g)^2 (proves the pass is undisturbed); fails on the pre-fix compiler; 330-model corpus identical pass/fail old vs new. A 12000-seed re-fuzz against the fully-fixed compiler shows 0 of this and the E-307/E-308 crashes -- the three distinct crashes this campaign found are all closed. That deeper run also tripped ONE `debug_assert!(cx.func.validate())` at sim_back/src/lib.rs:175 (seed 11633): NOT a shipped crash (release compiles it fine), but the same assertions-only malformed-MIR class as E-286..E-294, documented for a separate follow-up.

E-310 *openvaf/mir_opt/src/simplify_cfg.rs*

Constant-branch fold left an SSA-invalid phi (the sim_back/lib.rs validate assert). The MIR-validity defect the E-307/308/309 fuzz campaign surfaced, now RESOLVED. const_fold_terminator folds 'br TRUE

E-313 *openvaf/hir_ty/src/inference.rs*

Two builtin argument type-coercion gaps, both emitted silently by the release compiler. Found by a fresh round of the same grammar-based middle/back-end fuzz campaign as E-307..310, run against the assertions build. (a) FILE/STRING FORMAT TASKS were never type-checked. `infere_display` parses the format string and inserts the int->real cast a `%g/%e/%f/%r` conversion needs, but was reached only by the CONSOLE tasks (`\$display/\$strobe/\$write/\$monitor/\$debug` + `\$fatal/\$warning/\$error/\$info`); the FILE tasks (`\$fdisplay/\$fwrite/\$fstrobe/\$fmonitor/\$f`) and STRING tasks (`\$swrite/\$sformat`) were missing from the dispatch match, so their format args were unchecked. A `%g` fed an integer kept its integer value while `print_callback` (osdi/compilation_unit.rs) types the callback parameter as `double` from the conversion -- so lowering passed a raw i32 to a double parameter: INVALID LLVM IR (`Call parameter type does not match function signature! i32 3 / double %35 = call ... @cb.2(..., i32 3)`). The verifier that catches this is a `debug_assert!(llmod.verify_and_print())`, compiled out of release, so release emitted a malformed .osdi whose callback reads the integer's bit pattern as a double -- garbage. Observable: a `\$sformat(s,"%g",5)` -> `\$sscanf(s,"%g",g)` round-trip used as a conductance reads back the denormal 2.47e-323 (bits of the integer 5) instead of 5, so a 5*V device collapses to ~0. Console tasks were unaffected (they carry the cast). Fix: add the file/string format builtins to the infere_display dispatch arm -- infere_display scans for string-LITERAL format strings, so the leading fd (integer) or destination (string variable) argument is naturally skipped and the real format string found; the console path's exact logic now applies to every format*

*task. (b) ddx WITH AN INTEGER ARGUMENT crashed the compiler. infere_ddx requires its first argument (the differentiated value) to be real via self.expect::<false>(expr, None, ty, [Val(Real)]) -- but expect records the needed cast on its FIRST parameter, and it was handed expr (the whole ddx call) while ty is the type of val (the first argument). For an integer val, expect recorded an int->real cast ON THE ddx CALL expression, which already has type Real; needs_cast then computed src=Real (the call's type) and dst=Real (the recorded cast) and tripped debug_assert_ne!(src, dst, \"cast types must be different\") at hir/src/body.rs -- with the assert compiled out, the release build aborted downstream with no .osdi (ddx(n, V(b)), integer n). Fix: record the requirement on val, the argument, not on expr; an integer argument is then coerced to real (ddx of a probe-independent value is 0) and a real argument is unchanged. Both fixes only touch previously-crashing/invalid paths: the whole 419-model corpus produces BYTE-IDENTICAL MIR before and after (deterministic --dump-mir oracle, 0/419 changed), the corpus replays clean through the assertions build, and a 15000-module re-fuzz on the fixed compiler is clean. Verify ( examples/vafargcoerce): 4 checks under both solvers, all failing on the pre-fix binary -- ddx(integer) compiles + simulates to I=1e-3*V, and $sformat("%g",integer) compiles + the round-tripped value is exactly 5. DEFERRED (same campaign, needs a design decision): a provably-infinite analog loop (`while (1) ...`) crashes the compiler via a degenerate no-reachable-exit CFG -- a DCE guard stops the first unwrap but the crash resurfaces in the CFG-validity machinery; left for a dedicated change (reject non-terminating analog loops with a diagnostic, or make the passes tolerate an unreachable exit).

E-314 *openvaf/hir/src/elaborate.rs, openvaf/mir_opt/src/const_eval.rs, openvaf/hir_ty/src/inference.rs*

Constant-evaluation / literal-materialization robustness -- one shipped DoS, two overflow aborts. From the E-307..313 grammar-fuzz family. (a) INTEGER CONST-FOLD OVERFLOW (three sites, one class): two hand-rolled integer const evaluators used UNCHECKED i32 arithmetic. elaborate.rs's Enhancement-91 bus-width folder had checked_mul but its parse_add +/- (`acc += / acc -=`) and parse_unary negate (`-parse_unary`) were unchecked, so `localparam integer k = 2147483647 + 1;` (with any `[...]` declaration present -- the folder is gated on the module text containing '[') overflowed; and mir_opt/const_eval.rs, where Enhancement-286 made Iadd/Isub/Imul wrapping and noted 'eval_unary already used this convention' but MISSED Opcode::Ineg, so negating i32::MIN (from `-(1<<31)`) overflowed. All three aborted the overflow-checked build; the shipped release wrapped silently. Fix: elaborate.rs uses checked_add/checked_sub/checked_neg -- declining the fold on overflow exactly as its * already did, and fold_parameter_widths handles None by leaving the declaration unchanged; const_eval.rs uses val.wrapping_neg(), matching its own wrapping binary ops. (b) UNBOUNDED REPLICATION -> SHIPPED COMPILE-TIME DoS: `{N{...}}` materializes N copies at COMPILE time (infere_concat/lower_string_concat build an N*

E-317 *openvaf/mir_llvm/src/builder.rs*

**idt** initial-condition in a dead branch crashed codegen. From the E-307..314 analog-operator fuzz family. An idt(,*IC) or idtmod placed inside a statically-false branch -- `if (ceil(0) > 1) w = idt(V(a),0);` -- crashed the shipped compiler (exit 101). ceil(0)>1 is always false but ceil() is NOT const-folded, so the dead branch survives into MIR; because w's idt initial-condition state is never used, codegen prunes the branch CONDITION's computation as dead, yet the Branch instruction survives into the derived osdi::setup::setup_instance function. When build_func reaches that branch and reads its condition, the condition's BuilderVal is still Undef, and BuilderVal::get hit unreachable!("attempted to read undefined value") (mir_llvm/builder.rs:143). Only idt/idtmod (integrator accumulator STATE with an IC) trigger it -- ddt/absdelay/transition/slew/laplace*/bare idt(x) are clean. Same Undef-value class as Enhancement-308 (which handled the PHI-INPUT case); this is the BRANCH-CONDITION case. Fix: when a Branch's condition is Undef, lower it as constant false -- the branch only guards dead code (the unused idt-IC state init), so the guarded path never executes on either edge, making this observationally equivalent and avoiding an undefined `br`. Verified corpus-bit-identical (only the reproducer, which was crashing, changes; the other

419 models are byte-identical MIR). Verify (examples/vafidtcfg): reproducer compiles (crashed before) + simulates to a finite op (i(v1)=-5e-4). DEFERRED from the same hunt: an assertions-only PHI-type mismatch (an uninitialized integer variable's default materialised as f64 F_ZERO -> i32-result phi with an f64 operand; release folds both to 0, correct) and a ngspice .tran convergence livelock on a specific fuzzer OSDI device (tiny 1e-15 cap + clamped exp diode + tanh + noise) -- both left for dedicated changes.

E-324 *openvaf/hir_lower/src/expr.rs*

**`\$fatal`** stranded code in an unreachable block -- TWO shipped-compiler crashes, one root cause. Found by a diverse-strategy fuzz campaign against the assertions build (~75000 generated well-typed models, 7 generation strategies); both crashes reproduce on the SHIPPED release, not just the instrumented build. `analog begin \$fatal(0); I(a,c) <+ V(a,c)/1k; end` panicked at mir_opt/src/dead_code_aggressive.rs:112 (`Option::unwrap()` on `None` from `self.func.layout.inst_block(inst).unwrap()` in mark_inst_live), and the mirror image `analog begin I(a,c) <+ V(a,c)/1k; \$fatal(0); end` panicked at mir_llvm/src/builder.rs:143 (`unreachable!("attempted to read undefined value")` in BuilderVal::get, reached from store_residual). ROOT: `\$fatal` lowered to print -> set the abort flag -> `exit()` -> create a fresh block with NO incoming edges -> keep lowering into it, on the assumption (stated in its own comment) that an edge-less block is simply removed from MIR. That is unsound for a compiled device: every ret-flag (RetFlag::Abort/Finish/Stop) is only a flag the simulator inspects AFTER the eval function returns -- all three lower to `set_ret_flag_*` callbacks in osdi/src/compilation_unit.rs -- so none can jump out of the middle of an evaluation, and the OSDI eval function has a MANDATORY epilogue (store residual/jacobian) the ABI requires to run. Terminating the MIR function early therefore stranded work in the predecessor-less block: a statement AFTER `\$fatal` was lowered there yet stayed referenced by the contribution bookkeeping (values held OUTSIDE the DFG are not reached by CFG cleanup -- the E-294 class), so aggressive DCE marked live an instruction belonging to no block; and with a statement BEFORE it the EPILOGUE ITSELF was emitted there, where the residual value does not dominate, so codegen read an Undef. `\$finish` and `\$stop` were already lowered correctly as set-flag-and-continue (no exit(), no unreachable block) -- `\$fatal` was the lone outlier. FIX: `\$fatal` sets its flag and falls through exactly like `\$finish`/`\$stop`; the CFG stays connected so neither failure mode can arise (one hunk, one file). Measured trigger surface: crashes required an UNCONDITIONAL `\$fatal` plus a contribution in the module -- a conditional `if(..) \$fatal(0);` never crashed (the join block is still reachable via the else path), nor did `\$fatal` with no contribution, nor `\$finish`/`\$stop` in any position. BEHAVIOUR UNCHANGED: `\$fatal`'s run-time meaning lives entirely in the set_ret_flag_fatal callback, untouched -- simctrl_examples (E-55) passes in full including "message printed", "abort error printed", "transient aborted early" and "parameter-only $fatal: setup rejected"; vafcodegen_examples (E-294) 19/19 including the parameter-guarded `\$fatal` arm which still SIMULATES correctly (I=V/1k); and a realistic guard (`if (r<=0) \$fatal(..)` followed by a `V/r` divide) still aborts cleanly at setup with its message and no NaN leaking. The one semantic difference -- statements after `\$fatal` now execute instead of being dead -- is exactly what `\$finish`/`\$stop` have always done and is not observable, since the simulator aborts as soon as eval returns. OUTPUT-PRESERVING: `--dump-mir` (deterministic, unlike .osdi bytes) over the full 462-model corpus is BYTE-IDENTICAL for 460; the only two that differ are precisely the models using `\$fatal` in a reachable analog context (simctrl_demo.va, staleuse.va), both of whose suites pass in full, and the two hisim2 industry models that also use `\$fatal` are unchanged. Verify (examples/vaffatalcfg): the two crash shapes compile (both fail on the pre-fix binary with a compiler panic and no .osdi) plus run-time behaviour guards; 6 checks.

E-325 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs*

Bound the MATERIALIZED SIZE of **`{...}`**/**`{n{...}}`**, not just the replication count -- a shipped compile-time HANG and a shipped silent WRAP. Enhancement-314 capped the replication COUNT at 2^20 after a `{'d999999999{"x"}}` DoS, but the count is only ONE FACTOR of the fi-

nal size, so two abusive shapes still reached the shipped compiler (found by the 7-strategy ~75000-model fuzz campaign). (a) STRING replication becomes LLVM FUNCTION ARITY: `lower_string_concat` builds an `elems.len()*rep_cnt` operand list plus a format string with that many `%s`, and `print_callback` turns it into a generated callback with ONE PARAMETER PER OPERAND. LLVM degrades super-linearly in arity -- measured on this compiler: 2000 operands 0.4s, 8000 2.9s, 16000 8.6s, 32000 never finished -- so `parameter string s = {200000{"x"}};` HUNG the compiler on one line of source (both binaries). NOTE the original hypothesis ( quadratic string interning in the const-folder) was WRONG: the const-fold is linear and negligible; the entire cost is in LLVM. (b) The size arithmetic OVERFLOWED u32: `len: total * rep_cnt` was an unchecked multiply (and the running `total += len` equally unchecked), so `real c[0:1]; c = {1048576{{1048576{1.0}}}};` -- where BOTH counts are individually legal ( exactly MAX_REP) -- computed 2^20*2^20 = 2^40, which panicked under overflow-checks and in the SHIPPED release silently WRAPPED TO 0, emitting the nonsense diagnostic `expected real[0:2] value but found real[0:0] value` instead of a real error. FIX: compute the size in u64 with saturating add/mul and bound it BEFORE narrowing to the u32 array length, with a dedicated `ConcatTooLarge` diagnostic that reports the TRUE expanded size (`expands to 1099511627776 elements` = 2^40, where the u32 path had wrapped to zero). Reusing the existing InvalidReplicationCount diagnostic would have been MISLEADING -- in case (b) the count IS a valid positive integer literal; it is the product that is too large. TWO limits, because the two paths have genuinely different measured cost profiles: MAX_CONCAT_ELEMS = 2^20 for numeric arrays (that path is linear and cheap -- 65536 elements compile in 0.41s -- so this only rules out the absurd and guards the u32 length) and MAX_CONCAT_STR_OPERANDS = 4096 for strings (arity is the real cost; 4096 keeps the worst case near a second, orders of magnitude above any legitimate literal). LRM POSITION: `{n{...}}` over strings/1-D arrays is this project's Enhancement-34 EXTENSION -- Verilog-A (LRM Annex C) has no vector concatenation at all and Verilog-AMS defines `{}`/`{n{}}` over bit vectors -- so no LRM-mandated semantics is violated by a documented expansion limit (the precedent E-314 set); and for (b) any expansion larger than the destination is a TYPE ERROR today, the only defect being that the compiler computed the size wrongly before it could say so. No legal program changes meaning. OUTPUT-PRESERVING: u64 saturating add/mul agree exactly with the old u32 wrapping arithmetic for every value below 2^32 and both caps are checked before the narrowing, so every model whose concatenation fits takes a bit-identical path -- confirmed with the deterministic `--dump-mir` oracle over the 465-model corpus: 464 byte-identical, and the single reported diff (lrm_p150_1.va, which contains NO concatenation at all) was PROVEN to be run-to-run nondeterminism of the multi-module dump order, not an effect of the change (the same binary on the same input produced both hashes across 5 runs). Verify (examples/vafconcatsize): the three abusive shapes are rejected cleanly (they hang or misreport on the pre-fix binary), the diagnostic reports the true 2^40 size, and legitimate concatenation still compiles AND simulates (a replicated array element drives i(v1) = -2 mA exactly); 6 checks.

E-326 *openvaf/sim_back/src/init.rs*

A cross-namespace **`mir::Value`** comparison mis-typed init cache slots -- the shipped compiler SIGSEGV'd inside LLVM. Highest-severity finding of the 7-strategy ~75000-model fuzz campaign: on a legal Verilog-A model the RELEASE binary died with EXC_BAD_ACCESS in `llvm::detail::DoubleAPFloat::multiply` (signal 11, no diagnostic); the assertions build instead had its LLVM verifier reject the module with `fmul reassoc .. i8 %8, double %6` and `trunc double %45 to i8`, and at -O0 release gave `LLVM ERROR: Do not know how to promote this operator!`. A boolean-typed (i8) value was reaching floating-point arithmetic. ROOT: NOT a missing cast, NOT autodiff, NOT casex. A `mir::Value` is a bare u32 index and the MAIN function and the INIT function each number their values from zero. `build_init_itern` inserts `self.val_map[&val]` -- an INIT-namespace value -- into `collapse_implicit`, which is correct for its other consumer `optimize()` (that runs DCE over `self.init.func`); but `build_init_cache` iterates `self.init_cache`, whose keys are MAIN-namespace values, and tested them against that same

set (`collapse_implicit.contains(&val)`) both in its dead-value filter and in its type selection. Comparing indices across two namespaces cannot fail loudly -- it silently SUCCEEDS whenever the two counters coincide. Confirmed under lldb on the minimized reproducer: the producer inserted init `Value(25)` (the `idt` collapse flag, a genuine Bool) while the consumer tested main `Value(25)`, which is `abs(p)` -- the NOISE POWER, a Real. Same number, unrelated values. CONSEQUENCE: a colliding f64 slot was recorded as `Type::Bool`, which lowers to `ty_c_bool()` = i8. The store side (`store_cache_slot`) then int-cast a slot it believed boolean, emitting `trunc double .. to i8`; and the noise loader reads `EvalOutput::Cache` RAW with the slot type -- unlike `load_cache_slot` it performs no bool normalization -- so the i8 became `base_pwr` and landed in `fmul i8 %x, double %y`. LLVM constant-folded that malformed fmul and read the i8 as an APFloat: the EXC_BAD_ACCESS. FIX: map through `val_map` before the lookup so both sides speak the same namespace -- 'let is_collapse_flag = self.val_map.get(&val).map_or(false,

E-327 *openvaf/hir_lower/src/{expr.rs,ctx.rs,lib.rs}*

**`ddx`** unknowns that do not lower to a bare MIR **`Param`** crashed the shipped compiler -- the bug was sitting behind a **`TODO`**. From the 7-strategy fuzz campaign. `ddx` lowering assumed its UNKNOWN argument always lowers to a bare `Param` and unwrapped one unconditionally: the DDX_POT arm did `let node = self.ctx.unwrap_node(unknown);` (itself `value_def(val).unwrap_param( )`) directly under a `// TODO how to handle gnd nodes?` comment, and the other arm did `CallBackKind::Derivative(self.ctx.dfg().value_def(unknown).unwrap_param())`. That assumption is false: `LoweringCtx::nodes` yields THREE shapes -- a `Param` for a forward-oriented probe (`V(a,b)`, `V(a)`); an **`fneg(param)`** INSTRUCTION for a REVERSE-oriented probe (`V(b,a)`, or one whose high side is ground, via either the `(None, Some(lo))` arm or the inverted-param arm); and **`F_ZERO`**, a constant, when the probe is ground only. The latter two hit `unwrap_param()` and panicked `Value is not a parameter` (mir/src/dfg/values.rs) on BOTH binaries -- a shipped crash on legal input. DECISION -- COMPILE, do not reject: both extra shapes have an unambiguous derivative. `V(b,a)` denotes the SAME branch with the opposite reference direction, i.e. `V(b,a) == -V(a,b)`, so `df/dV(b,a) == -(df/dV(a,b))`; and ground is not an unknown of the DAE system, so `df/dV(gnd) == 0` -- exactly what the backend already does for a derivative callback that never became an unknown. Erroring would also be incoherent: openvaf already ACCEPTS `V(a,b)` as a ddx unknown (an extension beyond the LRM's single-net/branch-flow rule, announced by its own L011 lint), so having accepted `V(a,b)` it cannot reject `V(b,a)`. FIX: peel an `fneg` off the unknown (recording that the result must be negated), then ASK rather than assert whether what remains is a parameter -- `match self.ctx.dfg().value_def(probe).as_param() { None => F_ZERO, Some(param) => ... }` -- negating the callback's result when an fneg was peeled; plus a non-panicking `ParamKind::pot_node()` replacing `unwrap_pot_node()` on the DDX_POT path and a `LoweringCtx::param_kind()` accessor so lowering can inspect a user-supplied unknown instead of asserting its shape. VERIFIED NUMERICALLY, not merely that it compiles: differentiating `V^2` (exact derivative 2V) at V=3 through a 1 mS scale gives i = -6.00000e-03 for the forward unknown `V(a,b)`, i = +6.000000e-03 for the reverse unknown `V(b,a)` -- the EXACT negative to machine precision -- and -3.00000e-09 for the ground unknown, which is purely the model's explicit 1e-9*V leak term, i.e. the ddx contributed exactly zero. OUTPUT-PRESERVING: the new code diverges from the old only when `value_def(unknown)` is NOT a `Param`, which is precisely the case that previously reached `unwrap_param()`/`unwrap_node()` and panicked, producing no output at all; when the unknown IS a Param (every model that compiles today) the fneg peel does not fire and the callback construction is unchanged. Confirmed with the deterministic `--dump-mir` oracle over the corpus: 0 changed. Verify (examples/vafddxunknown): the two crashing shapes compile and their derivatives are checked against the closed form; 4 checks.

E-328 *openvaf/hir/src/body.rs, openvaf/hir_lower/src/expr.rs*

**`BodyRef::get_expr`** was a PARTIAL function used as if it were total -- a dynamic array index in a contribution crashed the shipped compiler. `get_expr` funnelled every `BitSelect` into `re-`

solve_path, which can only resolve expressions that have a Ref (Ty::Var, Ty::Param, a function var/return, a nature attribute) and panic!s otherwise. A dynamically-indexed array read has NO backing variable: inference types it Ty::Val(..) and records the element variables, per-dimension bounds and index expressions OUT-OF-BAND in dynamic_index_refs. So any caller that merely probed an expression's SHAPE -- a literal-zero test, a literal-condition fold, an aggregate check -- crashed on legal input with invalid HIR: path BitSelect { .. } was not resolved Val(Real). The asymmetry is the tell: x = g[k]; I <+ V*x; compiled while I <+ V*g[k]; panicked -- lower_expr short-circuits on dynamic_index() BEFORE consulting get_expr, and the contribution path has no such short-circuit. (Note an earlier analysis reported this shape as already working; re-verification on the shipped binary showed it still crashed in both the uninitialised-index and assigned-index spellings -- worth re-testing rather than trusting.) FIX: restore the invariant that get_expr is TOTAL -- add an Expr::DynIndexRead variant and answer the shape question directly, gated on the same dynamic_index_refs map the value path already consults. The VALUE is still lowered by lower_expr's existing dynamic_index() short-circuit, so nothing about code generation changes. Making the enum non-exhaustive pointed the compiler at EXACTLY ONE match needing an update (lower_expr's, which already short-circuits and so gets an unreachable! arm), confirming how tightly scoped the change is. VERIFIED NUMERICALLY: a dynamic index must select the RIGHT element, not merely stop crashing -- four instances selecting g = {1,2,3,4} mS at V=1 give exactly -1.00000e-03/-2.00000e-03/-3.00000e-03/-4.00000e-03, agreeing bit-for-bit with the x = g[k] spelling that always worked. OUTPUT-PRESERVING: the new branch is gated on dynamic_index_refs.contains_key(&expr); for any expression not in that map -- every expression in every model that compiles today -- get_expr executes exactly the previous code path, and for an expression that IS in it the previous behaviour was a panic, so there is nothing to preserve. Confirmed with the deterministic --dump-mir oracle. Verify (examples/vafdynidx): 3 checks.

E-329 *openvaf/sim_back/src/topology/small_signal_network.rs*

A GRAVESTONE phi operand crashed the small-signal network builder. ddt of a NEGATED flow probe plus a statically-false branch containing a loop -- r0 = ddt(-I(a,b)); if ((1*s0)-s0) begin lc3=0; while (lc3<4) lc3=lc3+1; end I(b,c) <+ r0*r1; -- panicked the SHIPPED compiler with internal error: entered unreachable code. ROOT: the value arriving is GRAVESTONE, Value(0), whose ValueDef is Invalid. It is the compiler's OWN placeholder, declared in mir/src/dfg/values.rs as "place holder for unused values that must remain (in phis)": the SSA re-builder puts it in a phi for an edge with NO reaching definition -- an edge from a block unreachable from the entry that simplify_cfg cannot delete. The small-signal network builder asserted such a value could never reach it, with ValueDef::Invalid => unreachable!() in BOTH analyze_value and analyze_dependency. FIX: a GRAVESTONE operand sits on a dead edge and therefore cannot be used at run time, so it contributes nothing -- analyze_value returns FlatSet::Zero (the same answer its neighbouring F_ZERO arm gives) and analyze_dependency returns Dependency::Independent (the same answer its 'Param(_)

E-330 *openvaf/hir_ty/src/validation/body.rs*

**ddx** in a runtime loop HUNG the shipped compiler forever -- a fixpoint that grows its own lattice. The last of the seven shipped crashes from the diverse-strategy fuzz campaign, and the only infinite loop rather than a panic: for (i=0;i<1;i=i+1) x = ddx(V(a)*x, V(a)); never returned on either binary. ROOT: live_derivative_fixpoint (mir_autodiff/src/live_derivatives.rs) asks, for every derivative already live at a ddx call, for one of ORDER+1 via raise_order_with. That is fine on a DAG -- the order chain is bounded by the number of ddx sites on the longest path. A loop back edge closes the circuit: the differentiated expression V(a)*x depends on x, which is the loop-carried phi fed by the ddx result itself, so round n creates order n+1 and interns it, populate_reachable marks it reachable at that instruction, the changed live set requeues the argument's defining instructions, and the back edge carries the new order around the phi back into the ddx's own input -- round n+1 begins. A monotone fixpoint terminates because

its lattice is FIXED; here the fixpoint GROWS THE VERY LATTICE IT ITERATES OVER, so it has no fixed point. Measured not inferred: `sample` puts 99.8% of 3944 samples in that one chain ( raise_order_with -> TiSet::ensure -> IndexMap::insert_full), RSS climbs monotonically and never plateaus, and the process was still running after 15 minutes -- true non-termination, not slowness. DECISION -- clean error, not a cap: ddx is SYMBOLIC and MEMORYLESS (openvaf computes it as a derivative callback on the MIR; its result is determined by the SHAPE of its argument). Inside a runtime loop `x = ddx(V(a)*x, V(a))` asks for a different symbolic form every trip -- iteration k needs the k-th derivative and the trip count is not a compile-time constant -- so THERE IS NO FINITE MIR THAT IMPLEMENTS IT, which is exactly why the fixpoint diverges rather than converging to something wrong. Ill-formed, not legal-but-unbounded. This is also what the language already says and what the compiler already does for every OTHER analog operator: ddx is classified as an analog operator by openvaf itself (hir_def/src/builtin.rs is_analog_operator()) and LRM 4.5.1 forbids analog operators in non-genvar loops -- `for(...) x = ddt(V(a));` and `for(...) x = idt(V(a)*x);` already produce `analog operator ... is not allowed in loops`. The divergence came from one explicit escape hatch in hir_ty/src/validation/body.rs, `_ if call.is_analog_operator() && call != BuiltIn::ddx`, which is CORRECT for conditionals (the industry CMC corpus has 192 ddx call sites inside `if` bodies across 19 models -- BSIM-BULK, BSIM-SOI, HICUM, ASM-HEMT, L-UTSOI) but too broad, since it also covers loops. FIX: track runtime-loop nesting in a dedicated `loop_depth` counter and reject ddx there, reusing the existing IllegalCtxAccess diagnostic so it is an ordinary compile error (exit 65, no new machinery). A separate counter is required rather than reusing `ctx`: `validate_condition_in` REPLACES the context instead of stacking it, so an `if` nested in a `for` resets it to Conditional, and it only becomes Loop when the controlling expression is non-constant, so `repeat(3)` would slip through. Trigger surface verified: `for`, `while` and `repeat` all hung, as did a ddx under an intervening `if` and one routed through a second variable; the multiplication is NOT essential (`ddx(V(a)+x, V(a))` hangs too) -- the precise condition is that the argument depends on the differentiation unknown AND on the ddx call's own result through a back edge. SCOPE, STATED PLAINLY: ddx outside a loop is untouched, including inside `if/else`, and is still numerically exact there (d/dV(V^2)=2V gives -6 mA at V=3). A corpus scan of 514 models finds 0 of 755 ddx call sites inside a loop body, and the --dump-mir oracle reports no corpus model changed. This is nonetheless the ONE fix in this series that slightly NARROWS the accepted language: a loop containing a ddx with no self-reference (`for(...) g = g + ddx(V(a)*V(a), V(a));`) compiles today and now errors. It is absent from the corpus, the rejection is LRM-conformant, and it is exactly the treatment ddt/idt/transition/laplace_* already receive -- but it is not strictly output-preserving and is not claimed to be; keeping it would require front-end dataflow that can distinguish self-referential from not, which the validator does not have. SEPARATELY DISCOVERED, NOT FIXED HERE: while bounding the derivative order a SECOND shipped crash surfaced -- 65 nested ddx calls panic in lib/bitset/src/lib.rs (`index out of bounds: the len is 1 but the index is 1`) via HybridBitSet::contains, reached from populate_reachable; a row that has gone dense keeps the word count it had at that moment, so once the derivative universe grows past 64 (one word) a query on the new index panics. Independent of loops and of this fix, left for a dedicated change. Verify (examples/vafddxloop): the hanging shape is now a prompt error citing the LRM, and ddx outside a loop still compiles AND stays exact; 4 checks.

E-331 *lib/bitset/src/lib.rs*

**`BitSet::contains`** panicked outside its domain -- a representation leaking out as a compiler crash. Found while bounding the derivative order for E-330: deeply nested `ddx` crashed the SHIPPED compiler with `index out of bounds: the len is 1 but the index is 1` at lib/bitset/src/lib.rs:114, via `BitSet::contains` <- `HybridBitSet::contains` <- `mir_autodiff::populate_reachable`. ROOT: `contains` indexed its backing storage with no bounds guard -- `(self.words[word_index] & mask) != 0`. A `HybridBitSet` stores a set either SPARSELY (a Vec of elements) or DENSELY (a bit array), switching as a size optimisation, and rows grow their domain only when something is inserted into them -- so a row the traversal never inserts

into keeps whatever width it had when it went dense. Once the live-derivative universe grew past 64, i.e. ONE WORD, a query for a higher element indexed off the end. The decisive detail is the ASYMMETRY between the two representations: `SparseBitSet::contains` searches a Vec and returns `false` for any absent element, while the dense `BitSet::contains` panics -- so the SAME logical query on the SAME logical set returned false while the set happened to be sparse and CRASHED once it happened to be dense. The representation, the very detail HybridBitSet exists to hide, leaked out as a crash. FIX: make the dense query total, matching what the sparse one already does -- 'self.words.get(word_index).map_or(false,

E-332 *openvaf/sim_back/src/topology/builder.rs, openvaf/hir_ty/src/validation/body.rs*

Summing THREE OR MORE **`ddt()`** terms into one contribution silently DROPPED CHARGE -- a wrong number, not a crash. `I(a,b) <+ ddt(V)+ddt(V)+ddt(V)` compiled cleanly and behaved as a 1 F capacitor instead of 3 F; N=1 and N=2 were correct, which is why it survived. Reachable from entirely ordinary code: a `generate` loop is unrolled at compile time, so `generate k (0,2) s = s + ddt(V);` is the same expression and was equally wrong. ROOT CAUSE was NOT in `ddt` handling but in TRAVERSAL ORDER. `create_dimension` replays the instructions consuming an analog operator's result to build its linear coefficient, and its Fadd/Fsub arms '(None, Some(x))

E-333 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs*

Integer division by a LITERAL zero compiled with exit 0 and no diagnostic, then killed ngspice with SIGTRAP and no output at all. `if ((5 / 0) > 0 && ione > 0) ...` -- LLVM treats `sdiv x, 0` as IMMEDIATE UNDEFINED BEHAVIOUR: the result folds to poison, poison reaching a branch becomes `unreachable`, and `unreachable` lowers to a `brk`, so the whole function became a trap and the first call into the compiled .osdi killed the host process. The short-circuit && is what made it reachable -- with a RUNTIME right operand the division lands in a conditionally-executed block where folding never collapses it, so it survives into code generation; the same expression without the && folded away, which is why it looked harmless. Every spelling was in fact undefined and only some trapped: before the fix `V(o) <+ (5/0) > 0 ? 1.0 : 2.0;` printed 0, which is not that expression's value but poison propagating. WHY E-286 LEFT IT: E-286 fixed a COMPILER crash here (folding 5/0 evaluated the division inside openvaf and killed it with an internal error) by declining to fold, reasoning that "a runtime zero divisor has always been accepted, so a literal one must be too". BOTH HALVES ARE WRONG -- (1) the literal case is not the runtime case, since only the literal one leaves a constant-zero divisor in the IR for the optimiser to exploit (verified: a parameter, a localparam and a derived constant `3-3` are all lowered as RUNTIME values and none of them traps, so the literal is the entire UB surface); (2) runtime acceptance is TARGET-SPECIFIC -- AArch64 returns a value where x86 raises SIGFPE, and this project ships x86 builds for macOS, Linux and Windows plus riscv64 among its targets, so there is no portable value to fold to. E-286 thus converted a compiler crash into a simulator crash. FIX: reject it, since no value can be defended -- a clean `error: integer division by zero` naming the zero and stating that a parameter/localparam zero is still accepted. The check is LITERAL-ONLY by design: that is exactly the set of programs reaching LLVM with a constant-zero divisor, and widening it to any folded zero would reject working models for no safety benefit. VERIFIED: every previously trapping or undefined spelling (inside &&, inside '

E-334 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs*

E-333 was INCOMPLETE: the same SIGTRAP survived for **`i32::MIN / -1`** and for an out-of-range shift distance. `if ((-2147483647 - 1) / (-1) > 0 && ione > 0) ...` and `if ((1 << 40) > 0 && ione > 0) ...` both compiled with exit 0 and no diagnostic, then killed ngspice with signal 5 and no output -- identical mechanism to the zero divisor E-333 fixed, only the operation differs. HOW IT WAS MISSED: Enhancement-286's own comment named ALL THREE cases ("a zero divisor -- and i32::MIN / -1, which overflows", "a shift distance outside 0..32 is poison in LLVM") and declined to fold each, which is exactly what leaves the poison in the IR for the optimiser to turn into `unreachable` -> `brk`. E-333 read that comment, fixed the divisor, and did not check whether

the other two it named had the same consequence. When a comment enumerates several cases sharing a mechanism, fixing one is not evidence about the others. FIX: the same front-end check extended to both operations, with a diagnostic each ("integer division overflows", "shift distance out of range" naming the distance). E-333's check tested for a LITERAL, which is insufficient here: i32::MIN CANNOT be written as an integer literal -- `-2147483648` exceeds i32::MAX and promotes to REAL, making E-286's `u = -2147483648 / -1` a real division rather than the integer overflow it was believed to test. The overflow is only reachable as `(-2147483647 - 1)`, so the check now folds constant integer expressions (literals plus + - * over them, wrapping arithmetic, bounded recursion) -- exactly the set the code generator sees as constant. It stays CONSTANT-OPERAND-ONLY: a parameter or localparam is a runtime value, never a constant operand in the IR, and rejecting those would break working models. VERIFIED: every previously trapping shape is now a clean exit 65 -- i32::MIN/-1, the same with %, 1«32, 1«40, (-1)»32, 1«(-1), and 1«(4*8) (which needs the folding, not just a literal); and everything legal still compiles AND simulates -- 1«31, 1«0, a runtime shift distance, parameter/localparam zero divisors, (-2147483647-1)/2, ordinary division and remainder, I = V/1k exactly. CORPUS: 480 models, ONE changes accept/reject -- E-286's constfold.va, whose `1 << 40` moved to the new negative test and was replaced by the largest LEGAL distance (1«31); its `-2147483648 / -1` line stays with a comment recording that it is a REAL division and never tested the integer overflow it was written for. STILL OPEN, not addressed: at RUNTIME an out-of-range shift distance is silently masked to 5 bits (`1 << n` with n=32 yields 1 where Verilog requires 0) -- a wrong answer rather than a trap, needing a codegen change rather than a front-end check. Verify (examples/vafintub): 6 checks.

E-335 *openvaf/mir_llvm/src/builder.rs, openvaf/mir_opt/src/simplify.rs*

Three IEEE 754 / IEEE 1364 semantics the compiler was not honouring -- silent wrong answers on ordinary code, no crash. (1) `!=` ON REALS WAS AN ORDERED COMPARE: `Fne` lowered to LLVMRealONE, which is FALSE whenever an operand is NaN, so `x != x` -- the canonical isnan idiom -- reported "not NaN", and `a != b` was NOT the complement of `a == b` on the same operands. `Feq` correctly stays ORDERED (OEQ, so NaN==NaN is false); the two are complements again only once Fne is UNORDERED (UNE). (2) OUT-OF-RANGE SHIFT DISTANCE LEFT TO THE HARDWARE: a Verilog-A integer is 32 bits so IEEE 1364 requires a distance >= 32 to give 0 (the right operand is unsigned, so a negative distance is huge and also gives 0), but openvaf emitted a bare shl/lshr/ashr and AArch64 masks the distance to 5 bits -- `1 << n` with n=32 gave 1 (i.e. 1«0) and `(-1) >> n` gave -1. Only the RUNTIME form was affected (a literal distance is poison, fixed in E-334), so the two spellings disagreed with each other as well as with the language. Now range-checked in codegen: « and » select 0 when the unsigned distance is >= 32, »> clamps to 31 which yields exactly the all-sign-bits result; ordinary shifts untouched. (3) THE SIMPLIFIER APPLIED ALGEBRA IEEE DOUBLES DO NOT OBEY: `Arithmetic for f64` set DIV_EXACT and HAS_SQRT true, enabling rewrites its own comment called "only fast math". With NODE VOLTAGES as operands these fired at run time -- `V(z)/V(z)` at z=0 gave 1, `sqrt(V(w))*sqrt(V(w))` and `exp(ln(V(w)))` at w=-4 gave -4, where IEEE requires NaN; `x-x`, `x+(-x)`, `x*0` and `x/-x` are unsound for the same reason. Each is exactly how a model guards a domain, so folding them replaced a deliberate NaN with a plausible wrong number. Gated on a new EXACT_ALGEBRA constant: true for i32 which really does obey algebra, false for f64, naming the property at each site instead of implying it from the type. Inverse-function cancellation tightened the same way -- the principal-value cases were already excluded, the DOMAIN/OVERFLOW ones were not (exp(ln x) for x<0, ln(exp x) and sinh/asinh on overflow, cosh(acosh x) for x<1, tanh(atanh x) for

E-336 *ngspice-46/src/osdi/osdiregistry.c, ngspice-46/src/osdi/osdiinit.c, openvaf/osdi/src/metadata.rs*

Three defects in how a compiled model is DESCRIBED to, and BOUND by, the simulator -- wrong values and a wrong array size, no crash. (1) AN INSTANCE PARAMETER NAMED `M` WAS CONSUMED AS THE MULTIPLIER: with `(* type="instance" *) parameter real M = 2.0` and `I <+ V*M`, an instance line `M=7` produced i(va)=14 = 7 x (1V x 2) -- applied as the DEVICE MULTIPLIER while the model's own M kept its default. ngspice decided whether to synthesize

its `m` alias for $mfactor with a case-SENSITIVE strcmp(name,"m"), so a model declaring `M` never suppressed it, while that same M was lowercased to `m` on registration; both became `m` and the alias won. The inconsistency was visible in the same loop -- the dtemp/dt/temp tests two lines below already used strcasecmp. Now `m` does too and M=7 reaches the model's own parameter ( i(va)=7). (2) PARAMETERS DIFFERING ONLY IN CASE SILENTLY LOST A VALUE: Verilog-A is case-SENSITIVE so `GAIN` and `gain` are distinct, SPICE is not, so both fold to one keyword and one loses -- `GAIN=3 gain=7` yielded 7.001, i.e. both routed to one parameter (last wins) while the other kept its default 1000. This CANNOT be resolved in the loader (a netlist is lowercased when parsed, so the names are indistinguishable by the time a value arrives) but it must not be SILENT: it now warns, naming the module and parameter, so the author learns the names are unreachable instead of debugging a wrong answer. Binding behaviour deliberately unchanged; only the silence is fixed. (3) num_resistive_jacobian_entries EXCEEDED THE ENTRY LIST: 8 against num_jacobian_entries=7 -- a count larger than the array it describes, which a consumer trusting it would use to walk past the end. num_jacobian_entries and the entry list both come from the CURRENT dae_system.jacobian, but the resistive/reactive counts came from a value cached by count_jacobian_entries() while the DAE system was still being built, and the jacobian can lose entries afterwards (node collapsing), leaving the cached number stale. Both counts are now derived from the same map that produces the entry list -- consistent by construction rather than by timing. VERIFIED: an independent OSDI descriptor reader reports 0 violations on the two models that previously showed 2 and 1, and still 0 on ordinary models, so the checker is specific rather than blanket-passing. Verify (examples/osdiparam): 3 checks.

E-337 *openvaf/mir_opt/src/simplify.rs*

E-335 went too far: removing **`x * 0 -> 0`** changed HiSIM2's DC drain current by 10x, and the 92-model corpus campaign caught it. E-335 removed the IEEE-unsound algebraic rewrites; `x*0 -> 0` belongs to that family (`inf*0` and `NaN*0` are NaN, not 0) and went with them. Diffing the campaign against the pre-E-335 toolchain showed ONE of 92 device-modules had moved: hisim2_ va, Id -1.30e-04 -> -1.33e-05 at Vg=0.7/Vd=1.0. Both Id-Vg curves were smooth and monotonic, so nothing looked broken -- which is why a DIFFERENTIAL was needed rather than an eyeball. ISOLATION: reverting the E-335 gates one at a time identified `x*0` as the sole cause -- restoring only that gate restored the value exactly, restoring any other did not. THAT IS ITSELF THE DIAGNOSIS: `x*0` is EXACT for every finite x, so removing the fold could only change a result if the operand were inf or NaN. HiSIM2 produces a non-finite intermediate at that bias which the fold had been silently absorbing. WHAT ACTUALLY TRIGGERS IT (two reproducer attempts missed this): one operand must be the interned CONSTANT zero and the other a RUNTIME value; if BOTH are constant, const_eval folds `0*inf` to NaN first (correctly) and the rewrite never applies; and a zero-valued PARAMETER is a runtime value, so `flag * term` with a zero-valued parameter flag does not trigger it either. The guarded case is a constant-zero coefficient multiplying a runtime non-finite term. DECISION: the fold stays, gated separately from EXACT_ALGEBRA and documented at the site. There is no evidence the un-folded answer is physically correct -- only that it differs -- and changing a production model's DC current by 10x for IEEE purity, on a fold whose unsoundness requires a non-finite operand the model was never expected to produce, is not a trade worth making. If that non-finite intermediate is itself a model defect, that is a finding about the model, not a reason for the compiler to change its answer. The rewrites that produced the REPORTED wrong answers (x/x -> 1, sqrt(x)*sqrt(x) -> x, exp(ln x) -> x, and the domain/overflow inverse cancellations) remain removed. VERIFIED: 92/92 corpus device-modules OK, worst KCL residual 2.3e-13 A; against the pre-E-335 toolchain NO device-module has a different drain current, and the only three differing rows differ solely in KCL residual at 1e-16/1e-17 -- orders of magnitude below the worst case and pure floating-point noise from the folds legitimately removed. All E-335 fixes survive. Verify (examples/vafmulzero): 2 checks.

E-340 *openvaf/hir/src/elaborate.rs*

Compiling the same source twice produced two different MIRs -- lrm_p150_1.va gave 2 distinct

and lrm_p209_1.va gave 8, run to run on the SAME binary and input. Known and dismissed TWICE with the wrong cause attached; both earlier explanations were tested here and DISPROVED -- ”multi-module dump ORDER varies” (it was the timing line) and ”parallel module compilation” (RAYON_NUM_THREADS=1 still gives 2 hashes), as was a third guess, ”hash-container iteration of implicit nets” (implicit_nets is only get/insert, never iterated). ACTUAL ROOT CAUSE: diffing two UNOPTIMISED MIR variants reduced it to four lines, the first being the cause and the rest consequences -- `Spur(27)->'aa0'` vs `'aa2'` (interner ids), then swapped node numbering and SSA value numbering. Both names are implicit nets introduced by ONE instance, `comparator C1(.cout(aa0), .inp(in), .inm(aa2))`. E-41 declares an undeclared connection actual implicitly, emitting the declaration the first time it meets each name while walking `for (port_name, binding) in port_raw.iter_mut()` -- and `port_raw` is a HashMap<Name, PortBinding>. Rust seeds its hashers RANDOMLY PER PROCESS, so the walk order varied every run; whichever implicit net was met first was declared first, interned first, and took the lower Spur. It takes TWO implicit nets on ONE instance to show -- with one there is nothing to reorder, which is why nearly the whole corpus is unaffected. RAYON_NUM_THREADS=1 does not help because the randomness is in the hasher SEED, not thread scheduling, which is exactly what made the parallelism theory look plausible and kept it alive. FIX: walk the bindings in the TARGET's declared port order -- deterministic, and the order a reader expects -- with the port name as a total tie-break. VERIFIED: the reproducer and both corpus models now yield ONE MIR over 12 compilations each (lrm_p209_1 went 8 -> 1); corpus-wide 488 models, 486 identical, 2 changed, NONDETERMINISTIC 0, the two changed being exactly the affected models now producing one canonical MIR instead of a random pick; and NOT a behaviour change -- the sigma-delta simulates byte-identically before and after (417 transient rows, same hash, four compilations per binary). WHY IT MATTERED: never a miscompile (the permutation is consistent, output unchanged), but builds were not bit-reproducible and it DEFEATED MIR-diff output-preservation checking -- a change under test could not be distinguished from the compiler disagreeing with itself. VA_TEST/mir_oracle.py --stable exists to tolerate exactly this; with the cause gone that flag is a safety net rather than a necessity. Verify (examples/vafdeterminism): 4 checks.

E-347 *OpenVAF-master-20260610/openvaf/mir_build/src/ssa.rs*

The SSA re-builder no longer mints an INVALID phi operand -- the deeper defect E-329 fixed the crash for and deliberately left open. The whole 496-file .va corpus now compiles under an ASSERTIONS build with zero panics. E-329 named the right FILE but the wrong PATH, and that mattered: the obvious fix -- repairing phis in FunctionBuilder::finalize(), where mir_build declares a function complete -- DID NOT WORK, because those phis do not exist yet. Instrumenting finalize() showed exactly ONE phi there, carrying no v0; a backtrace armed at phi creation showed the gravestone is minted by SSAVariableBuilder (the 'add values to an ALREADY FINISHED MIR function' builder) during topology linearisation, long after mir_build is done. That accounts for the symptom that had looked contradictory: validate() PASSES at sim_back/src/lib.rs:175 and FAILS at :179, on either side of Topology::new. Two other candidates were instrumented and CLEARED rather than eliminated by argument: the topology builder's own phi rebuild already defaults unmapped operands to F_ZERO (traced, never fires), and try_remove_phi_edge* in mir/src/dfg/phis.rs does write GRAVESTONE into an arg slot but only ever targeted inst11, not the failing inst27/inst30. An earlier ALIAS-TIMING hypothesis (value_def reports Invalid for Alias(_) too, so the operand might be an unresolved alias at finalize) was tested and REFUTED by the same instrumentation. Fix: in finish_predecessors_lookup, a GRAVESTONE operand now takes a LIVE SIBLING operand of the same phi. Why a sibling and not a synthesised zero -- the operand sits on an edge out of a block unreachable from the entry, so the edge CANNOT EXECUTE and the value is never read, making any sibling equally correct; and this builder is TYPE-AGNOSTIC (it holds no Type for the place), so a sibling is the only TYPE-CORRECT value available. F_ZERO would be wrong for an integer or boolean place. A phi whose operands are ALL gravestones is left alone. Fixing it here covers BOTH callers, which is what makes it the root fix

rather than a patch at one call site. Output preservation: --dump-mir oracle over the corpus = 496 files, 495 IDENTICAL, 1 CHANGED, 0 nondeterministic -- and the one change IS the E-329 reproducer, whose diff is VALUE RENUMBERING ONLY (normalising v\d+ makes the dumps byte-identical; value count 131 -> 130, exactly what reusing an existing sibling instead of leaving a placeholder produces). It still computes E-329's own criterion exactly: ssn == ssn_ref at i=-5. 0e-4, v=0.5. The three example models were each verified to TRIP the assertion on a PRE-FIX assertions build and to be clean after -- proven triggers, not decoration. A fourth candidate using a `for` loop did NOT trip and was DROPPED rather than shipped as filler. Regression 279/279. Verify (examples/ssavalid): 3 checks (the definitive check needs an assertions build, and the doc records the command).

E-352 *openvaf/sim_back/src/dae.rs, dae/builder.rs, openvaf/osdi/header/osdi_0_4.h, openvaf/osdi/src/{lib,metadata,inst_data,load,eval}.rs, metadata/osdi_0_4.rs*

SUPERSEDED BY E-359 (the capability stands and is still verified by the same oracles; the compiler-emitted-tensor IMPLEMENTATION below was replaced by numerical differentiation of the analytic jacobian in ngspice, removing the compile-time regression, the 30MB objects, the runtime penalty and the ABI bump). **`.disto`** now includes a Verilog-A model's nonlinearities; previously it printed a warning and left them out. Distortion is a VOLTERRA analysis: it needs each device's Taylor expansion of I(v) to THIRD order, not the operating-point linearisation the Jacobian provides. Built-in devices hand-code those coefficients (diodset.c computes id_x2/id_x3), which is why only FOUR of ~58 built-ins implement DEVdisto at all -- diode, BJT, BSIM1, MES -- and the OSDI ABI stopped at first derivatives, so cktdisto had nothing to ask an OSDI device for. KEY ENABLER, verified before anything was built: OpenVAF's autodiff is ALREADY arbitrary-order (mir_autodiff's DerivativeIntern chains through `previous_order`, with a comment anticipating 8th order), and nested ddx(ddx(ddx(...))) reproduces closed form to every printed digit. So the compiler work was not implementing higher-order AD but ASKING for it: build_taylor_tensors re-runs auto_diff over its own first-order results for the second, and again for the third. NO 3-VARIABLE CEILING: ngspice's framework caps at three controlling variables (Dderivs holds derivatives "w.r.t 3 variables"; the DFx helpers take up to 27 scalar arguments with no fourth-variable form, DFn2F12 taking a struct because the list outgrew the calling convention). That cap belongs to those hard-coded helpers, not to the mathematics -- the contribution is a symmetric tensor contraction -- so osdidisto.c contracts directly over however many inputs a device has and never calls D1x/DFx. Proven with an ideal mixer I(out)=k*V(a, ref)*V(b,ref) whose ONLY nonlinearity is the mixed partial (both diagonal 2nd derivatives identically zero, so a dropped cross term gives exactly zero): matches closed form 0.5*k*A^2*R EXACTLY. *TWO SILENT-ZERO BUGS found on the way, both indistinguishable from a linear model: (1) the tensors were OPTIMISED AWAY -- ensure_optbarriers protects residuals/noise/jacobian but the Taylor values feed nothing else in the MIR, so the optimiser deleted them and every coefficient arrived as zero (they also take the mfactor scaling, since m parallel devices inject m times the distortion current); (2) NOTHING STORED THEM -- eval's jacobian/residual stores are gated on CALC_* flags that .disto never sets. Plus one wrong-by-a-constant bug only a reference implementation could catch: D1nF12 returns 0.5*S2vF12(...) and S2vF12 ALREADY sums both index orderings, so without that 0.5 the mixed-tone products came out exactly 2x -- and because IM3 consumes the f1-f2 kernel its error was a non-obvious 2.246x. KNOWN LIMITATION AT THE TIME, SINCE LIFTED BY E-353: a model whose contribution goes through $limit emitted NO tensors (the residual depends on the limited value, not the raw voltage read; build_jacobian handled that via intern.lim_state and the tensor pass did not). Limiting is standard in production diode/BJT/MOS models, so this was a real restriction; E-353 folds the limited values into the derivative chain and removes it. What remains unreachable is a nonlinearity in a GROUND-REFERENCED probe (the tensors are indexed by model input and a bare V(a) is not one, having no hi/lo pair). Registering DEVdisto removed cktdisto.c's blanket warning, which would have turned such a model into a SILENT ZERO -- worse than before -- so OSDIdisto warns for itself. VERIFIED against four oracles, two involving no simulator at all: HD2/HD3 vs a closed-form polynomial (0.0 and 1.2e-16); the A^2/A^3 scaling laws*

*(4.0000, 8.0000); f1+f2, f1-f2 and IM3 vs the built-in diode (1.87e-06, the bound the 0.24 ppm $vt difference justifies); IM3 vs transient+FFT in a DIFFERENT DOMAIN, converging 5.7% -> 1.4% -> 0.21% as the drive halves, exactly the A^2 signature of the 5th-order content a 3rd-order truncation excludes; and the multi-variable cross term (0.0). TWO HARNESS TRAPS recorded because both would have produced a confident wrong answer: the first IM3 transient was measuring SPECTRAL LEAKAGE from the fundamentals rather than IM3 (exposed by the A^3 law giving 2.93 instead of 8, not by the value looking wrong); and a first parameter choice made HD3 IDENTICALLY ZERO (the two contributions cancel exactly when c3 = 2c2^2/Ytot, and 2*(1e-4)^2/2e-3 is precisely the 1e-5 chosen) -- a zero that would have passed a naive check for the wrong reason, the same trap as the old campaign's "HD2 ~ 0 on a LINEAR network". The suite now refuses to score a both-zero comparison. Regression 284/284. The example is a proven trigger: on the pre-fix binaries 5 of its 6 checks fail, and the one that passes is the one that should. Verify (examples/osdidisto): 6 checks.

E-353 *openvaf/sim_back/src/dae/builder.rs, openvaf/osdi/src/load.rs, examples/limitdisto_examples/**

SUPERSEDED BY E-359 ($limit models still work -- the suite below passes unchanged -- but the chain-fold no longer exists; differencing the real jacobian makes the problem unable to arise). **`.disto`** now works for Verilog-A models that use $limit -- i.e. every production compact model. E-352 gave OSDI devices distortion analysis, but only for models reading their controlling voltages DIRECTLY; a model passing one through $limit contributed NOTHING, and limiting is how every production diode/BJT/MOS model converges, so the feature did not reach the models people actually run. ROOT CAUSE: $limit hands the model a LIMITED copy of a branch voltage and the residual is written in terms of that copy, so differentiating the residual by the RAW voltage read yields zero -- nothing in the expression depends on it. The model is not linear, but every derivative the tensor pass asked for was. This was never a problem for AC or transient: build_jacobian has always folded the limited values back in via intern.lim_state; E-352's build_taylor_tensors simply did not perform the same fold. FIX: each model input now maps to a CHAIN of autodiff unknowns rather than a single one -- new taylor_unknown_chain returns the limited values from intern.lim_state.raw (each with the sign it contributes) followed by the input's own unknown. The 2nd-order pass sums over every PAIR drawn from the two chains and the 3rd-order pass over every TRIPLE, each term carrying the XOR of its entries' signs; an entry is negated when the model limits a REVERSED branch, $limit(V(ref,a),...). VERIFICATION uses an exact expectation with no appeal to another simulator: $limit is a convergence aid, so at convergence the limited value equals the actual one and a model written with it is THE SAME DEVICE as the same model written without it -- every distortion product must agree. Both spellings are generated from ONE body string, which is what guarantees they differ only by $limit and cannot drift apart. Five shapes cover the chain combinatorics (where this can go wrong -- a model limiting a single branch never puts more than one entry on either side of a pair): two independently limited diagonals; a cross term with BOTH inputs limited (2x2 pairs); a cross term with ONE input limited (asymmetric chains, 2x1); a THIRD-order cross term whose chain has 3 entries, so the triple loop genuinely runs; and a reversed-branch limit, the only spelling producing a NEGATED chain entry. All 13 live comparisons agree, worst 7.1e-10 -- convergence noise, since the two spellings settle 8.3e-11 apart (limiting changes the Newton path, not the solution) and d2/dv2 of exp(v/vt) amplifies that by 1/vt^2. THREE WAYS THIS COULD HAVE PASSED WHILE WRONG, each caught and closed because each produces a confident green: (1) BOTH-ZERO comparisons -- 'no distortion' is precisely the pre-fix behaviour, so scoring 0 == 0 as agreement would certify the bug; the suite scores a both-zero product as a FAILURE except where the mathematics requires zero (a bilinear term k*v1*v2 has no third derivative, so its IM3 is correctly zero and is asserted as such). (2) COMPARING MAGNITUDES -- a dropped sign on a negated chain entry flips the result and leaves

E-359 *openvaf/sim_back/src/dae.rs, dae/builder.rs, openvaf/osdi/header/osdi_0_4.h, openvaf/osdi/src/{lib,metadata,inst_data,load,eval}.rs, metadata/osdi_0_4.rs, openvaf/test_data/{dae,init}/*.snap*

**.disto** for Verilog-A rebuilt on NUMERICAL differentiation of the analytic jacobian; the compiler-side tensor machinery is DELETED. E-352 obtained the 2nd/3rd-order Taylor coefficients by having the compiler emit a symbolic closed form. The capability was right but the cost was out of all proportion to what .disto does: on ASMHEMT 707k intermediate derivatives -> 1.4M MIR instructions -> a 30.2MB .osdi and a 126s compile against a 3.6s baseline (35x; hisimhv 20x), plus -- because the tensors were computed inside eval() -- a 3.3-3.9x RUNTIME penalty on every DC sweep, transient, AC and noise run, and an OSDI 0.8->0.9 ABI bump that made every already-compiled model unusable without a rebuild. ROOT MISTAKE: .disto needs the coefficients at ONE point, once per instance, once per analysis; E-352 computed a closed form valid at EVERY point in order to evaluate it once. WHAT REPLACES IT: the model already publishes its FIRST derivatives analytically -- that is the jacobian every analysis loads -- so the higher ones follow by differencing THAT at the operating point (d2 R_r/dv_p dv_q = d/dv_q[J_rp]; d3 = d2/dv_q dv_s[J_rp]). Differencing an already-analytic first derivative rather than the residual is what keeps it accurate. Two properties make it cheap: the tensors are evaluated at the DC OPERATING POINT so they are frequency-INDEPENDENT (built once at D_SETUP, reused by every mode and every frequency), and they are built in NODE coordinates where an instance has 5-20 nodes rather than the 52 'model inputs' ASMHEMT reports. Entries keep the exact (row, col...) shape the analytic path produced, so the whole verified Volterra contraction is reused UNCHANGED -- only the tensor source and the kernel differ. NODE COORDINATES ALSO LIFT A REAL LIMITATION: E-352 indexed by model input (a branch voltage with a hi/lo pair), so a nonlinearity in a GROUND-REFERENCED probe V(a) had no pair, was never recorded, and the device reported 'contributes no distortion' and returned zero; it now works and is pinned to a closed form (-4.16666665e-02 vs exact -4.16666667e-02). E-353's $limit chain-fold disappears for the same reason -- differencing the real jacobian sees what the simulator sees, so there is no chain to fold, and E-353's seven-shape suite (written specifically to break that logic) passes unchanged against an implementation containing none of it. ACCURACY, measured against EXACT CLOSED FORMS rather than against the old implementation: cubic 4.0e-09/5.4e-09, exponential diode 2e-10/~1e-10, internal-node 3.6e-12/1.2e-08, ground-referenced 4e-09, MEXTRAM vs the analytic implementation 5e-09/1e-07 -- all below the 1.9e-06 floor the built-in-diode oracle already sits at (a 0.24 ppm $vt difference). THE STEP SIZE IS THE WHOLE GAME: truncation is O(h^2) and roundoff O(eps/h) at 2nd order but O(eps/h^2) at 3rd, so one step cannot serve both (a single 1e-3*V step put the result 8.5e-5 out); and the right yardstick is not the operating voltage but the CURVATURE scale (v_t = 26mV for a diode, volts for a polynomial), so a voltage-relative step left a cubic biased at 0V 11% wrong. The 2nd-order pass measures

E-363 *openvaf/mir_opt/src/simplify_cfg.rs, openvaf/hir/src/elaborate.rs*

Two compiler CRASHES on legal Verilog-A, found by a fuzzer that COMPOSES features each developed in isolation (mutation fuzzing cannot reach them -- mutants die at the parser). (1) `simplify_unconditional_jmp_term(src, dst)` merges `src` into `dst` -- retarget every predecessor, then delete `src` -- with no guard for `src == dst`. A block whose terminator jumps to ITSELF (`block2: jmp block2`, which is what a `case` inside a `do-while` folds to) made the retarget a no-op, and the block was deleted anyway while live terminators still named it; `mir_llvm::Builder::new` allocates LLVM blocks only for blocks IN the layout, so codegen unwrapped `None` (builder.rs: 655/656/690). The CFG layer already called this invalid -- `ControlFlowGraph::replace` opens with `debug_assert_ne!(old, new)`, exactly what fires on an assertions build. Now declines the self-merge: a self-loop is a legitimate CFG shape and must reach codegen. (2) A module has THREE array collections -- `buses`, `var_arrays` (E-4) and `param_arrays` (E-14) -- and instance flattening renamed only the first two, so every instance re-declared `cf[0]`, `cf[1]`, ... under the same names: a module with an array parameter could not be instantiated twice, and two DIFFERENT modules sharing an array-parameter name collided too. Now chains `param_arrays` into the rename. Equivalence: all 380 corpus files produce BYTE-IDENTICAL optimized MIR (the `.osdi` itself is not comparable -- the same binary on the same input hashes differently each run). Left OPEN and documented: a provably non-terminating analog loop still fails, because its contribu-

tions are then unreachable and there is no correct object code for such a model; the fix is a diagnostic, not a substituted value. CLOSED BY E-375, which added that diagnostic -- and found that the crash described here had already stopped reproducing, the CFG repair above having turned it into a silently EMITTED model that hangs the simulator instead.

E-375 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, examples/vafloop_examples/**

A loop that provably cannot finish is now a COMPILE ERROR. A Verilog-A module body must complete one evaluation, and a loop that cannot exit makes that impossible. This went through three behaviours, of which the middle one was the worst: originally the compiler PANICKED (an `unwrap()` on a loop-exit block that was never created); after E-363 repaired the CFG it stopped panicking and started EMITTING -- `while (1)` produced a well-formed 36,920-byte `.osdi` that ngspice loaded without complaint and then hung forever on the first device evaluation, with no diagnostic at all. A compile-time panic is loud and immediate; a simulator that never returns just looks slow, and the model is the last place anyone looks. There is no third option: emitting the loop hangs the simulator and substituting a value for the unreachable code invents a device that was never described. THE CHECK reports when the controlling condition provably cannot change, with two distinct messages -- `loop condition is always true` for a non-zero literal (certainly infinite) and `loop condition can never change` when nothing in the loop writes what the condition reads (either never entered or never left, not decidable here, and the report must not claim more than it knows). THE ANALYSIS IS SOUND IN THE REJECT DIRECTION: every bail-out means say nothing, so it can miss a hang but must not reject a model that terminates -- `repeat(n)` is counted; a literal-zero condition is a zero-trip loop, not an infinite one; `\$finish`/`\$stop`/`\$fatal` leave the loop; a user function may write through an OUTPUT ARGUMENT so every name passed to one counts as written, and a user call in the CONDITION abandons the check outright; `\$random`/`\$dist_*`/`\$rdist_*` return a fresh value per call. Names are matched SYNTACTICALLY rather than resolved to VarIds, which also errs safe: a shadowing declaration in a nested block makes an unrelated name look written, suppressing the diagnostic rather than inventing one. NOT DETECTED, undecidable in general: a loop whose condition variables are written but never toward the exit -- nested loops sharing an index, where `for(i=0;i<10;i=i+1) for(i=0;i<3;i=i+1)` runs forever while the same shape with the bounds swapped terminates. THREE CODEGEN CRASHES CLOSED ON THE WAY: `disable <block>` (LRM 5.4) is Verilog-AMS's loop break and would be the obvious escape hatch, but it is deliberately NOT counted as one. It works, and keeps working, for a loop that can also finish normally -- such a loop's condition changes, so the check never looks at it. As the SOLE exit from a loop whose condition cannot change, the code after the loop is reachable only through the `disable` edge and OSDI codegen aborts with `unreachable!("attempted to read undefined value")` at mir_llvm/src/builder.rs:143; verified on the pre-fix binary for a literal `while (1)`, a constant-folding `while (1 > 0)` and a non-constant `while (i < 10)` whose `i` is never written -- 3/3 crashed, with and without the loop result being used. So reporting them cannot regress a working program, because there is no such program: it replaces a compiler crash with an actionable error, and the diagnostic says so rather than steering the user into it. `\$finish`/`\$stop`/`\$fatal` are treated differently precisely because they DO compile today. VERIFICATION: a DIFFERENTIAL RUN over all 726 `.va` files in the repo, new binary against the shipped one -- 0 files rejected by the new check, 0 regressions (the one apparent hit was a pre-existing `generate for: only '<' and '<=' loop conditions are supported`, matched by a too-loose substring). hir_ty/hir_lower/hir_def/basedb 28/28. Regression 300/300. Verify (examples/vafloop): 21 checks, and the ACCEPT half is the more important one -- it is what proves nothing broke. A proven trigger: pre-fix 10/21, with 6 cases compiling silently (the hangs) and 3 CRASHING; the example refuses to score a crash or a compiler hang as a pass, since replacing those is the entire point.

E-376 *openvaf/hir_ty/src/builtin.rs, openvaf/hir_lower/src/expr.rs, examples/distint_examples/**

**`$dist_*`** now returns INTEGER, not real; **`$rdist_*`** stays real. Found by a correctness campaign

over all 94 $-prefixed system functions. THE LRM SPLIT: the $dist_* family comes from Verilog-2001 17.9.2 where every parameter and return value is an integer, and Verilog-AMS added $rdist_* BECAUSE the originals are integer-only -- if $dist_* already returned real there would be no reason for a second family to exist. openvaf-r returned real from both, and TWO RECORDS IN THIS REPO ALREADY SAID OTHERWISE: E-10's doc lists $rdist_* (real-valued) and $dist_* (integer-valued), and an Enhancement-49 audit comment sitting directly above the signatures calls them "the integer-distribution $dist_* functions" while correcting their ARGUMENT types on LRM grounds -- the return types were missed in that same pass. $random/$arandom have always been -> Integer. WHAT IT COST: LRM-conformant code did not compile -- $display("%d", $dist_uniform(seed,10,20)) gave type mismatch: expected integer value but found real value, which is how the campaign hit it. THE FIX IS IN TWO PLACES AND EITHER ALONE IS WORSE THAN NEITHER: (1) eight DIST_* signature entries -> Real become -> Integer (the eight RDIST_* untouched); (2) each $dist_* lowering arm now ends in ficast, matching $random. Changing only the signature was tried first and is a WRONG-CODE BUG -- the type checker accepts %d while the lowering still emits a real MIR value, so every downstream integer consumer reads it as 0; measured, not theorised: $dist_uniform(s,10,20) printed 0 (outside its own range) and an integer assignment got 0, while a real assignment still worked. THE DRAW DOES NOT MOVE: where the runtime value is not already integral it is ROUNDED FIRST (rng_round_real = floor(x+0.5), correct for negatives too, which $dist_t and a zero-mean $dist_normal require) and ficast then truncates an exactly-integral double, losslessly; UniformInt and Poisson are already integral and are only cast. Means and variances over 20000 draws are IDENTICAL before and after (uniform 15.0214/9.9241, normal 99.8969/222.83, poisson 4.0319/4.0785) -- only the static type moved, and the discrete/continuous split stays visible in the variances ((n^2-1)/12 = 10.08 for $dist_uniform vs 100/12 = 8.33 for $rdist_uniform), which is itself a check that the two families remain distinct. THE $rdist_* HALF IS THE TRAP: $rdist_poisson lowers through the SAME RngFun::Poisson call as $dist_poisson, so a textual edit casts it too and silently converts the real family to integer -- that happened during development and was caught by an arm-by-arm audit asserting ficast present for all seven $dist_* and absent for all seven $rdist_*, which is why the example asserts the NEGATIVE case (every $rdist_* draw must be non-integral) rather than only the positive one. COMPATIBILITY checked in the other direction too: %g applied to a $dist_* result still compiles on both binaries, and assigning the draw to a real still works by implicit widening, so existing models are unaffected. Differential over all 726 .va files in the repo: 0 regressions. Regression 301/301. Verify (examples/distint): 22 checks, a proven trigger -- pre-fix 19/22.

E-377 *openvaf/osdi/stdlib.c, examples/simparamdiag_examples/**

OSDI diagnostics were unreadable and carried no severity. Found by the correctness campaign over all 94 \$-prefixed system functions, when a wrong \$simparam\$str("analysis") reported OSDI(debug) n1: unknown \$simparam_stranalysisOSDI(debug) n1: unknown \$simparam_str.... FOUR defects in one line. (1) NO SEPARATOR: concat("unknown \$simparam_str", name) joins with nothing, so the function name and its argument run together and the reader cannot see where one ends -- the numeric \$simparam path had the same bug; now unknown \$simparam\$str "analysis", and reported with the \$simparam\$str spelling the user actually writes rather than the internal \$simparam_str. (2) NO NEWLINE: osdi_log writes fprintf(dst, "%s", msg) and no message carried a \n, so consecutive reports concatenated into one line -- which also HID the repetition, the old output looking like two lines where the fixed one shows 373 (the same 373 reports were always there). (3) WRONG SEVERITY FOR EVERY MESSAGE IN THE OSDI LAYER -- the one with reach beyond \$simparam: ngspice's osdi.h had #define LOG_LVL_MASK 8, but the level occupies the low THREE bits (DEBUG 0 .. FATAL 5) so the mask must be 7; 8 selects bit 3, which no level ever sets, so lvl & LOG_LVL_MASK was 0 = LOG_LVL_DEBUG for EVERY level. \$display, \$info, \$warning, \$error and every fatal diagnostic alike were labelled "OSDI(debug)" and written to STDOUT -- severity invisible to the user and to any log scraping, and \$error/\$warning never reaching stderr. OpenVAF's

own copy of the header (osdi/header/osdi_0_4.h) has always said 7, so the two sides of the ABI disagreed on how to decode the level. Measured: before, all four severities print "OSDI(debug)" on stdout; after, `OSDI n1:/OSDI(info) n1:` on stdout and `OSDI(warn) n1:/OSDI(err) n1:` on stderr. (4) A LEAK: the message was malloc'd, passed to `osdi_log` and never freed -- `free` was not even declared in the runtime's `NO_STD` extern block -- so a failing operating point leaked 373 allocations, not one; both call sites now go through one helper that frees. THE REPETITION IS FIXED BY E-378: making the messages visible exposed that there were 373 of them, left open here as "a convergence-path change" -- an understatement. `CKTop` read the fatal's `E_PANIC` return as "did not converge" and walked the gmin/source-stepping ladder, re-evaluating the device each pass, then blamed `timestep too small`. E-378 aborts the operating point on a fatal, so the report now appears once. Regression 302/302. Verify (examples/simparamdiag): 12 checks, a proven trigger -- pre-fix 1/12, and that single pass is a negative check that succeeds for the wrong reason on the old binary.

E-379 *sourcegen/src/{ast,osdi,lib,hir_builtins}.rs, verilogae/verilogae/src/{back,compiler_db}.rs*

**`cargo test --workspace`** now builds, and no longer DELETES checked-in source. It could not be used as a gate: it failed to build, and once it built it destroyed shipped work. (1) `verilogae`, the Python-binding crate, had drifted behind the compiler it binds to -- 4 errors: `IntNumber::value()` returns `Option<i32>` (None for a literal too large for 32-bit `integer`) and the call site still wrote `val.value() as f64`, now the documented `value_as_f64()` fallback; and `stub_callbacks` was missing 13 `CallBackKind` variants (the string/file-IO/scanf runtime E-11/105/106, `ac_stim` E-51, the RNG builtins E-10) plus 3 `ParamKinds`. Each stub signature is taken from `osdi::compilation_unit`'s `general_callbacks` rather than guessed -- the call is built from the MIR arguments, so a stub with the wrong arity or return type is malformed IR, not a wrong number; `return None` is used only for `ScanBegin`, which returns nothing, because the builder treats a missing callback as a no-op and a value-returning one would leave its result undefined and abort codegen. (2) THE SERIOUS HALF: three `sourcegen` tests call `ensure_file_contents`, which OVERWRITES the file it generates, and all three generators have fallen behind. `generate_builtins` strips six builtins added by later enhancements (`table_model`, `realtime` E-59, `rtoi`/`itor` E-104, `fgetc` E-107, `ungetc` E-108), two `ParamSysFun` methods without which `hir_def` does not compile, and silently re-adds an `is_unsupported` gate over ~30 builtins that ARE implemented (`zi_*`, `last_crossing`, `slew`, `transition`, the whole file-IO family) -- the checked-in file already carried a note that E-8's regeneration did exactly this once before. Both halves (hir_def/src/builtin.rs and hir_ty/src/builtin/generated.rs) are hand-maintained and must agree index-for-index with `BuiltIn`, so regenerating one and not the other is worse than neither; neither is written now. A MEASUREMENT TRAP worth recording: the hir_ty half was first reported "in sync, 0 changed lines" -- measured against a baseline the test had ALREADY clobbered; against git it differs too (117 entries vs the 111 the generator emits). Snapshot before running a test that writes. (3) `ast::ast` and `osdi::gen_osdi_structs` are QUARANTINED (`#[ignore]`), DISABLED NOT FIXED, with the reason recorded in the source: repairing them means bringing the grammar and the header parser up to ~370 enhancements of hand edits. Regenerating the AST DELETES `DisableStmt` entirely -- `disable <block>`, Verilog-AMS's loop break, has NO rule in the grammar at all and exists only in the checked-in file, and E-375 depends on it -- and collapses `width()` from an `Option<Range>+widths()` pair into a single `AstChildren<Range>`, renaming accessors the parser and lowering call; the osdi header parser cannot read the current `osdi_0_4.h`, panicking in `parse_ty` on `eat_ident().unwrap()`. TWO REAL GENERATOR BUGS WERE FIXED ON THE WAY AND ARE KEPT, correct independently of whether the tests are re-enabled: `Rule::Rep(Seq(..))` is now lowered (the grammar's `BitSelectExpr = base: Path ('[' index: Expr ']')*`, multi-dimensional selects from E-15, used to panic with "unhandled rule"; it is the only non-comma-list repetition, the rest going through `lower_comma_list`), with `pluralize` learning the irregular `index -> indices` since a bare "+s" would rename the accessor to `indexs`; and `Header::new` no longer unwraps the version parse, since archived snapshots live beside the live header and `osdi_0_4_enhancement1.h` strips to "4_enhancement1", killing the whole gener-

ator with a bare `ParseIntError`. VERIFIED: 207 passed, 0 failed, 158 ignored (pre-existing slow tests plus the two quarantined); before this the suite did not build at all. NON-DESTRUCTIVE, checked explicitly -- all five generated files are byte-identical to their committed versions after a full suite run. Regression 303/303 (unchanged; this touches no simulation path).

E-387 *openvaf/parser/src/grammar/expressions.rs, openvaf/preprocessor/src/processor.rs, openvaf/linker/src/lib.rs, examples/vafice_examples/**

An empty `()` crashed openvaf-r, plus two more compiler defects. (1) INTERNAL COMPILER ERROR on a five-line source with no CLI flags, on the SHIPPED release binary: `analog I(p,n) <+ ();` -> "OpenVAF encountered a problem and has crashed!" (exit 101). `paren_expr` loops on `while !p.at(EOF) && !p.at(T![')'])`, so for `()` the body never ran -- nothing was parsed and, the real defect, NO DIAGNOSTIC WAS EMITTED. The PAREN_EXPR completed with no child, hir_def lowered it to `Expr::Missing`, and `hir/src/body.rs` has arms for nine expression variants but none for `Missing`, so it hit `_ => panic!("invalid HIR")`. THE HOLE WAS ONE TOKEN WIDE: `{}`, `{1,}`, `a[]`, `? :`, `sqrt()` and `1+` were all already rejected in the parser and never reached lowering; only `()` and its nestings got through -- which is why it survived the E-347 assertion-replay campaign, being a `panic!` rather than a `debug_assert` and reachable in release. (2) `-DNAME=VALUE` DEFINED A MACRO CALLED `NAME=VALUE`: every `-D` flag became a macro whose name was the whole flag string, so `-DEXT=5.5` was unreachable as `` `EXT `` AND as `` `ifdef EXT ``. Now split on the first `=`. HALF DELIBERATELY DEFERRED (closed since by E-388): the VALUE was not yet substituted, because a macro body is a `Vec<ParsedToken>` whose text resolves BY SPAN against a real source file and an argv value has no backing text -- doing it properly needs the defines materialised as a source file. (3) A NONEXISTENT OR READ-ONLY `TMPDIR` ABORTED through an uncaught C++ exception (`libc++abi: terminating due to uncaught exception ... filesystem_error`, `clang: error: unable to execute command: Abort trap: 6`) and the final message blamed "linking" without naming the cause; CI runners and sandboxes routinely set TMPDIR. `link()` now probes it first -- existence AND writability, since a read-only TMPDIR fails identically -- and names it. HOW (1) WAS FOUND: not by fuzzing the language but by following (2) -- a valueless `-DEXT` defines an EMPTY body, so `1e-3*(\EXT)expanded to1e-3*()and crashed. Two defects, one reachable only through the other. ALSO FOUND, DEFERRED (closed since by E-388): the E-148 expression-depth guard reports over-deep expressions (>998 operator terms) through the GENERIC "unexpected token" path --expr_too_deep()says so in its own comment -- instead of saying the expression nests too deeply, which the analogous include guard does. Diagnostic wording on a deliberate guard, no correctness impact; carrying a new SyntaxError variant through both error enums, the tree builder's irrefutablelet‘ and the renderer is a change of its own.` Verified by 11 checks (11/11 fixed, 5/11 pre-fix; the 3 accept checks pin nested ordinary parentheses, an end-to-end 2 V / 1 kOhm = 2 mA through a parenthesised model, and an ordinary compile with a valid TMPDIR since the new probe runs on every link). Compact-model corpus unaffected: 92/92 compile and 92/92 correctness.

E-388 *openvaf/hir/src/db.rs, openvaf/preprocessor/src/processor.rs, openvaf/parser/src/error.rs, openvaf/parser/src/grammar/expressions.rs, openvaf/syntax/src/error.rs, openvaf/syntax/src/parsing/tree_builder.rs, openvaf/basedb/src/diagnostics/syntax_error.rs, examples/vafdefine_examples/**

Closes the two items E-387 deferred. (1) `-D` VALUES ARE NOW SUBSTITUTED. E-387 fixed the NAME but left the VALUE, for a concrete reason: `-D` flags were synthesised straight into `Macro { body: vec![], .. }`, and a macro body is a `Vec<ParsedToken>` whose text resolves BY SPAN against a real source file -- a value from argv has no backing text, so no body could be built. The body was therefore always empty, so `-DK=5.5` could not substitute 5.5 AND a bare `-DK` expanded to nothing instead of the documented "1". The synthesis is removed entirely: the flags are written into a VIRTUAL SOURCE FILE as ordinary `define` directives, which the preprocessor parses as if the user had typed them, spans and all -- nothing about macro expansion had to change, the definitions simply became real. `-DK=5.5`->5.5, `-DK=42`->42, `-DK=1e-3`->0.001, `-DK=-2.25`->-2.25, -

`DK=(2.0+3.0)->5` (an expression, since it is now really parsed), `-DK->1`. ACCEPT HALF IS THE POINT HERE: this moves EVERY `-D` flag onto a new path and STANDARD_FLAGS (`__OPENVAF__`, `__VAMS__`, `__VAMS_COMPACT_MODELING__`) travel it too -- compact models branch on those, so a fix that defined the user's flags but dropped the standard ones would break real models while every `-D` test still passed. Each is asserted individually. The preamble is skipped when empty, so a compile with no `-D` flags takes exactly the old path. (2) THE EXPRESSION-DEPTH GUARD SAYS WHAT IT MEANS. E-148's guard reported through the ordinary "unexpected token" path (`expr_too_deep()` said so in its own comment), so a 999-term chain came back as "unexpected token identifier; expected '('..." -- a complaint about a valid token with no hint a limit exists. It now reports `expression nests too deeply` with a help note naming the 1000 limit, the same courtesy the preprocessor's include guard has always had. Took a new variant through BOTH SyntaxError enums, the tree builder's conversion -- an IRREFUTABLE `let`, valid only while the parser enum had one variant -- and the renderer. Limit unchanged (998 terms still compile) and the post-recovery cascade is identical: 129 diagnostics before and after. A TEST THAT HAD ENCODED THE OLD BEHAVIOUR: E-387's example probed `-D` with `V(p,n)*1e-3 `EXT ;` which parsed ONLY because a `-D` macro expanded to nothing; once values substitute it became `... 1e-3 5.5 ;` and failed. The test was asserting the defect, not the intent -- now `1e-3*(`EXT)`. A test written against a broken behaviour fails when the behaviour is fixed, and that failure is the fix working. Verified by 16 checks (16/16 fixed, 7/16 against the PRE-E-387 binary, so one of the nine failures was already E-387's). vafice stays 11/11. Corpus unaffected: 92/92 compile, 92/92 correctness.

E-389 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/inference.rs, openvaf/hir_lower/src/expr.rs, openvaf/parser/src/grammar/items/module.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/syntax/veriloga.ungram, sourcegen/src/{ast,ast/src,osdi,lib}.rs, examples/vafopenitems_examples/**

The four items still open for openvaf-r, closed together. (1) A NON-TERMINATING LOOP THAT E-375 MISSED. E-375 rejects a loop that provably cannot finish, but it asked whether the condition's variables are WRITTEN, which is not whether they can CHANGE: `for (k = 0; k < 10; k = k + 0)` -- and `k = k`, `k = 0 + k`, `k = k - 0`, `k = k * 1` -- compiled cleanly, emitted a valid .osdi, and then HUNG ngspice at the operating point with no diagnostic, the exact outcome E-375 exists to prevent, reached by a different shape. A provably value-preserving write no longer counts as progress. The arithmetic identities are honoured only on INTEGERS: on reals `k + 0.0` turns -0.0 into +0.0 and a condition can observe that (`1.0/k < 0` flips -inf to +inf), so reals get only the exact copy. `k = k - 1` is deliberately NOT rejected -- it terminates by 32-bit wrap after ~2^31 iterations (measured i(v1) = -2.147e+06, exit 0), slow but finite. (2) ANSI ANALOG FUNCTION ARGUMENTS. `input real x;` and `analog function real f(input real x);` were both parse errors; both now work, with LRM inheritance in an ANSI header (`f(input real x, y)`). The type is really APPLIED -- an integer argument declared either way rejects a real literal. Array arguments still need the separated form. (3) `$table_model` RUNTIME ARRAY DATA (LRM p274) -- array variables filled in by the body were rejected with "requires a bit-select". Inference needed its own path for the same reason `laplace_*` does; lowering keeps the compile-time interpolator's SHAPE so autodiff differentiates it identically -- the small-signal conductance is the exact analytic segment slope and matches the compile-time table to every printed digit. (4) BOTH QUARANTINED CODE GENERATORS (E-379) REPAIRED AND RE-ENABLED. gen_osdi_structs: `trim` skipped whitespace but not COMMENTS, so the first `/* */` inside a struct body panicked parse_ty; comments were then DISCARDED, which is why running it always deleted documentation. They now carry through from the header. That surfaced what the drift hid -- EVAL_RET_FLAG_DISCONT was in the header but MISSING from the generated Rust, and OSDI_VERSION_MINOR_CURR read 4 in the header and 5 in the Rust while the compiler stamps 7. ast: KINDS_SRC lacked 8 keywords and 11 node kinds (casex/casez, repeat, do, paramset, defparam, or, concat/replication/generate-if/generate-case/disable/paramset) and the ungram had no rules for them, so regenerating DELETED shipped language features; the gener-

ator also lacked the dual singular/plural accessor and the skip a replication needs past its count ( getting that wrong first produced a broken BitSelectExpr::indices and broke bus_basic.va). Regeneration is now PURELY ADDITIVE -- nothing lost -- which is what makes the tests safe to run. cargo test --workspace 207 passed/158 ignored -> 209/156. Verify (examples/vafopenitems): 28/28 fixed, 11/23 pre-fix; 12 of the 28 are the accept half, since making a compile-time check stricter is what can break working models. Output-preserving: 124/124 corpus .osdi BYTE-IDENTICAL by sha256 (same output path -- the file embeds its own name), and the corpus replays through an ASSERTIONS build with 0 panics, where E-317's last deferred item (an assertions-only PHI type mismatch) would have shown. Regression 312/312 -> 313/313.

E-390 *openvaf/hir_lower/src/stmt.rs, openvaf/hir_lower/src/expr.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/parser/src/grammar/items/module.rs, examples/vafcaseloop_examples/*, examples/lrm_examples/va/sample.dat*

Eight defects from a one-hour hunt; three were in E-389's own new code. (1) A `case` INSIDE A `do-while` COMPILED INTO AN INFINITE LOOP. `lower_case` opened each arm with `ensured_sealed()`, which seals the CURRENT block -- on the first arm that block still belongs to the CALLER, and when a `case` is the first statement of a do-while body the caller's block is the loop BODY HEAD, which must stay unsealed until the back edge exists. Sealing it early completed its phis against the entry edge alone, so a variable updated in the loop read back as its PRE-LOOP value; `while (k<3)` folded to constant true and the optimised MIR came out as `block2: jmp block2` with the contribution block unreachable. TWO SYMPTOMS decided by what encloses it: bare, it compiled clean and HUNG ngspice forever with no diagnostic; inside a for/while/repeat it CRASHED the compiler at mir_opt/dead_code_aggressive.rs:112 (E-324's site). The E-375 pattern a THIRD time -- the 2026-07-26 binary crashed here, E-363 fixed the panic, and what it emitted afterwards looped forever. Explains every boundary: wrapping the case in an `if` avoids it (the if opens a block first), `do{case}while(0)` is fine (no back edge), and a case in a FUNCTION called from a do-while fails identically (it inlines into the body head). Both calls removed; every block lower_case creates is already sealed explicitly. (2) An ANSI function arg with a TYPE but NO DIRECTION returned 0 -- neither input nor output, so the caller's value was discarded and the body read 0; `f(3.0)` gave 0 instead of 6. Verilog defaults a function arg to input. Only E-389's ANSI path could produce one; the separated and combined forms reject it. (3) `analog` blocks inside `generate` were SILENTLY DISCARDED -- generate-for x3 and generate-if both contributed nothing with ZERO diagnostics. The grammar had no case for `analog`, so it hit the catch-all, and the parse error was then SWALLOWED because elaboration re-renders the generate region from its syntax tree (E-67); the malformed node rendered to nothing. Generate worked for INSTANTIATION all along. (4) `disable <unresolvable>` was a SILENT NO-OP -- a typo'd label, a variable, even the module name: the statement did nothing and a loop meant to exit early ran to completion. Now resolved during validation against the same enclosing-block stack lowering uses. (5-7) `\$table_model`: the compile-time forms SORT and DE-DUPLICATE their breakpoints and the runtime-array form did neither, so identical data gave different answers (descending: 0.5/2.5/5.5 from a literal, -0.5/1.5/2.5 from arrays); a duplicated or unassigned abscissa divided by zero and the NaN surfaced only as Timestep-too-small; and the cubic code 3 was IGNORED at runtime (0.35 vs 0.5). All fixed IN KIND: an unrolled sort/dedup network (capped at 64 knots), guarded divisions, and a natural cubic spline solved in MIR by an UNROLLED THOMAS ALGORITHM -- the knot COUNT is compile-time known even when the knots are not. Its extrapolation continues the end TANGENT, matching the compile-time spline; extending the cubic instead is invisible inside the grid and wrong only outside it. (8) An unusable DATA FILE was a silent zero (missing, unreadable, empty, one column). The file is read during lowering, which has no diagnostic channel, so the check runs when the report is BUILT -- the first point with both the root file and the VFS. It immediately caught a real instance: lrm_examples/va/lrm_p274_1.va references sample.dat, A FILE THAT WAS NEVER IN THE REPOSITORY, and had been compiling only because of this defect; the data file is now supplied. NOT A DEFECT (recorded so it is not re-chased): paramset-

cannot-override was a TEST ERROR -- an override is `.g = <expr>;`, and `parameter real g = ...;` redeclares the target's parameter and is rightly rejected. Verify (examples/vafcaseloop): 45/45, 15 of them the accept half -- change (1) alters block sealing, which every model containing a `case` goes through, and (3) adds a keyword to a grammar. OUTPUT-PRESERVING: 124/124 corpus .osdi BYTE-IDENTICAL by sha256, so runtime correctness is unchanged by construction. Regression 313/313 -> 314/314.

E-391 *openvaf/hir_lower/src/expr.rs, openvaf/hir_lower/src/stmt.rs, examples/vaftabledup_examples/**

Closes the one case E-390 left open: a REPEATED ABSCISSA in a runtime **`\$table_model`** with CUBIC interpolation. E-390 made the runtime array form sort, de-duplicate and honour the cubic code, but its de-duplication carried the first value forward over each repeat. That is exactly equivalent to the compile-time `dedup_by` for LINEAR interpolation -- a zero-width segment whose endpoints are equal contributes nothing -- but NOT for a spline, where the dead knot still occupies a row of the tridiagonal system and perturbs every moment in it. The compile-time forms de-duplicate by SHORTENING the point vector; a runtime array cannot shrink, since its length is fixed at compile time and only its values are unknown. FIX: partition the repeats to the END (a stable 0/1 bubble network on an is-a-repeat flag that travels with its point) and give the trailing slots the last distinct knot, so the live prefix IS the de-duplicated table. Two things then follow that prefix rather than the array: the NATURAL BOUNDARY `M = 0` is forced onto the last LIVE knot and every replica after it, not merely the final slot; and the upper END TANGENT is computed from the last two LIVE knots. A zero-width interval also evaluates to its own knot value rather than a guarded zero, and is skipped during segment selection -- without that the highest qualifying index wins and a dead segment shadows the last live one. THE MISTAKE WORTH RECORDING: the end tangent was wrong on the first attempt, and the SHAPE of the failure is the lesson. After compaction the last two SLOTS are both replicas, so their spacing is zero and the guarded division silently turned the extrapolation into a CLAMP -- every value inside the grid exact, only the points past its end wrong. Interior agreement is NOT evidence a spline is right; the bug was visible only because the check probed x beyond the last knot as well as inside it. Verify (examples/vaftabledup): 20/20 fixed, 12/20 on the E-390 binary -- the eight failures are precisely the repeated-abscissa cubic cases and the twelve passes are the accept half, so the change lands on the residual without disturbing what E-390 already made agree. Covered: repeats at start/middle/end, two separate repeats, one abscissa three times, unsorted AND repeated, every abscissa equal, a table the body never fills in (all zeros, so every abscissa repeats), and both clamping and L extrapolation. COST: compaction adds an O(n^2) network on top of the sort and runs on every runtime CUBIC table including those with no repeats, bounded by the same 64-knot cap; the linear path is untouched. OUTPUT-PRESERVING: 124/124 corpus .osdi BYTE-IDENTICAL against the E-390 compiler. Regression 314/314 -> 315/315.

E-392 *openvaf/hir/src/elaborate.rs, openvaf/hir_def/src/body/lower.rs, openvaf/hir_lower/src/expr.rs, openvaf/hir_lower/src/stmt.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/syntax/src/ast/expr_ext.rs, examples/vafinstcheck_examples/**

MODULE INSTANTIATION WAS NOT VALIDATED AT ALL -- the port connection list was zipped POSITIONALLY against the target's declared ports and named connections were bound without asking whether the port exists, so the wrong port COUNT (surplus actuals dropped, missing ones leaving the port unconnected and the device contributing nothing), a port name the target does not have, and a PARAMETER the target does not have (`#(.zz(4e-3))` -- the override silently did nothing and the DEFAULT was used) all compiled clean with zero diagnostics. Two contrasts already inside this project: ngspice rejects the same mistake on a `.model` card, and `defparam` rejects it too. Verilog-A creates IMPLICIT NETS so a mistyped NET name can never be caught here, which is why checking what CAN be checked matters more rather than less. Instantiation is now checked for arity and for port/parameter existence; the discriminator for "is this connection named?" is the DOT, not the name, because a SYSTEM parameter override (`#(.\$mfactor(7))`) carries no NAME child -- keyed off `name()` it fell into the positional branch and set the target's

FIRST declared parameter to 7, which is what the LRM's own page-263 example does (`r = 7` instead of its default 1.0, a sevenfold wrong answer). PLUS: (2) `generate` did not rename analog-block LABELS or analog FUNCTION names -- both fell outside `collect_declared_names`, so two iterations redeclared the same name (`'ab' was already declared`), newly reachable because E-390 made `analog` legal in `generate`; (3) the renaming REWROTE THE NAME IN A NAMED CONNECTION -- substitution is lexical over the token stream, so `resistor #(.r(1e3)) r(...)` (instance name colliding with the child's parameter) became `.r_0(1e3)`, named no parameter and was silently dropped back to the default, which halved the resistance of `examples/generate_examples/resistor_ladder_generate.va`; the name after the dot belongs to the INSTANTIATED module's namespace and is now pinned to itself with an identity hole; (4) the runtime `\$table_model` sort SILENTLY GAVE UP above 64 knots while the compile-time path sorts at any size (65 reversed knots: cubic 160.0 vs 6.2566, 25x off) and `compact_distinct_runtime` had NO cap at all, so de-duplication ran on unsorted data -- replaced by a BATCHER ODD-EVEN MERGE network, O(n log^2 n) comparators, which reaches 256 points for less emitted code than the old one used at 64, with a shared cap and a compile error beyond it. THE STABILITY TRAP: de-duplication keeps the FIRST of any repeated abscissa in ORIGINAL order (the compile-time `dedup_by` does, because Rust's `sort_by` is stable), and the old odd-even TRANSPOSITION network was stable for free since it only exchanged neighbours -- Batcher's compares elements far apart and can swap two equal abscissae, invisible until their ordinates differ. Stability is restored by carrying the original index and breaking ties on it. A FASTER ALGORITHM IS NOT A DROP-IN REPLACEMENT WHEN A CONSUMER DEPENDS ON A PROPERTY THE SLOW ONE HAD BY ACCIDENT; (5) a `localparam` was rejected as a generate bound though it is fixed at elaboration and is already accepted as a constant everywhere else (array bounds, bus widths, parameter defaults, `repeat` counts) -- this composes with E-92, which freezes any parameter that shapes a declaration width INTO a localparam, so such a parameter may now size a generate too; a parameter that shapes nothing is still rejected; (6) `INT_MIN` DEVIATED FROM TWO'S-COMPLEMENT WRAPPING WHEN CONSTANT-FOLDED -- `-2147483648` parses as unary minus applied to `2147483648`, whose magnitude does not fit i32, so the operand became a REAL literal and the whole expression acquired real semantics (`(-2147483648)/3` rounded to -715827883 where integer division truncates toward zero to -715827882; `(-2147483648)-1` saturated instead of wrapping to INT_MAX; `(-2147483648)*2` saturated instead of giving 0), while the SAME value arriving at runtime from a `.model` card stayed an integer and was right in every case -- one expression with two meanings. The sign is folded into the literal where the prefix expression is LOWERED (`Literal::new` cannot see the enclosing minus, which is why fixing the syntax-level constant evaluator alone changed nothing). `(-2147483648)/(-1)` must STILL be rejected -- it is the one integer operation that genuinely traps -- and making the literal correct is what first brings it within reach of E-334's guard. VERIFIED: examples/vafinstcheck 41/41 (16/41 against the E-391 binary; the 16 are the accept half, which is the substance of the risk -- every LRM-sanctioned unconnected-port form still compiles), corpus byte-differential 92/92 industry models BYTE-IDENTICAL and 400 example models byte-identical with ZERO newly-rejected files (the only two differences are the `\$mfactor` files the old binary bound wrongly), workspace tests 209/0, regression 316/316.

E-393 *openvaf/hir_ty/src/inference.rs, openvaf/hir/src/elaborate.rs, examples/vafconstidx_examples/*, examples/lrm_examples/

A **`localparam`** could SIZE a bus but not INDEX one -- `localparam integer K = 3; electrical [0:5] n; ... V(b, n[K])` was rejected with "bus bit-select index must be a constant", though a localparam is fixed at elaboration (the LRM forbids overriding one externally) and was already accepted as a constant for array bounds, bus widths, parameter defaults, `repeat` counts and (E-392) generate bounds. The same gap rejected a plain literal expression `n[2+1]`, because the index folder recognised only a bare literal, optionally negated. E-392 left this open and named it. THREE PLACES RESOLVE A BIT-SELECT and all three had to change, because two of them run BEFORE NAME RESOLUTION and cannot ask for a parameter's value at all: (1) an index in the analog body, folded by hir_ty inference; (2) a BRANCH ENDPOINT `branch (n[K], n[0])`,

folded in `item_tree::lower` while the item tree is built; (3) a PORT CONNECTION `kid c(.p(bus[K]))`, which the instantiation elaborator resolves by synthesizing the textual name `bus[K]`. (1) is fixed semantically -- resolve the localparam and continue the fold INSIDE THAT PARAMETER'S OWN BODY, which has its own expression arena and scopes. (2) and (3) are fixed in the textual declaration pre-pass that already folds parameter-dependent WIDTHS (E-91/E-92), the only point early enough to serve them; it gains a fourth pass folding SINGLE-index brackets, while ranges and part-selects (which contain a `:`) stay with the width pass that owns them. WHICH NAMES MAY BE FOLDED IS THE WHOLE QUESTION, and both halves answer it identically: only a `localparam`, a localparam built from other localparams, and a `parameter` that E-92 FROZE INTO a localparam because it shaped a declaration width (indexing the very bus that parameter sized is then consistent by construction). A plain `parameter` is refused, and so is a localparam built from one -- a parameter binds at simulation time and baking its default into a node selection would silently ignore a model-card override, with the model still compiling and simulating against the WRONG NODE. The semantic folder refuses it STRUCTURALLY rather than by a special case (folding such a localparam requires folding the parameter, which it will not do); the textual pass mirrors this by seeding its fixpoint only from names already elaboration-constant. THE DISTINCTION THAT HAD TO BE PRESERVED: hir_ty already had a `const_int_expr` and merging the two would have been wrong -- `const_int_expr` asks "is this a constant the CODE GENERATOR can see?" (its job is spotting LLVM poison, E-333/E-334, and a localparam is lowered as a runtime value so it is deliberately excluded, which is why a localparam zero divisor still compiles as `vaf-codegen_examples/constfold.va` asserts), while the new folder asks "is this fixed before the OSDI descriptor is built?". They are kept separate, each commented with why. VERIFIED: examples/vafconstidx 25/25 (11/25 against the E-392 binary), every accept case SIMULATED over a resistor ladder and compared against the literal spelling of the same index (compiling is not the claim -- selecting the right element is); corpus byte-differential 92/92 industry models and 402 example models BYTE-IDENTICAL with ZERO bytes differing and ZERO newly-rejected files, the only change being two newly ACCEPTED files; workspace 209/0; regression 317/317. The LRM's own page-169 RC-line example (`lrm_p169_1`, `lrm_p169_2`) compiles for the first time and graduates from `limitations/` to `va/`, having needed THREE enhancements between them -- E-96 (bare module-level generate-for), E-92+E-392 (N sizes `[0:N]` so it freezes to a localparam, and a localparam is a valid generate bound) and E-393 (`n[N]` as an index); documented limitations drop 17 -> 15.

E-395 *openvaf/hir_lower/src/expr.rs, openvaf/hir/src/body.rs, openvaf/hir/src/elaborate.rs, openvaf/hir_ty/src/lower.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, ngspice-46/src/spicelib/parser/inpdpar.c, ngspice-46/src/spicelib/parser/inpgmod.c, examples/langguard_examples/**

Nine defects from a one-hour hunt at openvaf-r: three wrong-number, five silently-accepted-bad-input, one crash. (1) HEADLINE -- a SEEDED `\$random` and the ENTIRE `\$dist_*`/`\$rdist_*` family (uniform, normal, exponential, poisson, chi_square, t, erlang) return THE SAME NUMBER on every iteration of a loop, so a Monte-Carlo model written the obvious way has exactly one sample of variation in it. The LRM makes `seed` an INOUT and prescribes advancing it in place -- that fix was written and then WITHDRAWN, because it is wrong. E-10 made these builtins pure functions of `(seed, salt)` with no persistent state DELIBERATELY, and says so in a design note atop `openvaf/osdi/stdlib.c`: draws must be stable across the solver's Newton iterations, and an in-place advancing seed changes on EVERY evaluation. MEASURED, not argued: with the seed advancing, a model carrying its seed across evaluations (`@(initial_step) seed = 91;`, sigma 0.5) fails dynamic gmin stepping, true gmin stepping, source stepping AND the transient op, and aborts -- the seed had reached -1.8e+08, which is also direct proof the variable persists across evaluations. A first pass at sigma 0.01 appeared to converge; that was TOLERANCE MASKING. Trading a wrong number for a non-convergent simulator is not a fix. RESOLUTION: the purity stays, the SILENCE goes -- a new **`rng_in_loop`** lint (default Warn, `openvaf_allow`/`openvaf_deny` controllable) names the call, states the draw is constant within the loop, states why it cannot

be otherwise, and suggests a separate call site per sample. It reuses E-330's `loop_depth` counter, so it covers every loop form and not merely those with a non-constant bound. A warning rather than an error because the code is WELL FORMED. (2) `laplace_np`, `laplace_zp` and `laplace_zd` built root products as prod(s - r_k) where LRM 4.5.11 says prod(1 - s/r_k) -- the same polynomial scaled by prod(-r_k), i.e. a silently wrong DC gain. `laplace_nd` (coefficients, not roots) was right, so THE FOUR SPELLINGS OF ONE FILTER DISAGREED: on H(s)=(1+s/1e5)/(1+s/1e4) with true DC gain 1, nd gave 1.000e-03 and np/zp/zd gave 1.000e-07 / 1.000e-02 / 9.999e+01, each off by exactly the root product of whichever vector was given as roots (1e8 with two poles). FIX: `laplace_roots_to_poly` divides by the root via a guarded complex reciprocal, with the LRM's zero-root exception (a root at the origin contributes a bare `s`, not 1 - s/0) selected explicitly rather than dividing by zero. THE ONE CHANGE HERE THAT ALTERS EXISTING SOURCE, stated rather than buried: a model written against the old behaviour pre-multiplied its numerator by the root product to cancel it, and that compensation is now a double count. THIS PROJECT'S OWN EXAMPLES DID EXACTLY THAT -- `complexpole_examples` asked for a resonant LPF as `laplace_np(V(in), '{w02}, {sig, wd, sig, -wd})` where the LRM numerator is `'{1.0}`, and its OWN pre-existing oracle ("abs(np - nd) < 1e-3 dB across the sweep") failed by 272 dB = exactly w0^2 and now passes at 0.00 dB; four `laplace_examples` models needed the same. No VA_TEST corpus model uses a root form, but a user's model may: the symptom is a DC gain off by the product of the roots, and the fix is to drop the compensating factor. A separate trap surfaced there: after normalisation the two spellings no longer reach a 300 dB NOTCH NULL by bit-identical arithmetic, so at that single bin both node voltages are ~1e-15 and the dB comparison measures the LINEAR SOLVER (Sparse 7.70 dB vs KLU 9.43 dB on the same netlist), not the filter -- the check now compares only above a -100 dB floor and agrees EXACTLY at 80 of 81 points, with the null's depth still asserted separately. (3) A RUNTIME `\$table_model` with linear extrapolation CLAMPED instead of extrapolating: a runtime table's arrays have a FIXED declared size, so a model with fewer distinct knots than slots leaves repeated abscissae, which E-391 compacts to the array END -- and the end-slope search ignored which knots were live, so it could land on one of those zero-width segments, whose slope is 0, which IS a clamp. On y=2x over [0,3] in a six-slot array with the top two repeated, x=4 gave 6.0 (want 8.0) and x=7.5 gave 6.0 (want 15.0); with the repeats at the bottom, x=-0.5 and x=-2 both gave 0.0. FIX: the low slope walks the segments in reverse and the high slope forward, each guarded on a degenerate width, so the surviving assignment is the FIRST and LAST non-degenerate segment respectively. (4) `\$table_model` SILENTLY IGNORED five of the LRM's (tables 9-30/9-31) control codes -- `2`, `D`, `I`, `E` were accepted and treated as something else, and a sub-string asking for a DIFFERENT extrapolation method at each end (`1CL`) honoured only one. Now rejected by name with what to use instead; the implemented codes still compile, whitespace inside a sub-string, the `;N` dependent-column suffix and per-DIMENSION sub-strings (`1C,1L`) included. (5) `\$discontinuity;` with NO argument PANICKED the compiler -- the argument is optional (degree defaults to 0) and the lowering read args[0] unconditionally. (6) AN ACCESS FUNCTION FROM A FOREIGN DISCIPLINE SILENTLY ALIASED THE NATIVE ONE: `Zi(p,n)` on an electrical branch behaved EXACTLY as `I(p,n)` whenever the unrelated nature declared `units = \"A\"`, because `DisciplineInfo::access` asked `NatureTy::compatible`, which compares UNITS STRINGS AND NOTHING ELSE -- it could tell a potential from a flow but not which discipline the access function belonged to. Now asks `NatureTy::related` (same base nature), the LRM's own rule, already implemented right beside `compatible`; every nature DERIVED from a discipline's own still resolves. (7) A `genvar` colliding with a parameter, variable or net of the same name was not diagnosed. (8) Three instantiation validation holes left by E-392: a duplicate NAMED PORT, a duplicate NAMED PARAMETER, and a connection list MIXING positional and named form -- the mixed-form check discriminates on the DOT, not on `name()`, for E-392's own reason (`\$mfactor` has no name child). (9) NGSPICE SIDE: an OSDI parameter and each of its `aliasparam` names register as separate IFparm entries carrying the SAME `.id`, so `N1 a 0 md w=1 width=4` wrote one slot twice and a spelling lost in silence; now reported on the instance line and the model card, scoped to OSDI devices (aliasparam is a Verilog-A construct, so no built-in's parsing changes) and living in the DECK parser so `alter` -- which

legitimately re-sets a parameter -- does not trip it. The model-card message deliberately does NOT name a winner: a model parameter is written straight through so the LAST on the card wins, while an instance-parameter default is pushed with `wl_cons` and replayed in REVERSE so the FIRST wins; a rule there would be wrong half the time. WITHDRAWN, with the evidence, because it is the more useful result: a parameter whose DEFAULT lies outside its own declared range is deliberately NOT range-checked (E-56 comments the site) -- `diode_cmc` declares `CORECOVERY = 0.0 from (0.0:1.0]` and gates on `if (CORECOVERY > 0)`, so the out-of-range default IS the encoding of "feature disabled"; checking defaults would reject shipping industry models. TWO SITES CARRIED THE ANSWER: this and the withdrawn finding below were both settled by reading the comment already at the site -- but this one ALSO needed the convergence failure measured before the naive fix could be ruled out. Verify (examples/langguard_examples): 110/110 fixed, 66/109 shipped -- 44 checks pin real defects, every one paired with an accept half that asserts the same NUMBER, not merely that it compiles. TRAPS RECORDED: an AC check whose deck had no `ac` stimulus made all four laplace forms read 0.0 and three comparisons passed on 0 == 0 (the checks now require a non-zero reference); and the first table probe used a CLEAN grid, which does not meet the defect's condition and passed on BOTH binaries. CORPUS DIFFERENTIAL for (6), which changes how every access function in every model resolves: all 124 `VA_TEST` files compiled with both binaries -- 107 compiled by both, 0 rc differences, 0 byte differences, and 0 of 124 models trip the new **`rng_in_loop`** lint (a new warning must be shown not to be noise); a first pass reported 107 byte differences because it gave the two binaries DIFFERENT `-o` paths and an `.osdi` embeds its own output path. `cargo test --workspace` 209/0; full regression 319/319.

E-396 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_lower/src/expr.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/item_tree/diagnostics.rs, openvaf/basedb/src/lints.rs, examples/limguard_examples/**

Ten defects from a one-hour hunt: one hard CRASH, two silent wrong answers, seven accepted-then-degraded. (1) HEADLINE -- **`\$limit`** with any unresolvable name or arity SEGFAULTED ngspice with ZERO bytes of output, from source that compiled clean. ngspice resolves the name at load time against `pnjlim`/2, `fetlim`/1, `limitlog`/1, `limvds`/0; on a mismatch it printed "ignoring. .." and left `func_ptr` NULL, and the compiled model then CALLED that pointer. The warning never reached anyone either -- it went to STDOUT and the crash destroyed the buffer before it was flushed, which is why the symptom was a silent death and not a warning. **`fetlim`** takes 1 extra arg here and 2 in the LRM (**`vto, vgst`**), so writing the STANDARD spelling was one of the ways to crash. No safe continuation exists (the call site is already an indirect call), so an unresolvable entry is a hard load failure naming the function, both arities and the supported set, on STDERR where a later abort cannot swallow it; the compiler also raises a new **`unknown_limit_function`** lint at build time -- a lint not an error, because the set is simulator-defined. (2)+(3) A COLLISION WARNING THAT FIRED ON ALMOST THE WHOLE INDUSTRY CORPUS. E-335 warns when two OSDI parameters fold to one lowercased SPICE keyword; it compared KEYWORDS ONLY. But a model declaring one of the loader's own names has two entries under that keyword BY DESIGN -- `osdi_create_registry_entry` ROUTES its built-in to the model's parameter (`dtemp` sets `dt = param_id`, `m` sets `has_m`, `temp` suppresses the loader entry) -- so both address the SAME id and nothing is unreachable. `dtemp` is a conventional CMC instance parameter, so this fired for PSP 103/104, MEXTRAM 504/505, VBIC, BSIM-BULK/CMG/IMG/SOI, HiSIM 2/HV/SOI/SOTB, L-UTSOI, EKV, MVSG, ASM-HEMT, JUNCAP200, r2/r3_cmc -- 69 warnings across the 124-file corpus, none a real problem, each worded "declared more than once differing only in case" when the spellings are IDENTICAL. Comparing parameter IDS separates the two; the corpus now emits 5, all genuine different-id clashes. WITHDRAWN FINDING, kept because the mistake is the instructive part: this began as "a model parameter named `m` silently defeats E-394's subcircuit multiplier", on the evidence that such a model gave 1x under `X1 ... m=3`. The probe declared **`m`** and never USED it. A real CMC model declares `m` AND scales its own output by it -- which is exactly what `has_m` exists for -- so the appended `m={m}` lands on the model's parameter and the multiplier is DELIVERED through it, verified at 1x/3x/5x. A warning added for

this was removed; it would have told every CMC model in the industry that a deliberate arrangement was a bug. (4) `\$table_model` DATA FILES accepted non-finite values -- `abc` was rejected but `nan`, `inf`, `−infinity` and an overflowing `1e400` were not (`f64::from_str` accepts the first three and returns an infinity for the fourth), and ONE such token poisoned the WHOLE table, so every query -- including points interpolating between good rows -- returned NaN. A missing-data marker is exactly how measured data spells one. Validator and both `hir_lower` readers now apply the same rule so they cannot drift. (5) **`@(timer)`** with a zero, negative or denormal period fired on EVERY evaluation -- 120 events over a 10 us transient where a 1 us timer gives 10 -- so a period computed as `1/freq` with `freq=0` silently became a per-iteration event; a negative start did the same. (6) `\$bound_step` unvalidated: zero and denormal aborted with a "Timestep too small" naming neither model nor call, and negative silently forced the minimum timestep everywhere (10001 output rows against 108). (7) `noise_table` shape unvalidated: an empty or 1-entry array contributed NO NOISE AT ALL, an odd length dropped the unpaired entry, and a NEGATIVE noise power produced the same spectrum as its positive twin. (8) Out-of-range array indexing is now rejected for every COMPILE-TIME-CONSTANT index (literal, negative, write target, localparam). SCOPE BOUNDARY: a RUNTIME index stays masked -- proven memory-SAFE with a canary (reads fold to element 0, writes discarded, nothing around the array disturbed, indices -100 to 1e6); diagnosing it means a bounds check on every array access in every compact model's inner loop, a permanent cost to catch what the constant checks already catch in the form models write. (9) A BUS PORT'S TWO RANGES could disagree -- `inout [0:2] b;` beside `electrical [0:4] b;` was accepted, the DIRECTION range won, and the net's other bits were discarded so the module had fewer terminals than its source said; each declaration registers its own `BusDecl`, so every bus carrying the port's name is compared. (10) A family of unvalidated CONSTANT arguments: `@(cross)` direction not in {-1,0,+1}, negative `transition` delay/rise/fall, non-positive `slew` rise rate, zero/negative `idtmod` modulus, negative `absdelay` delay and delay > maxdelay. Checked ONLY for spelled-out constants -- deliberately narrow, with a runtime-expression case in the accept half of each. Verify (examples/limguard_examples): 81/81 fixed, 42/81 shipped -- 39 checks pin real defects. The crash check asserts a POSITIVE return code and a message naming the supported set, because the defect's signature was `rc = −11` with an EMPTY output stream, which a naive "did it fail?" test would have scored as a pass. E-394's multiplier is re-pinned end to end (1x and 3x) beside the new warning, since (2) is a regression channel for that fix rather than a defect in it. CORPUS: all 124 `VA_TEST` files -- 107 compiled by both, 0 rc differences, 0 byte differences. A CORRECTION WORTH RECORDING: the first corpus check compared only the COMPILER's output and reported "0 models trip any new diagnostic" -- true and irrelevant, because the diagnostics at issue are emitted by ngspice when the .osdi is LOADED, which that check never did. Loading all 107 corpus models is what surfaced both the withdrawn finding and the 69 spurious warnings. A differential must exercise the stage the change actually altered. Load sweep: 107 loaded, 0 load failures, 0 crashes, warnings 69 -> 5; the new `\$limit` load failure never fires on the corpus (`typedpnjlim_new` and friends live only inside \ifdef XYCE`branches that are never compiled, and VBIC'slimRTHis a user−defined analog function)`.cargo test -- workspace' 209/0; full regression 320/320.

E-398 *openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/item_tree/diagnostics.rs, openvaf/hir_def/src/data.rs, openvaf/hir/src/lib.rs, openvaf/hir_lower/src/expr.rs, examples/paramsetguard_examples/**

Four silent **`paramset`** defects. HEADLINE: **`paramset`** was the ONLY supply path that bypassed parameter RANGE validation. With `parameter real k = 1.0 from (0:inf);` in the target, `paramset dut basemod; .k = −1.0; endparamset` put -1.0 into the model with no word from either tool -- while a model card, an instance line, `alter`, `altermod`, a `.param` and a subcircuit parameter ALL reject it and abort with "Parameter k is out of bounds!". Six paths enforced the range, one did not; `exclude` and integer `from [1:3]` were bypassed identically, as was the open-bound `.k = 0.0`. ROOT CAUSE: binding an override turns the target parameter into a LOCALPARAM, and `param_body_with_sourcemap` returned `bounds: Vec::new()` for it -- the

constraint was discarded before anything could check it, so `insert_param_init` had nothing to emit, and because a localparam is not settable ngspice's runtime validation never saw the parameter either. The value fell into the gap between the two checks. FIX: check in `lower_paramset`, where the override and the target's constraints are both still syntax; folds LITERAL values only, because an override built from the paramset's own netlist-settable parameters is not knowable there -- boundary pinned by an accept-half case. NOT a re-run of E-56: that deliberately refuses to range-check a parameter's DEFAULT (CMC models use an out-of-range default to mean "feature disabled" -- `diode_cmc`'s `CORECOVERY = 0.0 from (0.0:1.0]`), and a paramset override is a SUPPLIED VALUE, not a default; that distinction is exactly why the netlist path checks and the default path does not. (2) An override naming a parameter the target does not declare was DROPPED IN SILENCE -- the binder looks each name up among the target's parameters and a name matching nothing simply never bound; the netlist path reports it ("unrecognized parameter (...) - ignored") and E-392 established this same check for `#(.param())`. (3) The same parameter assigned TWICE was accepted and the FIRST won (the binder takes the first match); E-395 reports this for netlist lines. (4) **`$param_given`** reported FALSE for a paramset-supplied value: a bound parameter is a localparam so it has no runtime given-flag and `ParamKind::ParamGiven` resolved to false -- netlist `basemod(g=5e-3)` gave 0.005 and "given", paramset `.g = 5e-3` gave 0.005 and "NOT given". `$param_given` is the standard CMC idiom for "did the user specify this, or is this my default?" (deriving one parameter from another only when unset), so through a paramset EVERY such derivation silently took the default branch while running the paramset's value; now reports given, while an ordinary localparam still reports not-given. SCOPE: paramset BINNING clauses (`.w from [0:10]`, LRM 6.4's geometry selection) remain unsupported and are a clean parse error -- a missing feature, not a wrong answer. Verify (examples/paramsetguard_examples): 41/41 fixed, 22/41 shipped -- 19 checks pin real defects. Every rejection is checked TWICE (that it is rejected, and that the message names the offending value and range, or the parameter and target module); the accept half pins the seven ways a paramset must keep working -- inside the range, BOTH closed bounds, no range declared, outside an `exclude`, an expression override, two different parameters, and a paramset of a paramset -- and the suite re-checks that the six already-enforcing paths still enforce, since the whole argument is that they were right. CORPUS: 124 files, 107 compiled by both, 0 rc differences, 0 byte differences, 0 models trip any new check (no corpus model uses a paramset, so that confirms the checks target a mistake, not practice). `cargo test --workspace` 209/0; full regression 322/322.

E-399 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/lower.rs, openvaf/hir_def/src/expr.rs, openvaf/hir_def/src/body/lower.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/item_tree/diagnostics.rs, openvaf/hir_lower/src/stmt.rs, openvaf/basedb/src/lints.rs, openvaf/basedb/src/diagnostics/preprocessor_error.rs*

Ten compiler-side defects from round 13, all one shape: input a careful author could write, accepted without a word, different answer. (1) A CONCATENATION `{..}` WHERE THE LRM WANTS AN ARRAY LITERAL `'{..}` made `\$table_model(2.0, {1,10,2,20,3,30})` return 0.0 against the array literal's 20.0, and `noise_table({..})` contribute EXACTLY NO NOISE while its `'{..}` twin worked -- every check here read `if let Expr::Array(..)` and a concatenation is a different variant, so the check was skipped rather than failed. Parameter-array init already rejected the same mistake. Only a concatenation is refused; an array-VARIABLE reference (E-4) still compiles, and `laplace_*` is deliberately excluded because it lowers `{1.0}` correctly ( measured identical AC response) and arraycast_examples relies on it. (2) NO ANALYSIS-NAME STRING WAS VALIDATED ANYWHERE: `analysis("tarn")` is false in every analysis and `@(initial_step("tarn"))` fires never, so a typo turns a whole branch or initialisation block into dead code -- new `unknown_analysis_name` lint (L021), a warning like E-396's `\$limit` treatment since the set is simulator-defined (ac, dc, ic, nodeset, noise, static, tran). (3) AN EVENT `or` LIST WAS TRUNCATED AT ITS FIRST `)`: `@(initial_step("nonsense") or final_step)` never fired though `@(final_step)` does -- a good event discarded because an EARLIER member hap-

pened to take an argument. (4) `@(cross/above/timer)` ACCEPTED ANY NUMBER OF ARGUMENTS AND NEVER CHECKED THEIR TOLERANCES, because the surplus was dropped by an unconsumed iterator and the tolerances were not in the HIR at all; `@(above)` takes THREE arguments per LRM 5.10.3, not two. (5) `@(cross())` WITH NO ARGUMENTS DEGRADED THE WHOLE EVENT CONTROL to an unconditional body, so the guarded statement ran on every evaluation; rejected in validation, which is where it must happen because lowering panics on a missing expression. (6) TWO NATURES THAT BOTH OMITTED `units` COMPARED COMPATIBLE because `None == None`, so a branch spanning two unrelated disciplines compiled in silence -- differing units and one-omits were both already rejected, only both-omit fell through; an absent units string is no longer evidence, falling back to the LRM's same-base-nature rule so `nature Derived : Base` still branches against Base. (7) `U+0060`, THE ASCII BACKTICK, WAS LISTED AS A UNICODE LOOKALIKE OF ITSELF, so a stray `` `endif `` advised replacing a character with itself over text byte-identical to the input, never naming the real problem. (8) A DECLARED RANGE NO VALUE CAN SATISFY (`from [3:1]`, `from (1:1)`) left the parameter silently unsettable -- the default bypasses range checking by design (E-56) so it still reads, but every netlist-supplied value is rejected at run time.

E-452 *openvaf/openvaf-driver/src/cli_process.rs, examples/optpath_examples/ (new)*

AN UNUSABLE **`-o`** DESTROYED THE SOURCE, OR CRASHED. Three destinations the driver never checked, each reaching the backend and failing there. (1) `-o` NAMING THE INPUT wrote the compiled module straight over the source and reported SUCCESS -- `openvaf -r m.va -o m.va` exits 0 with 'Finished building' while turning 111 bytes of Verilog-A into a 36888-byte Mach-O; the source is simply gone, with no warning and nothing in the output to suggest it. Reachable from a shell loop whose output variable is accidentally the input one. (2) An EMPTY `-o` reached `dst.file_stem().expect("destination is a file")` at osdi/src/lib.rs:107 and (3) an UNWRITABLE directory reached `assert_eq!(llmod.emit_object(..), Ok(()))` at line 579 -- both PANICKED: exit 101, a crash banner, a crash-log file and a request to open a GitHub issue, for a typo. `emit_object` already returns a `Result`; the caller asserted on it instead of propagating it. Now validated in `matches_to_opts` BEFORE any parsing -- the last point that still knows both the input and the requested output -- so a path naming no file, a path resolving to the input, a missing directory and an unwritable one each cost one line and exit 65. Overwriting a previous OUTPUT stays legal (rebuilding onto an existing `.osdi` is the normal workflow, so only the input is protected, pinned by a control), and writability is tested by creating and removing a probe file rather than reading permission bits, because the backend writes one object file per module BESIDE the target -- which is exactly what line 579 was failing to do. Verified optpath 16/16 and 9/16 on the previous compiler, the seven failures there being exactly the defect checks while every control passes on both; the whole model corpus recompiled with the new driver, regression 365/365.

E-400 *openvaf/sim_back/src/diagnostics.rs (new), openvaf/sim_back/src/dae.rs, openvaf/sim_back/src/dae/builder.rs, openvaf/sim_back/src/dae/tests.rs, openvaf/sim_back/src/context.rs, openvaf/sim_back/src/lib.rs, openvaf/sim_back/src/init/tests.rs, openvaf/hir/src/body.rs, openvaf/hir/src/lib.rs, openvaf/basedb/src/lints.rs, openvaf/osdi/src/lib.rs, openvaf/osdi/tests/data_tests.rs, openvaf/openvaf/src/lib.rs*

A branch contributed as BOTH a potential and a flow source kept only the LAST contribution and discarded the other in silence -- identical physics, a different answer, decided by statement order. Through a 1 V source and 1 kOhm, `V(a,b) <+ 0.4; I(a,b) <+ 1e-3;` gives v(a)=0.0 / i=-1.0 mA (behaving as I-only) and the reverse order gives 0.4 / -0.6 mA (V-only); each matches the corresponding single-contribution model exactly. The round-13 finding E-399 left open. THE DETECTION E-399 PREDICTED DOES NOT WORK: it expected `BranchInfo`'s other-kind contribution to be non-trivial while `is_voltage_src` is constant, but `hir_lower::stmt::contribute_value` resets the OPPOSITE place to zero as it writes -- that statement IS the discard -- so instrumenting `build_branch` shows `V;` `I;` and `I;`-alone arriving byte-identical. The evidence is therefore assem-

bled from THREE stages, none sufficient alone: the DAE build knows the branch is not a switch (a genuine switch branch has a RUNTIME `is_voltage_src` and takes the third arm of `build_branch`, where both kinds stay live); the HIR still holds both statements with their spans and lint attributes, exposed by a new `Module::contribution_sites` that buckets contributions by branch, normalising node order and ground references the way `hir_lower` does so `I(a,b)`/`V(b,a)` and `V(a,gnd)`/`V(a)` resolve to one branch; and the UNOPTIMIZED MIR decides whether the model or the OPTIMIZER made the branch single-kind. THAT LAST DISTINCTION IS THE RELEASE'S REAL WORK. SCCP also folds an `if` whose condition is a configuration constant: BSIMSOI writes the switch-branch idiom correctly around a `B4SOIbodyMod` test (`V(b,p) <+ 0;` in one arm, two `I(b,p)` contributions in the other), and `B4SOIbodyMod` is a plain 0 whenever `PORT_CONNECTED` is undefined, so the `else` is provably dead and `is_voltage_src` folds to TRUE -- indistinguishable, in optimized MIR, from a single-source branch with a stray contribution. `Context::new` therefore records `unconditional_branch_kind` before any pass runs; lowering has already collapsed a phi whose arms AGREE, so a branch that is one kind on every path the author wrote still reads constant, while BSIMSOI's differing arms stay a phi. BSIMSOI was flagged before that refinement and is silent after it. SECOND HALF OF THE WORK -- `sim_back` had no way to report anything raised during the DAE build; rather than invent a channel, the one `collect_modules` already takes was extended: a single `ConsoleSink` now lives for the whole compilation in `openvaf::compile` and is threaded down to `DaeSystem::new`, so this is a real lint and not an `eprintln!` -- `--allow`/`--deny` and `(* openvaf_allow="discarded_contribution" *)` on the statement, block or module all work, and `--deny` removes the emitted object files and links no `.osdi`. A THIRD NARROWING CAME FROM BSIM4: its standard `rdsMod` idiom pairs a conditional `V(s,si) <+ 0.0` collapse with an unconditional `I(si,s) <+ white_noise(..)` ~150 lines later, which fixes the branch as a flow source on every path -- but a literal-zero contribution carries no value to lose, because `hir_lower::stmt::contribute` recognises it as a NODE-COLLAPSE REQUEST delivered by a `CollapseHint` callback and not by the residual. A discarded contribution of literal zero is therefore not reported, using the same `is_zero` test lowering itself uses; the reverse pairing `I(a,b) <+ 1e-3; V(a,b) <+ 0;` still is, because there the discarded side is the 1e-3. NOT REPORTED, each measured rather than reasoned about: the if/else switch branch (including the op-dependent form and the LRM's own page-114 `relay` and page-155 `parares`), `I(..); if (sw) V(..);` (still a runtime switch), a bare `V(a,b) <+ 0;` node collapse, and a switch whose selector folds. REPORTED: both orderings, named branches, reversed nodes, ground references, a discarded REACTIVE potential (`ddt`), an indirect contribution `V(out) : V(a,b) == 0;` followed by a flow one, `I(..); V(..) <+ 0;`, and `if (sw) V(..); I(..);` where the potential can never win. ONE REAL MODEL IS WRONG: of 124 `VA_TEST` models exactly one is reported -- FBH HBT 2.3, three times (`niiy`, `niiiy`, `nivy`), where `I(niiy) <+ Iniix; V(niiy) <+ I(niiy);` sit in sequence and BOTH arms of the enclosing `if (Fcorr==0)` leave the branch a potential source, so the flow contribution is dead on every path and the correlated-noise network does not do what those two lines say. CORPUS: 107 compiled by both at the SAME `-o` path, 0 rc differences, 0 byte differences -- this release changes what the compiler says, never what it emits. A SECOND SWEEP over all 515 `.va` files this repository ships (`examples/` + `integration_tests/`: BSIM3/4/6, BSIMSOI, BSIMBULK, HICUM, HiSIM, PSP, MEXTRAM, ASM-HEMT, DIODE_CMC...) trips the check 0 times; both narrowings above were found THERE and in the corpus, not reasoned out in advance -- the first draft flagged BSIMSOI and BSIM4, and both are correct code. `cargo test --workspace` 210/0 (209 plus `sim_back::dae::tests::discarded_contribution`); full regression 322/322.

E-401 *openvaf/sim_back/src/dae/builder.rs, openvaf/sim_back/src/dae.rs, openvaf/sim_back/src/lib.rs, openvaf/sim_back/src/dae/tests.rs, openvaf/sim_back/src/init/tests.rs, openvaf/osdi/src/lib.rs, openvaf/osdi/src/compilation_unit.rs, openvaf/osdi/header/osdi_0_4_enhancement3.h (new), openvaf/test_data/{dae,init,osdi}/*.snap*

**`V(a,b) <+ 0`** between two module TERMINALS connected NOTHING -- a silent open circuit where the model wrote a short. `V(a,b) <+ 0` lowers to a node-COLLAPSE request (a `CollapseHint` callback, never a residual), and ngspice cannot honour one whose endpoints it allocated

itself; `build_branch`'s potential arm meanwhile builds no equation for a trivial contribution. Neither is wrong alone; together they drop the branch. The LRM's own page-155 `parares` therefore became an OPEN at small r (`i(v1) = 0.0` against -5.0e-4), and `diode_lim.va` read 1000 V of self-heating rise at its default `rth=0`. FIX: `collapse_is_ignored` (every non-ground endpoint is a port) makes the branch carry a real 0 V source, in BOTH the constant-potential arm and the switch arm -- `parares` takes the SWITCH arm, since its condition is parameter-dependent so `is_voltage_src` is a runtime value while `op_dependent` is false, and the branch had been dismissed as "just node collapsing". That alone is WRONG, and the regression is what proved it: a 0 V source between two nodes the netlist already shorted is SINGULAR (collapsing is idempotent, an equation is not), which broke five examples. So the compiler also EXPORTS the branches -- `OSDI_TERM_SHORT_COUNTS/_INFOS`, `{node_1, node_2, flow_node}`, the same ADDITIVE symbol pattern as absdelay and last_crossing, documented in a new `osdi_0_4_enhancement3.h`; no `OsdiDescriptor` layout change and no ABI version bump -- and ngspice drops the branch current whenever the terminals do not resolve to two distinct connected circuit nodes. Recorded ONLY where the model never reads the branch current, which is what makes dropping it safe. Terminal-to-internal collapse untouched and still exact. Regression 322/322, `cargo test --workspace` 210/0 (six snapshots regenerated, each a `collapsible` entry becoming a `flow(..)` unknown plus its equation), corpus 107 compiled by both, 0 rc differences, 20 byte differences -- the first bytes this run has moved, all thermal models that ground a thermal terminal, correlating exactly with the new metadata; HICUM L2 on a normally-wired circuit is -1.86927e-02 on both binaries.

E-403 *openvaf/hir_ty/src/inference.rs*

A type mismatch on **`ac_stim`** reported "expected string literal, string literal or string literal". The signature checker collects the requirement at the failing argument position from EVERY surviving candidate signature and hands the list to a renderer that joins it verbatim; `AC_STIM` has four signatures and three of them take `Literal(String)` at argument 0, so the same expectation was printed three times. The list is now deduplicated with first-seen order preserved. Genuinely distinct alternatives are untouched -- `\$limit` still reports "expected string literal or function". The other half of this release is ngspice-side (five device noise routines adding a nominal temperature in Celsius to a temperature difference); see the ngspice report. Corpus differential 107 compiled by both, 0 rc differences, 0 byte differences -- the change touches a diagnostic string only, and the bytes confirm it. `cargo test --workspace` 210/0; regression 322/322.

E-404 *openvaf/hir_def/src/item_tree/lower.rs, openvaf/sim_back/src/dae/builder.rs*

A module declaring a wide bus compiled in time QUADRATIC in the bus width -- 31.5 s for **`[65535:0]`**, roughly 4x for every doubling. The second of the two round-13 findings that E-399 left open, and recorded there as quadratic AND NOT A HANG -- a distinction that came from measuring the scaling rather than from watching a wide compile appear to stall, and one that pointed straight at a linear scan per bit. PROFILED RATHER THAN GUESSED: `sample` on a live `[65535:0]` compile put 9906 of 9912 samples in `Ctx::lower_port_decl` with essentially nothing below it. `find_node_for_decl` ran up to THREE full `nodes` scans per declared bit -- the first two are one logical lookup split in two to satisfy the borrow checker -- while `nodes` itself grew to N, and the module-head port loop held a fourth in `nodes.iter().all(..)`. An `FxHashMap<Name, LocalNodeId>` now answers all four in O(1), maintained by a `push_node` helper so all five push sites keep it in step, holding the FIRST node under each name because that is what the linear `find` it replaces returned. Two details decide correctness: the merge path RENAMES a node -- the first bit of a bus claims the module-head placeholder declared under the bare base name and becomes `a[0]`, so the index entry moves with it and later bits then find no placeholder and push, exactly as the scan did -- and the original scan is KEPT as a fallback for a name whose first node is already claimed, which nothing well-formed can produce (the head loop dedups by name, every other push carries a distinct bit name) but which makes the semantics identical rather than identical-in-the-cases-considered. THE BOTTLENECK THEN MOVED, SO THE PROFILE WAS TAKEN AGAIN: that change alone gave 31.5 s -> 2.4 s, but the top of the table still grew ~3x

per doubling, and re-profiling against a temporary debug-info build (fat LTO had collapsed the frames) put 100% of samples in sim_back's **`build_jacobian`** -- not bus elaboration at all, but the remaining quadratic in this compile. It reserved a DENSE `unknowns^2` jacobian that is never built (4.3e9 entries at `[65535:0]`) and sparsified by walking every column of every row, one row per unknown. Instrumenting the builder showed what that scan was looking at -- `unknowns == residual == N+2` with `sim_unknown_reads` EMPTY, so the entire scan ran over zeros; every declared node becomes a simulation unknown before collapsing. The row build now records the columns it actually reaches and sparsifies only those, and the reservation drops to the diagonal. The sort is the part that matters: the dense scan emitted a row in ascending column order and insertion order would not, so `SimUnknown`'s `Ord` is used for a `sort_unstable + dedup` that restores exactly the old order -- which is what makes this a pure speedup and not a re-layout of the matrix. RESULT: `[65535:0]` 31.50 s -> 0.62 s (51x), and scaling is linear out to `[262143:0]` at 2.80 s, four times wider than anything measured before. OUTPUT BYTE-IDENTICAL, TWICE OVER: corpus differential at the same `-o` path gives 107 compiled by both, 0 return-code differences, 0 byte differences, and because NO corpus model declares a wide bus that was repeated on modules that do -- `[15:0]`, `[255:0]`, `[1023:0]`, `[8191:0]`, plus `[7:0]`, `[63:0]` and `[255:0]` giving EVERY BIT ITS OWN CONDUCTANCE so a mis-mapped bit would move the bytes: all seven byte-identical. Bit-to-node identity was also measured in the simulator rather than inferred (an 8-bit bus whose bit k conducts (k+1) mA reads i(v0..v7) = -1.0e-3 .. -8.0e-3 on both binaries). No cost to real models: bsim4 1.47 s before, 1.48 s after, min of three. `cargo test --workspace` 210/0; full regression 322/322. Severity is LOW -- no real compact model declares a bus at this scale -- and the fix was made on that basis, trading nothing for the speed.

E-405 *openvaf/hir_lower/src/expr.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs, openvaf/hir/src/body.rs, openvaf/hir/src/elaborate.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/test_data/ui/ddx.log*

The four z-domain filter forms exist to express the SAME filter, and they disagreed: one pole at z=0.5 gave DC gain 2.0 through **`zi_nd`**/**`zi_zd`** coefficients and -1.0 through **`zi_np`**/**`zi_zp`** roots. -1.0 is exactly 1/(1 - 1/0.5) -- the root DIVIDED `z^-1` instead of multiplying it, so a pole written 0.5 landed at z=2; passing 2.0 reproduced the pole at 0.5, zeros reciprocated identically, and complex conjugate pairs matched the reciprocal prediction exactly (1.0 and 2.5), so it was systematic. `lower_zi` called `laplace_roots_to_poly`, which builds the s-domain `prod(1 - s/r)`; the z-domain factor is `(1 - rho*z^-1)`. A new `zi_roots_to_poly` builds that, with NO zero-root special case -- the LRM's exception exists because `(1 - s/r)` divides by the root and `(1 - rho*w)` does not. THE COMMENT IN THAT VERY FUNCTION SAID SO: E-395 ended with "The `zi_*` family is separate and already correct: the z-domain form is 1 - z^-1*rho, where the root MULTIPLIES rather than divides" -- it was not separate, `lower_zi` called straight into the function that comment lives in. Both that claim and the function's own header (still describing the pre-E-395 `prod(s - r)`) are corrected. `laplace_*` was and remains right: DC gain 1.0 for a lone real pole, a conjugate pair, and the zero-at-origin exception. A COMPILER HANG AT 38 GB: `zi_nd(x, '{1.0}, '{}, T, 0)` never terminated, RSS oscillating 8-38 GB, because `den.len() - 1` UNDERFLOWED to usize::MAX and the bilinear expansion looped over all of it -- `num` beside it was already saturating. Predicted from the source then confirmed: `zi_np`/`zi_zp` are SAFE, their roots going through an expansion that always returns at least `[1.0]`. Both subtractions now saturate and hir_ty rejects an empty coefficient denominator outright. AN IMPROPER NUMERATOR WAS SILENTLY TRUNCATED: the controllable-canonical realization carries at most a feedthrough term, so `laplace_nd(x, '{1,tau}, '{1})` quietly became the constant 1, `'{1,tau,tau^2}` over `'{1,tau}` returned EXACTLY the `1/(1+s*tau)` answer, and the pure differentiator `'{0,tau}` over `'{1}` returned IDENTICALLY ZERO. Such a filter has unbounded gain at high frequency and no state-space realization, so it is rejected rather than approximated. ONLY the s-domain: in `z^-1` a higher-order numerator is an ordinary FIR filter and `lower_zi` already pads both polynomials, so the first draft would have broken working filters -- the narrowing came

from MEASURING `zi_nd('{1,0.5,0.25}, '{1})` at its correct DC gain of 1.75. PARAMETER ARRAYS NOW TAKE A RUNTIME INDEX: `for (k=0;k<3;k=k+1) y = y + ap[k];` was refused with "bus bit-select index must be a constant" -- for something that is neither a bus nor a bit-select -- while the identical loop over a `real arr[0:2]` variable had worked since E-14. Parameter elements are ordinary MIR values, so the same select chain applies; 1-D and 2-D, parameter and localparam, real and integer. The read path is a SEPARATE type from the dynamic-index WRITE path so that `p[i] = v` cannot start looking expressible, and a vectored NET still needs constant indices because its bits map to distinct simulator unknowns. ARRAY BOUNDS NOW ACCEPT CONSTANT EXPRESSIONS: `[0:2]`, `[-1:1]` and a named `[0:P]` worked, but `[0:3-1]`, the ubiquitous `[0:`N-1]` and even the parenthesised `[(0):(2)]` did not. A small syntactic folder handles parens, unary sign and integer `+ - * / % << >>`; deliberately NOT `const_int_expr`, since it runs in the item tree BEFORE name resolution and must not resolve names -- every step is checked, so an overflowing bound reports as non-constant instead of wrapping. A DECLARED **`genvar`** WAS REPORTED AS UNDECLARED, byte-identical to the message for a name that never existed, because generate elaboration erases genvar declarations textually before name resolution runs; it now says what it is. The LRM's own pages 91, 117 and 134 use the construct, and those three stay pinned as known limitations, re-pinned to the new message. ONE FINDING WITHDRAWN: `ddx` of an unnamed branch flow was made valid and `test_data/ui/ddx.va` immediately failed -- it lists `I(a,c)` and `I(<a>)` under "these must be rejected", so the restriction is deliberate and the change was reverted rather than overriding it. What WAS wrong is the help, which advertised `I(a,b)` as accepted inside the very error rejecting it. ONE ROOT-CAUSED AND LEFT ALONE: compile time grows ~2.5-2.9x per doubling of the PARAMETER COUNT (not arrays -- scalars scale identically, and `-O0` shows the same shape, so not LLVM). Profiling at -O0 puts 12299 of ~14358 samples in `mir_opt::simplify_cfg::iteratively_simplify_cfg`, which rescans EVERY block each round until a round changes nothing. The fix is a worklist, which rewrites a fixed-point optimiser whose simplification ORDER can change the result -- not clean and local, so it is recorded rather than attempted; bsim4's ~1000 parameters compile in 1.5 s. CORPUS 107 compiled by both, 0 rc and 0 byte differences (no corpus model uses a z-domain filter); regression 322/322; `cargo test --workspace` 210/0. The regression earned its keep twice -- it caught a draft rejecting an empty numerator, a documented H=0 case the example suite asserts, and then caught the new order helper computing `0 - 1` on a usize, the exact underflow this release fixes.

E-406 *openvaf/hir/src/body.rs, openvaf/hir/src/lib.rs, openvaf/sim_back/src/module_info.rs, openvaf/sim_back/src/diagnostics.rs, openvaf/basedb/src/lints.rs, examples/probeshort_examples/ (new)*

Two 1 kOhm sections in series draw 0.5 mA; adding one line that only READS a current made them draw 1.0 mA, rc=0, no diagnostic. `branch (a,mid) br;` with `I(a,mid) <+ ..` contributed and `iprobe = I(br)` probed doubles the terminal current, because a declared branch and the node pair it spans are DIFFERENT branches and probing the flow of a branch nothing contributes to makes it an IDEAL AMMETER (a 0 V source, E-36) -- which then lands in parallel with the real branch and shorts it. A probe changed the circuit. THE SEMANTICS ARE NOT THE BUG and are deliberately unchanged: the DAE keys `BranchWrite::Named` and `Unnamed` as distinct unknowns, E-400's `same_branch` returns false across the two, measurement agrees (`V(br) <+ 0.4` beside `I(a,mid) <+ 1e-3` yields 0.4 -- a voltage source in parallel with a current source -- and raises NO discarded-contribution lint, while same-spelling pairs do), and the LRM compliance notes document the ammeter as intended. Merging the branch kinds would break a documented feature; the silence was the defect. New `probe_only_branch_short` lint (L023, warn) fires when a branch's flow is probed, the branch has no contribution of its own, AND another branch spanning the same node pair IS contributed to. That third clause is why the naive rule is wrong: "warn on any probe-only branch" would flag the deliberate sense-ammeter idiom, where nothing else drives the pair and the short IS the intended circuit -- SIX branches in the shipped corpus rely on it. Node order is normalised, so `branch (mid,a)` against `I(a,mid)` is caught, as are both directions of the mistake. NO MIR NEEDED: the check sits beside module collection and never touches the DAE, because both facts are already in the HIR -- E-400 collects contributions and a new `Module:`

`:flow_probe_sites` collects flow probes, mirroring the branch resolution `hir_lower` performs for `BuiltIn::flow`; a branch present among the probes and absent from the contribution map IS the probe-only case. Doing it there buys the label that matters: the report points at the PROBE, not at the correct code around it. Two details make that work -- lint attributes attach to STATEMENTS not expressions, so each probe is anchored on its innermost enclosing statement (`(* openvaf_allow=\"probe_only_branch_short\" *)` works on the probing statement and every enclosing scope), and a named branch declared over a PORT is skipped since lowering routes it to the port's own flow. SILENT on: same spelling both sides, the deliberate sense ammeter, a declared-but-never-probed branch, a potential-only probe (`V(br)` cannot short anything), a port-flow probe, and a module with no branches. ZERO false positives over every **`.va`** file this repository ships (640 files, 552 compiling). `--allow` silences, `--deny` gives rc=65, `--lints` lists it. Corpus differential 107 compiled by both, 0 rc and 0 byte differences -- this release changes what the compiler says, never what it emits; regression 322/322; `cargo test --workspace` 210/0. FOUND BY digging into the tenth finding of the E-405 hunt, whose description -- "the DC solve fails" -- was WRONG IN THE DIRECTION THAT MATTERS: the failed solve was an artifact of that probe circuit putting a voltage source directly across the shorted branch, and in an ordinary topology nothing fails, the answer is simply wrong. That is why this got a lint rather than a footnote. THE NEIGHBOURING MESSAGE WAS STALE TOO: the deliberate sense-ammeter case draws `trivial_probe` (L017), which read "Current probe always returns zero" with a note that "branches are open circuted by default". Both describe the behaviour E-36 REPLACED -- probing such a branch synthesises the LRM's 0 V source, so it is an ideal ammeter and the probe reads whatever current then flows; MEASURED 1.0e-3 A while the message insisted on zero. Wording only: the firing condition is untouched and the corpus fires it the same 9 times across 5 files, including E-36's own `signalflow_demo` where the new text finally describes what the example demonstrates. THE TWO LINTS DO NOT OVERLAP: `hir_ty`'s validator reduces a named branch to `BranchWrite::Unnamed` before recording it, so a node-pair contribution marks the declared branch non-trivial and L017 stays silent on the trap, while the DAE keeps the two distinct -- which is what creates the phantom ammeter. They partition the space exactly: L017 when nothing drives the pair, L023 when something drives it through the other spelling. That frontend/backend disagreement is left alone: both models are self-consistent, the DAE's is the documented one, and changing either would move behaviour rather than wording. THE FOUR CONTROLLED SOURCES are pinned in the example, since VCVS/VCCS/CCVS/CCCS are where a false positive would hurt most and the two current-controlled families MUST probe a branch current: every ordinary spelling of all four is silent under L023 and gives the right answer (VCVS 10.0, VCCS -1.0, CCVS 0.1 V, CCCS -100 V), while the ONE module L023 reports -- `cccs_mixed`, driving the node pair with `V(ps,ns) <+ 0` and probing the declared branch, so two 0 V sources sit in parallel across the same nodes -- does not merely answer wrong, it FAILS TO SOLVE. Zero false positives, one true positive. The bare sense-branch forms WORK and draw the advisory L017, which is exactly where the old wording was worst: it told the author of a working current-controlled source that the probe "always returns zero" while the model delivered rm*I_sense and beta*I_sense. One trap found while writing it and NOT a compiler defect: naming the modules plainly made ngspice refuse them, because it registers BUILT-IN vcvs/vccs/ccvs/cccs devices ("device is already registered", then "incorrect model type!"); openvaf's `reserved_module_name` lint (L018) had said so at compile time and its help text predicted that precise failure -- missed only because the first sweep grepped for L017 and L023 alone. The example asserts the file is L018-clean. Regression 323/323 with the new example.

E-407 *openvaf/hir/src/elaborate.rs, examples/genvarloop_examples/ (new), examples/lrm_examples/*

Three examples taken VERBATIM FROM THE LRM -- the page-91 ADC, the page-117 DAC and the page-134 genvar expression -- did not compile, and all three fail the way the standard itself writes the construct: a `genvar` for-loop inside an `analog` block. The loop has to be a genvar because a vectored net's bit-select must be a CONSTANT (each bit is its own simulator unknown), so an ordinary counter is refused with "bus bit-select index must be a constant" and a run-time loop cannot express "contribute to every bit" at all; declaring the index a genvar is how the LRM

says UNROLL THIS AT ELABORATION. THE STANDARD SUPPLIES ITS OWN ORACLE: page 117 ships BOTH forms in one file -- `dac` with the genvar loop and `dac8` hand-unrolled -- so the unrolled form is not an interpretation but the specification written out. Driven identically the two now agree exactly at 1.953125e-02 (= 1/64 + 1/256 for the two bits set, checked by hand rather than merely for equality), and the page-91 ADC matches a hand-unrolled copy bit for bit (o7..o0 = 1,0,1, 1,1,1,1,1 at in=0.75). THE ELABORATOR ALREADY HAD EVERY PIECE, AND IN THE RIGHT ORDER: `fold_parameter_widths` runs BEFORE `elaborate_generates`, and E-92 rewrites any `parameter` that shapes a declaration width into a `localparam`, so by unroll time `bits` in `out[0: bits-1]` is already a compile-time constant; `substitute_index` already replaces an identifier with a literal AND folds the resulting bit-select brackets. What was missing was the loop itself. Two details were not free: (1) the existing evaluator has NO RELATIONAL OPERATORS -- every previous caller folds a width or an index, never a predicate -- so a new `eval_const_cond` splits on the single top-level relational operator and folds each side with the existing arithmetic evaluator, and it must accept a two-character operator arriving either as ONE token or as TWO adjacent ones, because handling only the split form silently missed every <= and >=, which is most real loop conditions and all three LRM examples; (2) E-406's genvar diagnostic had to MOVE, since it rejected any genvar in an analog block before the unroller could see it -- it now runs on the UNROLLED text, where a genvar the unroller consumed leaves no trace and one that survives is a genuine misuse. DELIBERATELY NARROW: only a `for` whose index is a declared genvar and whose init, condition and step all fold to integers; an ordinary `integer` loop stays a run-time loop and a module-level `generate for` still elaborates through its own path. Since every affected shape was REJECTED OUTRIGHT before, nothing that previously worked can change. Each remaining failure keeps its own message -- a genvar read as a value, a bound on a settable `parameter`, and a runaway loop capped at 4096 statement copies (E-148's reasoning: each iteration is a full copy of the body). RESULT: `lrm_examples` goes from 44 accepted / 15 limitations to 47 / 12, suite 7/7 on both solvers -- three files verbatim from the standard, previously uncompilable. Sweeping every `.va` file this repository ships, the count that compiles rises by exactly those three (plus the two files this release adds); the three still failing with a genvar-related message are pre-existing `generate if` limitations, untouched. Regression 324/324, `cargo test --workspace` 210/0, corpus differential 107 compiled by both with 0 rc and 0 byte differences -- no corpus model uses the construct. FOUND BY following up E-406, which noted the LRM's own pages use this and that supporting it was a feature worth having; the feasibility check came FIRST -- hand-unrolling the page-91 ADC showed it compiled in 0.12 s and simulated correctly, proving the target form was already fully supported and only the unrolling was missing.

E-414 *openvaf/hir/src/elaborate.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/item_tree/diagnostics.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation/types.rs, openvaf/hir_ty/src/inference.rs, openvaf/sim_back/src/module_info.rs, openvaf/basedb/src/lints.rs, openvaf/basedb/src/diagnostics/sink.rs, openvaf/openvaf-driver/build.rs, examples/elabguard_examples/ (new)*

A genvar loop body of any statement shape, and the **`else`** that changed which **`if`** it belonged to. The analog genvar unroll is TEXTUAL, so it must find where one statement ends; `statement_extent` knew exactly two shapes -- a `begin`..`end` block and "everything through the next top-level `;`" -- so every other statement was cut at the first `;` inside it and the remainder spliced in AFTER the generated block. Usually a spurious parse error at `endmodule`; once a WRONG ANSWER, because for `if (d) for(i..) if (cc) x=..; else x=..;` the orphaned `else` re-attached to the ENCLOSING `if` -- rc=0, no diagnostic, three of four (d,cc) combinations wrong, and the combination anyone would spot-check was the one that agreed. Now scanned by statement shape, recursively: `case`/`casex`/`casez`, `if` with a block branch, `if`/`else`, `while`, `repeat`, nested `for`, `do`..`while`, `@(event)` and a nested genvar loop all work as unbraced bodies. A `begin : blk` label is suffixed per iteration (N copies used to collide), and the expansion is line-preserving so a diagnostic below a loop keeps its line. Second correctness fix, found while root-causing a crash: E-92 freezes a parameter that shapes a declaration WIDTH into a localparam, and the pass matched `[lo:hi]`

groups mentioning a parameter -- which is also how a parameter's `from [lo:hi]` VALUE constraint is spelled. `bb = 4 from [aa:8]` therefore froze `aa`, so `.model .. (aa=5)` was accepted and DID NOTHING; and the freeze then rewrote text the range fold had already claimed, so two overlapping rewrites panicked the compiler whenever a range mentioned its own parameter. Constraint brackets are skipped; overlapping rewrites are dropped, not fatal. Also: any `aliasparam` cycle was a CRASH DUMP with no diagnostic (`resolve(db).unwrap()` on the resolver's cycle recovery) and is now an error naming the chain; a self-referential `parameter/localparam` (`ls = ls + 1` gave 2, `l2 = l2*3+7` gave 28) is reported instead of folded twice; a `noise_table` DATA FILE is validated like a `\$table_model` one (an unusable file gave an empty table, and an empty noise table contributes NOTHING -- the spectrum came out identical to a model with no noise source); a negative literal noise power is refused instead of silently used as its positive twin; `branch (a,a)` warns (new L024) instead of silently discarding every flow contributed to it; the too-many-arguments message quotes the MAXIMUM (`absdelay` said "at most 2" while three are legal); a diagnostic landing in a synthesised elaboration buffer says so; and `--version` falls back to the crate version instead of printing "unknown". Three findings deliberately NOT changed, with evidence in the write-up: a parameter's DEFAULT stays exempt from range checking (E-56, the CMC "feature disabled" idiom -- 3 of 1551 ranged parameters in 700 corpus models rely on it), `transition`/`slew` do not abort a transient (the report came from a harness `reltol=1e-11`; both are correct at default and 1e-6 tolerances), and an out-of-range integer literal stays a real constant (what makes a large `laplace_nd` coefficient work).

E-415 *openvaf/hir_lower/src/stmt.rs, examples/evtnoise_examples/ (new)*

A **`@(timer)`** event the solver stepped straight over -- 891 of 1000 lost. `cross`/`above` watch a SIGNAL, so a sign change across an accepted interval is still noticed and the event is only reported late; a timer watches nothing, so an instant the solver never stops near simply does not happen. A 10 ns timer over a 10 us run fired 109 times out of 1000 at a 1 us step (207 at 0.1 us), and a model implementing a clock or a sampled system ran at whatever rate the step controller picked. The tell was a contrast: for ngspice's OWN pulse edge at 1 us the nearest timepoint is EXACTLY 0 away, while for an OSDI timer at 1 us it was 5.6e-08 away and the grid ran 8.56e-07 -> 1.056e-06 with nothing at 1 us -- the breakpoint machinery works, OSDI device events never reached it. Fixed with NO ABI change: `lower_timer` already computes the next event time and Enhancement-24's `\$bound_step` channel already runs to `osditrunc.c`, so the model now requests at most `next_event - now` and the following timepoint lands on the event. Combined with min, never an overwrite, so a model's own `\$bound_step` still holds beside a timer (1000 events AND the 5 ns bound) and a one-shot whose pending time is INFINITY does not pin the step for the rest of the run (59 timepoints, unchanged).

E-420 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_lower/src/expr.rs, examples/vafdegen_examples/ (new)*

Six findings from a round-26 hunt, not one of them a crash: every one was ACCEPTED BY THE COMPILER AND THEN SILENTLY DEGENERATE. (1) THE SUBSTANTIVE ONE -- **`2 ** -1`** with two integer operands returned 1; IEEE 1364-2005 Table 5-6 says 0. `Power` was the only arithmetic operator pinned to `REAL_BIN_OP`, so both operands were promoted to float, `llvm.pow.f64` ran, and the real 0.5 was rounded back AWAY FROM ZERO on the way into an integer. §5.1.5 fixes the type too -- the result is real only if either operand is real -- so the fix is BOTH HALVES: `Power` takes the two-candidate `NUMERIC_BIN_OP`, and `lower_bin_op` grows an `INT_OP` arm implementing Table 5-6. Either alone is worse than neither: a signature change without the lowering leaves the same float `pow` behind an integer type, the exact wrong-code trap E-376 recorded for `$dist_*`. `lower_int_pow` is branchless (every operand is a comparison or a multiply by 0/1, so the folder collapses it for literals) and CLAMPS THE EXPONENT THE FLOAT PATH SEES to non-negative -- not tidiness: `pow(0, -1)` is infinity and `llvm.lround` of an infinity into an i32 is undefined. That clamp fixed a SEVENTH defect the hunt never reported, found only by the differential: `0 ** -1` returned 2147483647, exactly `lround(inf)` saturating; Table 5-6 calls it `'x`,

which is 0 as an integer. The real path does not move (`2.0 ** -1` is still 0.5, and so are `2.0 ** -1.0` and `2 ** -1.0`). (2) A `laplace_*` denominator that is IDENTICALLY ZERO -- `laplace_nd(V(a,b), '{1.0}, '{0.0})` is the transfer function 1/0; it compiled clean and then killed the operating point with "Transient op failed, timestep too small", which names neither the model nor the call, so the author debugs the netlist. Rejected only when EVERY coefficient folds to a literal zero and only for a COEFFICIENT list: `'{0.0, 1.0}` is the denominator `s` (a pure integrator) and an all-zero ROOT list (`laplace_np/laplace_zp/zi_np/zi_zp`) is poles at the origin -- both legitimate, both still compile, as does the concatenation form E-399 deliberately allows here. (3) A `zi_*` SAMPLING PERIOD of zero -- `zi_nd(V(a,b), '{1.0}, '{1.0}, 0.0, 0.0)` compiled, ran, and returned the INPUT UNCHANGED (y = 1.0 for a unit input): a filter that is not a filter. (4) A `last_crossing` DIRECTION outside the LRM's {-1, 0, +1} -- `last_crossing(V(a,b) - 0.5, 7)` was accepted and behaved as 0, returning the same 5.0e-07 the `either` form gives. (5) A `$discontinuity` DEGREE BELOW -1, silently handled as an ordinary non-negative degree. THE FIRST VERSION OF THIS CHECK WAS **`require_non_negative`** AND IT WAS WRONG: `$discontinuity(-1)` is the LRM's marker for a LIMITING discontinuity, written inside a `$limit` function, it appears verbatim in the LRM's own page-261 `spicepnjlim` diode, and `lower_builtin` already routes it to `LimDiscontinuity` -- it is implemented, not tolerated. `examples/lrm_examples` compiles that page and failed on the first regression sweep; the check that ships rejects only what is BELOW -1. The regression, not the reasoning, drew that line. (6) `ac_stim` naming an analysis that can NEVER match got no diagnostic at all -- the project's most repeated shape, handled for one construct and silently not for its sibling: `analysis("nosuch")` has warned since E-399 (L021) and `$limit(.., "nosuchlimit", ..)` since E-396 (L020). ngspice gates the stimulus on `strcmp(src->analysis, "ac")` in `osdiacld.c`, so an unmatchable name leaves the source PERMANENTLY INACTIVE -- an AC source that never sources anything. `ac_stim` now runs the same `check_analysis_name` helper, with a note specific to it (inactive source, not the dead branch `analysis()` produces); a warning rather than an error for the same reason L020 and L021 are, since the name set is simulator-defined. A SECOND DELIBERATE NARROWING: `ac_stim("tran")` is NOT warned. ngspice's gate is `"ac"` alone so it is inactive there too, but the LRM lets a stimulus name any analysis and another OSDI consumer may honour more -- the claim the evidence supports is "can never match anywhere", so that is the only claim the check makes; recorded as an open item rather than a silent one. All five validation checks fold LITERALS ONLY (`const_num`), matching every neighbouring check -- a runtime-valued argument is the model's own business, and each has a runtime case in the accept half. VERIFICATION: `examples/vafdegen_examples` 91/91, roughly half of it the ACCEPT half; Table 5-6 is MEASURED as opvars from a real ngspice operating point -- 20 integer cases, 5 real ones, and 8 more with base and exponent supplied as RUNTIME PARAMETERS so the lowering is exercised and not just the folder; the `ac_stim` consequence is measured too (a one-terminal `V(out) <+ ac_stim(name,1,0)` into a one-point `.ac` reads mag 1.0 for `"ac"` and EXACTLY 0.0 for `"nosuch"`) rather than taken from the diagnostic. 40-model `integration_tests/` corpus compiled with the previous shipped binary and this one: 0 changed diagnostics (the corpus contains no ** operator at all, so it bounds the validation changes, not the type change -- `vafdegen` and `lrm` bound that). `cargo test --features llvm18` clean, no MIR or OSDI snapshot moved; full regression 337/337. Every finding was reproduced on the SHIPPED binary first: all five validation cases compile silently there and `2 ** -1` returns 1.

E-421 *openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/item_tree/diagnostics.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/diagnostics.rs, openvaf/syntax/src/validation.rs, openvaf/syntax/src/error.rs, openvaf/basedb/src/lints.rs, openvaf/basedb/src/diagnostics/syntax_error.rs, examples/rangeguard_examples/ (new)*

Five findings from a round-27 hunt; THREE ARE THE SAME SHAPE -- a check that exists, is correct, and was never applied to the sibling spelling. (1) An `exclude` that covers the whole `from` range made the parameter UNSETTABLE, in silence: `from [0:10] exclude [0:10]` compiled

clean, then every netlist value aborted with "Parameter x is out of bounds!" while only the default read (E-56 exempts defaults by design). That is exactly the end state E-399 already reports for **`from [3:1]`** -- it checked `from` and never looked at `exclude`, which is the round-12 lesson ("enumerate EVERY supply path") applied to every way of spelling an empty range. Reached by ordinary routes, not just exact cover: an exclude WIDER than the range it guards (`from [1:2] exclude [0:10]`, a copy-paste) and two excludes that tile it. Decided by a small interval sweep over literal bounds, and endpoint INCLUSIVITY is the whole difficulty: `from [0:10] exclude (0:10)` leaves exactly `0` and `10` settable and must still compile, `exclude [0:10)` leaves exactly `10`, `exclude [0:5] exclude [6:10]` leaves the gap -- so the sweep tracks POSITIONS, not values, because "just past an open endpoint" is not a float. (2) An INVERTED `exclude [3:1]` excluded NOTHING and said nothing -- the author wrote "keep 1 through 3 out" with the bounds reversed and every value in that band was accepted; `from [3:1]` with those same bounds has been an error since E-399. The more insidious of the two: (1) fails loudly at run time, (2) never fails at all -- it silently permits exactly what the model meant to forbid. `exclude [1:1]`, both bounds closed, is a legitimate point exclusion and still compiles. (3) **`$simparam`/`$simparam$str`** names were never checked, and they are the only sibling whose bad name is FATAL -- new `unknown_simparam` lint (L025). The severity ordering was exactly inverted: `analysis("nosuch")` warns (L021, E-399) and the branch is merely dead, `$limit` warns (L020, E-396) and the load is merely refused, `ac_stim` warns (L021, E-420) and the source is merely inactive -- while `$simparam` said nothing and `EVAL_RET_FLAG_FATAL` kills the analysis. THE NAME LIST IS NGSPICE'S, NOT THE LRM'S, AND THAT IS THE ENTIRE POINT: ngspice serves 14 numeric and 2 string names (`src/osdi/osdiload.c`), the LRM names `minr/imelt/shrink/imax/rthresh` which ngspice serves NONE of, and for `$simparam$str` the two sets do not intersect at all (LRM: `cwd/module/instance/path`; ngspice: `analysis_name/simulator`) -- so an LRM-derived check would have warned on precisely the names that work and stayed silent on the ones that abort. The non-fatal `$simparam(name, default)` form is deliberately NOT warned (it is how a model stays portable) and the diagnostic points at it by name; `$simparam$str` has no such form, so every unresolvable name there is reported. A MISREADING CORRECTED IN FLIGHT: the hunt first reported this as "the compiler blesses names the simulator kills", on a `known` list in validation/body.rs -- that list means "does not vary between iterations" (it drives L015 in a CONST context only) and claims nothing about existence; the finding is stated without it and the runtime cross-table carries it alone. (4) More than one `default` arm in a case statement was accepted (IEEE 1364-2005 9.5 makes it illegal). The behaviour was never wrong -- the first runs -- which is why it is worth saying: the later arm is unreachable code that looks like it does something. Checked in syntax validation where the `default` TOKENS are, so the report points at the offending arm and back at the winner; a duplicate case ITEM is legal in Verilog and stays accepted. (5) Two garbled diagnostic strings: L015's help said "paramaeters" and closed a double quote with an apostrophe; an argument-type error read "typed mismatch invalid function arguments". SCOPE, stated rather than discovered later: `inf` does not fold so `from [0:inf)` is untouched exactly as E-399 leaves it; one unfoldable bound anywhere makes the cover question unanswerable and nothing is said; several `from` clauses are a UNION and are skipped; and the sweep is real-valued, so on an INTEGER parameter `from [0:2] exclude 0 exclude 1 exclude 2` genuinely is a full cover and is NOT reported -- under-reporting is the safe direction, the evidence was real-valued, and the example suite pins the gap deliberately so it is known rather than accidental. VERIFICATION: `examples/rangeguard_examples` 72/72, about half of it the accept half, with the inclusivity boundaries pinned as BEHAVIOUR and not merely as silence -- `from [0:10] exclude (0:10)` is compiled and driven through ngspice to confirm `x=0` and `x=10` are accepted while `x=5` is rejected; the L025 note's claim that an unresolvable name is fatal rather than zero is measured by running one; and the lint is exercised AS a lint (`-A` silences it, `-E` makes it an error). 124-model VA_TEST industry corpus differential: exactly ONE difference, and it is the typo fix -- 34 of those models use `exclude`, not one changed verdict, and no corpus `$simparam` name trips L025. That is the differential that mattered, since compact models use ranges heavily and an over-eager cover check would have surfaced there. 40-model `integration_tests/` corpus 0 changed diagnostics; `cargo test --features llvm18` 210/210

with no snapshot moved; full regression 338/338.

E-422 *openvaf/hir_ty/src/validation/types.rs, openvaf/hir_ty/src/validation.rs, openvaf/osdi/src/ndatable.rs, examples/natureref_examples/ (new)*

One reference, three different outcomes. A `nature` makes up to three references to other natures (`parent`, `ddt_nature`, `idt_nature`) and a `discipline` two more (`potential`, `flow`) -- the same reference, resolved by the same function. A name that failed to resolve produced three different outcomes depending on which one you wrote: `parent` CRASHED THE COMPILER, `ddt_nature`/`idt_nature` were silently discarded, and `potential`/`flow` were reported much later against the MODEL BODY. (1) THE CRASH: `nature Vd : Vbaze;` -- one character wrong -- exited 101 with a crash report, `Option::unwrap()` on `None` at `osdi/src/ndatable.rs:79`. Nothing validated the name: `hir_ty::lower` resolves it with `lookup_nature(..).ok()`, which throws the error away, and OSDI codegen then unwrapped the missing name-map entry. Six spellings reached it -- a typo, an undeclared name, a DISCIPLINE name, an ACCESS FUNCTION name, the MODULE name, and a discipline-qualified parent whose discipline does not exist. AND IT DID NOT HAVE TO BE USED: a stray `nature Stray : nosuch;` that no discipline and no module mentions crashed the build all the same, so one bad declaration in an included header killed every model that included it. The tell was five lines below the panic: `resolve_nature_index`, added by E-39 for the `ddt_nature`/`idt_nature` references, was ALREADY written as `unwrap_or(u32::MAX)` -- one side hardened, the other left to crash. Fixed on both sides: the names are diagnosed in `hir_ty` so a bad reference never reaches codegen, and `resolve_nature_ref` returns `NATREF_NONE` instead of panicking if one ever does again. (2) NATURE INHERITANCE CYCLES were silent -- `nature A : A;` plus two- and three-cycles all compiled, because salsa recovers from the query cycle by re-resolving with the parent dropped, so the nature silently becomes its own base nature, inherits no units, and therefore changes which disciplines it is compatible with (compatibility falls back to same-base-nature when units are absent, per E-399). Parameter cycles, `aliasparam` cycles (E-414) and analog-function recursion are all rejected by name; nature cycles were the family member nobody checked. The walk uses `nature_data` + `lookup_nature` rather than `nature_info` precisely so it SEES the cycle instead of salsa's recovery, and reports only from the member the walk returns to, so an N-cycle gives N reports and not N^2. (3) `abstol` was unvalidated in TWO ways: as a VALUE (zero, negative, and a literal overflowing to infinity such as `1e400` all compiled and reached the OSDI nature descriptor) and as an EXPRESSION (`1.0/0.0`, `0.0/0.0`, `1e-6+0.0`, even `"abc"` did not fold, so the lowering silently discarded the attribute and the nature ended up with no abstol at all -- not what the declaration says). A nature with NO `abstol` attribute stays legal, as the LRM allows. (4) A discipline whose `potential`/`flow` named a missing nature said ”illegal access of branch '(p, p)'” -- naming neither the discipline nor the missing nature, and pointing at a line that is not wrong. SCOPE, recorded rather than hidden: the old body-level complaint still CASCADES; what changed is that the declaration error now comes FIRST. Suppressing the cascade means teaching the body walk that the discipline is known-broken, which is wider than the evidence asks for, so the suite pins the ORDERING. THE HARNESS LESSON, and the reason this took three rounds to surface: openvaf installs a panic HOOK -- a hard crash exits 101 with a polite banner containing NEITHER ”panicked” NOR a backtrace, so a detector keyed on signals or on the word ”panicked” scores it as an ordinary rejection. That is why the round-26 and round-27 hunts both reported ”no crashes found”; round 27's batches were re-run with the corrected detector and stayed clean, round 26 was not re-verified. The example suite therefore asserts the banner is gone and no crash log was written, not merely that rc != 0. VERIFICATION: `examples/natureref_examples` 51/51, about half the accept half -- correctly spelled parents, discipline-qualified references that resolve, three- and four-deep acyclic chains, real `ddt_nature`/`idt_nature`, four legal abstol values, a nature with no abstol, and a model on CUSTOM natures that still simulates (-1 mA, measured). The standard library is pinned explicitly (a stock `electrical` model, an `electrical` + `thermal` model, and a user nature derived from the stock `Voltage`), which is the check that matters since `disciplines.vams` is a dense set of parent/`ddt_nature`/`idt_nature` references and an over-eager check would break every model in

existence. 124-model VA_TEST industry corpus and 40-model **integration_tests** corpus: the only difference is E-421's typo fix, which the shipped binary predates -- not one nature or discipline diagnostic fires across 164 real models. `cargo test --features llvm18` 210/210 with no snapshot moved; full regression 339/339.

E-423 *openvaf/parser/src/error.rs, openvaf/parser/src/grammar/expressions.rs, openvaf/parser/src/grammar/call.rs, openvaf/syntax/src/error.rs, openvaf/syntax/src/parsing/tree_builder.rs, openvaf/basedb/src/diagnostics/syntax_error.rs, examples/commaexpr_examples/ (new)*

A parenthesised comma list was an expression, and only its FIRST element survived. `r = (1.0, 2.0);` compiled clean and gave 1.0; `(3.0, 2.0, 1.0)` gave 3.0. Accepted EVERYWHERE an expression is wanted -- expressions, `parameter`/`localparam` defaults, elements of an array literal, `from`-range bounds, contributions, access-function arguments, array indices (`a[(0,1)]` -> `a[0]`) and call arguments (`max((1,2),3)` -> 3). THE SHARPER HALF: the dropped elements were never checked at all, so the construct was a place errors went to HIDE -- `(1.0, nosuchname)`, `(1.0, nosuchfunc(3))`, `(1.0, "a string")`, `(1.0, max(1))` (wrong arity) and `(1.0, last_crossing(V,7))` (an argument E-420 rejects) ALL compiled clean, while `nosuchname` alone is rejected and `(nosuchname, 1.0)` is rejected -- proof that only the first element was ever analysed. ROOT CAUSE: `paren_expr` carried a TUPLE-PARSING LOOP over from rust-analyzer, with its original Rust test comment still sitting in it (`// test tuple_attrs` / `const A: (i64, i64) = (1, #[cfg(test)] 2);`). Verilog-A has no tuples; `hir_def/src/body/lower.rs` then did `ParenExpr(e) => collect_opt_expr(e.expr())`, taking the FIRST child, so later children never reached the HIR and nothing downstream could see them. E-387's comment sits directly above that loop and lists the malformed forms it enumerated while fixing the `()` crash (`{}`, `{1,}`, `a[]`, `? :`, `sqrt()`, `1+`) -- this one was missed because it is NOT MALFORMED to that loop: it parses perfectly and just means something else. The aggregate siblings `{1,2}` and `'{1,2}` were already TYPE-rejected and still are; the recurring handled-for-one-form-not-its-sibling shape. WHY IT MATTERS, measured not asserted: compact models are full of parenthesised sums split across lines, and a `,` where a `+` was meant -- `I(p,n) <+ (gm*V(p,n) , gds*V(p,n));` -- takes i(v1) from -5 mA to -1 mA, a 5x wrong answer from source that looks right; a deleted builtin name does the same (`pow(gm,2)` written `(gm,2)` gives 3 instead of 9). Both are checked in the suite as NUMBERS. THE FIX: `paren_expr` parses exactly one expression and reports a dedicated `CommaExpr` error on a following comma, then consumes the rest of the list so the caller resynchronises on `)` instead of cascading. Its own variant rather than `UnexpectedToken` for the reason E-387 gave `ExprTooDeep` one -- the source is not malformed, so an "expected '(', '{', identifier..." message would describe a problem the source does not have. ALSO: a TRAILING COMMA in a call argument list -- `max(1.0, 2.0,)` ended `arg_list`'s loop on the `)` and was accepted as two arguments, while `max(1.0, )` was caught only later by the arity check and described as a count problem rather than a stray comma; a comma must now be followed by an argument. VERIFICATION: `examples/commaexpr_examples` 41/41, about half the accept half and checking VALUES not just silence -- every ordinary parenthesised form (plain, sum, nested, two factors, unary minus, ternary condition, multi-line sum, two- and three-argument builtins, a parenthesised call argument, and an array literal which legitimately takes commas) is compiled and read back from a real ngspice operating point; and each of the five hidden errors is asserted to surface. 164 real models -- the 124-model VA_TEST industry corpus and the 40-model **integration_tests** corpus -- compiled with the previous shipped binary and this one: ZERO differences. That answers the question the hunt left open: no real model contains the shape, so none was silently wrong and rejecting it breaks nothing -- a textual grep could not have answered that, the differential could. `cargo test --features llvm18` 210/210 with no parser or MIR snapshot moved; full regression 340/340.

E-424 *openvaf/hir_def/src/builtin.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, examples/noiseloop_examples/ (new)*

A noise source inside a run-time loop contributed NOTHING, and registered no source at all. `for (k=0;k<1;k=k+1) I(p,n) <+ white_noise(4e-21);` gave an `onoise_total` bit-identical

to a model with no noise source in it -- and printing the per-source vectors in the `noise2` plot showed `onoise_total_n1_unnamed0` for the working spelling and NO **`n1`** source whatsoever for the loop one. That distinction is what turned ”the number looks wrong” into a diagnosis. Affected `white_noise`, `flicker_noise`, `noise_table` and `noise_table_log`, in `for`, `while` and `repeat` alike, plus **`ac_stim`**, which rides the same small-signal pipeline (E-51) and went from mag 500 to 0. Silent even under `-E all`. THE SIBLINGS DECIDED THE FIX, not reasoning: `ddt`, `idt`, `absdelay`, `transition` and `laplace_*` were ALREADY rejected in exactly this position (”analog operator 'ddt' is not allowed in loops”, LRM 4.5.1) -- the noise builtins are simply not in `is_analog_operator()`, so they fell past that arm and were discarded further down instead of reported. The restriction is LOOP-ONLY and deliberately so: a noise source inside an `if`/`else`/`case` is legitimate (gating noise on a mode flag is ordinary compact-model practice) and works correctly, so this uses its own `is_small_signal_source()` predicate rather than being folded into `is_analog_operator()`, which also drives the conditional and main-block checks -- folding it in would have broken working models. **`generate`** keeps working and is the right answer for per-finger noise: a genvar loop unrolls at elaboration and creates one source per iteration, checked as a NUMBER -- two iterations give two sources and, after subtracting the resistor floor's power, exactly 2.000000000000x the noise power of one. ONE BEHAVIOUR CHANGE, NAMED: assigning a noise source to a variable INSIDE a loop and contributing outside it used to WORK and is rejected now. Not an over-reach, and the suite asserts why -- `t = ddt(..)`, `t = idt(..)` and `t = laplace_nd(..)` in exactly that position are ALREADY rejected, so the restriction has always been on the CALL SITE being inside a loop, not on how the result is used; leaving noise alone would have kept one family member behaving differently from all the others, which is how the defect arose. Also: `$finish(n)`/`$stop(n)` accepted any n where IEEE 1364-2005 17.1.2 defines exactly three (0 prints nothing, 1 the time and location, 2 adds statistics) -- same shape as the `last_crossing` direction E-420 rejected, literals only so a runtime argument is left alone. VERIFICATION: `examples/noiseloop_examples` 36/36, with the accept half measured as NUMBERS rather than silence (no-noise model registers no `n1` source and sits at the resistor floor; a plain source registers one and rises above it; the conditional spelling gives a total IDENTICAL to the plain one; genvar registers two at exactly twice the power). 164 real models -- 124-model VA_TEST industry corpus + 40-model **`integration_tests`** -- compiled with the previous shipped binary and this one: ZERO differences, so no industry model was silently losing noise and rejecting it breaks nothing. `cargo test --features llvm18` 210/210 with no snapshot moved; full regression 341/341. HARNESS, recorded because it cost two false readings: a single-frequency `noise` run warns ”Noise measurement at a single frequency” and produces no total at all, which reads exactly like ”the source contributes nothing”; and the output node must not be the source node.

E-427 *openvaf/hir_lower/src/stmt.rs*

A **`@(timer)`** event landing exactly on the transient's **`tstop`** never fired. `dt = 1e-8` over `tstop = 1e-6` gave 100 ticks where 101 were due, independently of the transient step -- and `@(final_step)` fires at that same instant, so the timepoint IS reached and only this comparison rejected it. `lower_timer` fires on `abstime >= next`, and `next` is built by repeated addition (one `fadd` per fire), so after N periods it carries N roundings and sits a couple of ULP away from the exact `N*period`. When `tstop` is an exact multiple of the period -- the ordinary case -- the schedule lands just PAST it. Reproducing the accumulation in isolation predicts all eight measured cases exactly: dt = 1e-8, 2e-8, 3e-8 and 4e-8 accumulate +3.2e-22 to +6.4e-22 OVER and lose their last event, while 5e-9 (-4.4e-21), 1e-7 (exactly 0) and 1e-9 (-1.1e-22) land at or below and were always correct -- which is why it looked sporadic. The comparison now carries a 1e-12 relative tolerance: four orders of magnitude above the observed drift and far below any physical timescale (1 ps early on a 1 s period). Written as a MULTIPLY, `next * (1 - 1e-12)`, rather than `next - eps`, so a one-shot timer that has already fired -- `next` is INFINITY -- stays INFINITY instead of becoming INF-INF = NaN. Residual of E-415, which fixed the 891-of-1000 lost-event case via `\$bound_step`. Costs nothing in stale artifacts: no **`.osdi`** is committed anywhere in the repo, every suite builds its own from **`.va`** at verify time. `cargo test --features llvm18` 210/210, no snapshot moved.

E-425 *openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/syntax/src/validation.rs, openvaf/syntax/src/error.rs, openvaf/basedb/src/diagnostics/syntax_error.rs, examples/tabledata_examples/ (new)*

Three things the compiler accepted that were not numbers. (1) A CORRUPT ROW IN A DATA FILE WAS SILENTLY DROPPED: a table `(0,0) (1,100) (2,20)` queried at x=0.5 gives 50, and replacing the middle row with `N/A N/A` / `abc def` / `--- ---` compiles clean and gives 5 -- a tenfold wrong answer. Both `table_file_is_usable` and the `hir_lower` readers did 'filter_map(

E-453 *openvaf/openvaf/src/cache.rs, openvaf/openvaf/src/lib.rs, openvaf/mir_llvm/src/lib.rs, openvaf/osdi/src/lib.rs, openvaf/parser/src/grammar/call.rs, openvaf/hir_ty/src/inference.rs, openvaf/hir_lower/src/fmt.rs, examples/batchkey_examples/ (new), examples/nullarg_examples/ (new)*

A STALE CACHE ARTIFACT, AN IMPOSSIBLE TARGET, AND TWO CRASHES. Five defects of one shape: a request answered with something other than what was asked for. (1) BATCH MODE keyed its cache on the source, defines, lints and compiler version but NOT on the settings that decide the machine code, so `openvaf -r m.va -b -O 0` followed by `-O 3` produced ONE entry -- the second run exited 0 and handed back the `-O 0` build, 113424 bytes where a real `-O 3` build is 36936. Debug once and every later optimized build is silently the debug one. (2) `--target` was answered from the HOST cache: `--target x86_64-unknown-linux` on an arm64 mac exited 0 with an arm64 Mach-O -- a Linux build that is a macOS build, reported as success. `--target-cpu` and `-C` are the same class of input (native vs generic produce different binaries) and are hashed too. (3) CROSS-COMPILING then PANICKED, because `initialize_llvm` registers only the NATIVE LLVM target and the failure was reached through `back.new_module(..).unwrap()` on a rayon worker: exit 101, a crash banner and a request to file a bug at osdi/src/lib.rs:197. These two are one story -- putting the target in the key turns the wrong-artifact answer into a real compile, which must then fail HONESTLY rather than abort, so fixing only the key would have converted a silently wrong answer into a crash. The target is now probed once before any work is spawned, with the same triple, cpu and features codegen would use, and the help line is built by PROBING each advertised target, so it reports what the binary can really emit (three of the eight `--supported-targets` here). (4) The LRM's NULL ARGUMENT was rejected: LRM 4.5.11 and 4.5.12 both say "the zeros argument may be represented as a null argument ... two adjacent commas („)", and the LRM's own worked example `laplace_zp(white_noise(k), , '{1,0,1,0,-1,0,-1,0})` did not compile ("unexpected token ','" plus a bogus arity from the failed parse), for all eight filters. The capability was never missing -- `'{}` says "no zeros" and is numerically exact -- only the LRM's spelling of it. An empty slot now parses as the same empty-vector node `'{}` produces, verified NUMERICALLY identical and not merely by exit code. The parser sees token KINDS only, never text, so it cannot know the callee; legality is left to inference, and only an INTERIOR or LEADING slot is a null argument -- the E-423 trailing comma is still an error. (5) An argument no `%` conversion consumes is rendered from its own type, only real/integer/string had a case, and the fallback was `unreachable!()` -- so `$strobe("x", 1 > 0)` (a Bool) and an array literal exited 101 with a crash report. A NAMED array was already caught cleanly; these were not, and `$strobe("flag", v > 0.5)` is an ordinary thing to write. `$strobe("%d", 1 > 0)` always worked because that path casts to an integer, so a Bool IS printable -- it now gets that cast on the default path and prints `1`/`0`, while an array is reported. Verified batchkey 17/17 vs 9/17 and nullarg 37/37 vs 13/27 on the previous compiler, the failures there being exactly the defect checks while every control passes on both. The parser change touches EVERY call expression and the inference change every display-family call, so the whole 124-model corpus was compiled with both drivers to the same output path (an `.osdi` embeds its own path, E-389): 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences. `cargo test` zero failures; regression 367/367 both solvers.

E-455 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/item_tree/diagnostics.rs, examples/valguard_examples/ (new)*

A GUARD, AND THE SPELLING NEXT TO IT. Seven defects sharing one shape: a check that exists, and a neighbouring spelling of the same mistake that walks past it. (1) INDEXING A SCALAR CRASHED THE COMPILER -- `r[0]` on a real, `i[0]` on an integer, `s[0]` on a string, `p[0]` on a scalar parameter: `infere_bit_select` resolved the base, found it was not a bus or array and returned `None` WITHOUT PUSHING A DIAGNOSTIC, so the type became `Err` and hir/src/body.rs: 381 hit `panic!("invalid HIR: path .. was not resolved")` -- exit 101, a crash banner and a request to open a GitHub issue, for an ordinary typo. Every scalar kind times every index form (`[0]`, `[i]`, `[0][0]`) did it, in expression and assignment-target position alike. (2) THE VALUE GUARDS WERE LITERAL-ONLY: `const_num`, which every one of them is built on, folded a literal and a unary minus and nothing else, so `white_noise(-1e-18)` was refused while `white_noise(0-1e-18)` was ACCEPTED and produced exactly the output noise of the positive power, silently -- and likewise `$bound_step(1-1)`, `transition(x,0,0-1n,1n)`, `slew(x,0-1e6,-1e6)`, `idtmod(x,0,2-2)` and `@(cross(e,3+4))`. It was not even consistent inside the compiler: the array-bounds and integer-division checks fold first and DO catch `arr[2+3]` and `1/(1-1)`. `const_num` now folds constant `+ - * /`; a PARAMETER is deliberately still not folded, since its default may be overridden and refusing a model for a default that will never be used would be wrong -- the reasoning that keeps a default out of its own range check, pinned by a control. (3) REVERSED RANGES SPELLED WITH `inf` WERE INVISIBLE: `from (5:1)` is refused but `from (inf:0)` -- one transposition from `from (0:inf)` -- was accepted and then enforced NOTHING, with -5, 0 and 5 all passing a range that admits no value. `inf` is a LITERAL TOKEN the bound folder had no case for, so it gave up on any range mentioning it; folding it is strictly better rather than riskier, since `from (0:inf)` has lo < hi and stays accepted. (4) A CONSTANT OUTSIDE A FUNCTION'S DOMAIN said nothing: `sqrt(-1.0)`, `ln(-1.0)`, `asin(2.0)` folded to NaN with zero warnings and the model then failed at simulation with "Transient op failed, timestep too small" -- a convergence message for a NaN written in the source -- while integer `1/0` and `5 % 0` have always been compile errors. A RUN-TIME argument is left alone. (5,6,7) Three declarations the LRM forbids were accepted: `$rdist_uniform` with reversed bounds returned what the correct ordering returns (LRM 9.13.2 "the start value shall be smaller than the end value"), an analog function with NO arguments compiled (LRM 4.7.1 "shall have at least one formal argument declared"), and a discipline naming ONE nature for both `potential` and `flow` compiled and produced a device that contributed nothing where its well-formed twin was right, and a node voltage of -999 when a contribution was forced (the two `NatureRef`s carry different `kind`s so they never compare equal as a whole -- it is the NAME that has to match). THREE FINDINGS WERE WITHDRAWN by re-verifying each against full compiler output before writing any code: unknown `analysis( )/initial_step` names are already `warning[L021]` (E-399), an unrecognised `(* type= *)` is already warned, and a real literal underflowing to zero is a decision stated in the source ("both of which IEEE 754 defines"). Since this adds new compile-time REJECTIONS the decisive check is that it rejects nothing real: corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences. Verified valguard 113/113 both solvers vs 82/113 on the previous compiler; `cargo test` zero failures.

E-456 *openvaf/hir_lower/src/lib.rs, openvaf/test_data/mir/analog_initial.mir, examples/analoginit_examples/ (new)*

**`analog initial`** RAN ON EVERY EVALUATION, NOT ONCE. LRM 5.2: "The analog initial block is executed once for each analysis." Its statements were lowered straight into the front of the eval function, CONCATENATED with the main analog block, so they re-ran on every evaluation and overwrote whatever the model had accumulated between timesteps -- destroying the one thing the construct exists for. A peak detector initialised with `analog initial begin peak = 0.0; end` did not hold its peak, it FOLLOWED THE INPUT BACK DOWN: on a 0->1->0 ramp it read 1.0 at 5us and then 0.4 at 8us and 0.1 at 9.5us, where the identical model with the initialisation removed correctly held 1.0. A counter never left zero (with the block) versus 1,3,6,9 (without). Nothing was reported either way, so the construct designed for one-time initialisation was the only thing that broke state -- and using it looked like the careful thing to do. Gated now on `ParamKind:`

`:IsInitialStep`, the flag `@(initial_step)` already uses: not a guess, since the SAME models written as `@(initial_step) peak = 0.0;` inside the main block always worked, giving a working reference for the target semantics inside the compiler, and the fixed models reproduce its numbers exactly. Measured, that flag fires ONCE PER ANALYSIS PER INSTANCE -- once for an op, once for a whole tran, once for an entire dc sweep however many points it has, twice for `op` then `dc`, once per instance -- which is the LRM's baseline rule (its follow-on clause about parameter-sweep sub-tasks is permissive, "*can* be executed", and its mandatory half applies only when a parameter the block references changes during the sweep). Statements still lower in source order and still run before the main block, so multiple initial blocks compose exactly as before. The MIR records the whole change: `br <is_initial_step>` into the initial statements or an empty else, then a phi merging the initialised value with the retained one -- initialise once, carry state afterwards. The gate is emitted ONLY when a module actually has an initial block; without that guard every model picks up an `IsInitialStep` parameter and an empty conditional it never asked for, which showed up at once as a 32-byte change in a MEXTRAM model that has no initial block at all. Verified analoginit 10/10 both solvers vs 6/10 on the previous compiler, the suite built around `_ref` twins so each fixed model must MATCH ITS REFERENCE AT EVERY PROBE rather than merely "hold a peak now"; corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences (against the pre-E-455 binary, so covering both enhancements); `cargo test` 47 binaries with the `mir/analog_initial.va` snapshot INTENTIONALLY updated -- it had captured the defect, the initial statements computed unconditionally in block0; regression 370/370.

E-457 *openvaf/parser/src/grammar/expressions.rs, openvaf/hir_def/src/item_tree.rs, openvaf/hir_def/src/item_tree/lower.rs, openvaf/hir_def/src/body.rs, openvaf/hir_def/src/body/lower.rs, examples/patternrep_examples/ (new)*

REPLICATION INSIDE AN ASSIGNMENT PATTERN DID NOT COMPILE. `'{4{0}}` failed at PARSE time with "unexpected token {; expected `,`", and so did the LRM's OWN initializer examples -- `real distort[0:2][0:2] = '{ 3{ '{3{0.0}}}};` and its string twin, whose comment reads "all elements are initialized to 0.0 using an assignment pattern and replication operator". Also `'{1.0, 2{3.0}, 4.0}` and the same construct in a Laplace coefficient vector. The only way to write an initialised array was to spell every element out -- nine copies of one number for the LRM's own 3x3. TWO CONSTRUCTS ONE APOSTROPHE APART: the replication that always worked, `{4{0}}`, is the CONCATENATION operator (E-34); `'{4{0}}` is an ASSIGNMENT PATTERN (LRM 4.2.14), a different construct that merely looks like it -- and LRM 4.2.13 flags the trap itself, noting `{ }` means concatenation in Verilog where C uses braces for array initialisers. `array_expr` read a plain comma-separated list, took the count as an element, expected `,` or `}` and met `{`. The parser now builds the SAME NODE SHAPE `concat_expr` produces for `{n{...}}` ( children `[count, elem0, ...]`), so `ReplicationExpr::count()/elems()` read it unchanged. Expansion had to happen in ONE place rather than three: a `'{...}` literal is walked by the leaf COUNT checked against the declared size and by the two per-element extractors (array variables, array parameters), and each counted a replication as a single leaf -- they now share one walker, so the count and the elements cannot disagree, since a pattern expanding to the right LENGTH but the wrong CONTENTS would satisfy the size check and still be wrong. A count that will not fold, is negative, or exceeds a 2^20 cap leaves the element unexpanded, so it reads as one leaf and the ordinary length-mismatch diagnostic reports it rather than a silently mis-sized array (the cap mirrors E-34's: a replication count is a size read from source). Verified patternrep 25/25 both solvers vs 10/20 on the previous compiler, the suite checking VALUES not merely acceptance -- every array carries distinct checkable numbers, which caught a real bug during development where a mixed pattern had the right length and the wrong middle elements. A trap worth recording: the value check first read 1.0 instead of 6.0 and the compiler was innocent -- the model had named an opvar `m`, and every ngspice instance already has an `m` MULTIPLIER parameter, so `@n1[m]` read the multiplier. Corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences (against the pre-E-455 binary, covering E-455/456/457 together); `cargo test` clean; regression 371/371. `lrm_examples` unaffected -- the LRM example holding these initializers is pinned on a

DIFFERENT diagnostic (`refers to module 'gen'`), which had masked this gap all along.

E-458 *openvaf/sourcegen/src/mir_instructions.rs, openvaf/mir/src/instructions/generated.rs, openvaf/mir/src/builder/generated.rs, openvaf/mir_llvm/src/builder.rs, openvaf/mir_opt/src/const_eval.rs, openvaf/mir_opt/src/simplify.rs, openvaf/mir_autodiff/src/builder.rs, openvaf/mir_interpret/src/lib.rs, openvaf/syntax/src/name.rs, openvaf/hir_def/src/builtin.rs, openvaf/hir_ty/src/builtin.rs, openvaf/hir_ty/src/builtin/generated.rs, openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_lower/src/expr.rs, openvaf/hir/src/body.rs, openvaf/parser/src/grammar/call.rs, examples/lrmfuncs_examples/ (new)*

EVERY LRM FUNCTION CHECKED IN EVERY ARGUMENT FORM THE SPEC WRITES FOR IT -- 117 builtins, 223 compiled-AND-RUN checks against numeric oracles, 17 failures, seven defects fixed. (1) **`ln1p`** and **`expm1`** DID NOT EXIST in either spelling though A.8.2 lists them beside `ln`/`exp`. They are their own MIR opcodes on libm `log1p`/`expm1`, NOT `ln(1+x)`/`exp(x)−1`, because precision near zero is their entire purpose: at x=1e-15 `ln1p` gives 9.999999999999995e-16 against an exact 9.9999999999999949e-16 while `ln(1+x)` gives 1.110223024625156e-15 -- wrong in the FIRST significant digit. A new opcode must be taught FIVE places and the compiler names only four: `simplify_unary_op` ends in `unreachable!()`, so the fifth is not a build error but a PANIC the first time a model uses the function. (2) Adding them as KEYWORDS broke eight shipping industry models -- HiSIM-SOI and HiSIM-SOTB each declare their own `analog function real expm1`, caught by the corpus differential as 8 models that had compiled for years and suddenly did not; they are now builtin names outside the reserved list, which a module can shadow (pinned: a user `expm1(0.5)` returns 50.0 while the builtin returns 0.4054651081081644 elsewhere). (3) **`$abs`**, **`$min`**, **`$max`** were the only three math functions whose **`$`**-spelling was never registered while 23 others worked -- and Table 4-14 explicitly encourages that spelling. (4) An array PARAMETER -- the form Syntax 4-3 lists FIRST -- was rejected by every Laplace and Z-transform filter with ”requires a bit-select [i]”; array VARIABLES worked for `laplace_*` but not `zi_*`, so the supported and unsupported cases were INVERTED relative to the spec. Parameter arrays now lower to parameter reads: literal, parameter array and variable array all give 4.00674 for H(s)=1/(1+1e-6s), and a `.model` override of `de[1]` moves it to 1.06531, proving the values are read at run time and not folded. (5) A TRAILING null filter argument (`laplace_np(x, n, )`) was a syntax error while the interior null worked; the trailing slot is now the same empty node, with E-423's trailing-comma typo still refused as a type error. (6) **`$simparam`** demanded a string LITERAL where LRM 9-10 allows ”a string literal, string parameter, or a string variable”. (7) **`$fatal;`** was refused though Syntax 9-7 makes the whole paren group optional and `$error`/`$warning`/`$info` already accepted it. (8) A string PARAMETER as a **`noise_table`** file name PANICKED the compiler (exit 101, no diagnostic) on an `as_literal().unwrap()`, while the sibling `$table_model` rejected the same model cleanly. ONE FINDING WITHDRAWN as the audit's own misreading: `$limit`'s user-function arity rule is CORRECT -- LRM 9.17 passes the function the current value, the internal state, THEN `$limit`'s extra arguments, so its arity is always 2+extra, and the `1.0` that looked like a phantom argument is the simulator declining to limit, which 9.17 permits. An array identifier as a **`noise_table`** stays REFUSED: LRM 4.5.1 allows it, but this table is materialised at COMPILE time and a run-time array would hand the builtin an EMPTY table -- E-399's silent no-noise failure -- so it is refused with that reason spelled out instead of ”requires a bit-select”; making the good message reachable required inference to ACCEPT the call and record its signature, since body validation only runs on a body that type-checked. Known remaining gap, pinned as refused: `parameter_identifier[msb:lsb]`, which needs range-select syntax rather than an inference change. Verified lrmfuncs 228/228 both solvers; corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 1 byte difference -- `bjt504.va` loses an internal unknown (`implicit_equation_1`) and a 56-point DC sweep matches the old build to 0.000e+00 in both terminal currents; `cargo test` 44 binaries; regression 372/372.

E-459 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/validation/body.rs, examples/filterslice_examples/ (new), examples/lrmfuncs_examples/verify_lrmfuncs.py*

A PART SELECT AS A FILTER COEFFICIENT VECTOR -- `de[0:1]`, the SECOND of the three forms LRM Syntax 4-3 defines for `analog_filter_function_arg` -- was rejected with "wrong number of array indices". E-458 implemented the other two; this is the form a model reaches for when it keeps ONE coefficient table and hands each filter the slice it needs, the alternative being a separate array parameter per filter. It looked like it needed new syntax: a part select of a 1-D array and E-15's multi-dimensional `m[i][j]` both arrive at inference as an `Expr::BitSelect` carrying TWO index expressions, so read there they are the same thing -- which is exactly what the diagnostic said. They are distinguishable ONE LAYER UP, and the machinery was already in place: the parser keeps the `:` token in the tree (E-85) and body lowering records every part select in `stray_part_selects`. Inference resolves the argument into its element slice and records it in the same whole-array maps a bare array identifier uses, so lowering carries it unchanged -- parameter elements to parameter reads, variable elements to variable reads. Consuming the expression is also what makes it LEGAL: E-85 reports every part select left in a body, since they are otherwise valid only in instance port connections which elaboration consumes textually, and that check is what rejected this form even once inference understood it; validation now skips exactly those inference resolved into a whole-array argument, so `y = de[0:1];`, `de[0:1]*2.0` and `I(p,n) <+ de[0:1];` stay refused and an ordinary `de[1]` is untouched. ORDER IS NOT COSMETIC -- the slice is built in the order WRITTEN, and since a Laplace coefficient k multiplies s^k a reversed slice is a DIFFERENT FILTER; pinned by VALUE rather than by acceptance, which is the difference between testing the feature and testing that it compiles: with `de = '{1.0, 1e-6, 9.0, 9.0}` the literal, the parameter slice and the variable slice all give 4.00674 for H(s)=1/(1+1e-6s) while `de[1:0]` gives 1.25e-05, and `de[0:3]` gives exactly what the bare `de` gives. **`zi_*`** needed one fix more than **`laplace_*`**: the Laplace filters resolve their arrays in their own arm and never look again, while the Z-transform filters pre-resolve and then fall through to the generic signature match, which re-inferred the slice and counted its two range bounds as two indices -- the error being fixed, reappearing one layer up. `infere_expr`'s BitSelect arm now returns the recorded array type for an already-resolved argument, the same early return its Path arm needed in E-458 for a resolved array parameter -- the second time that asymmetry has cost a fix. Verified filterslice 20/20 both solvers (value equivalences, the reversed-slice check, all eight filters accepting a slice, three out-of-range spellings refused with "bus bit-select index out of range" and never a crash, E-85's restriction holding in four places); E-458's lrmfuncs pin flipped from rejected to accepted, 228/228; corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences against the E-458 binary; `cargo test` 44 binaries; regression 373/373.

E-460 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs, openvaf/hir_ty/src/validation.rs, openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation/types.rs, openvaf/hir_lower/src/expr.rs, openvaf/openvaf-driver/src/cli_process.rs, examples/ctxguard_examples/ (new)*

FIVE DEFECTS FROM A ONE-HOUR HUNT: a crash, a silent table, two dropped statements, an ignored command line. (1) **`a.potential.access`** CRASHED THE COMPILER (exit 101, crash report, no diagnostic). LRM Syntax 5-4 names the case exactly -- "this syntax shall not be used for the `access`, `ddt_nature`, or `idt_nature` attributes of a nature, nor any other attribute whose value is not a constant expression" -- because those hold an IDENTIFIER, so `nature_attr_ty` found no value type and returned None, which pushed NO diagnostic, left the expression typed `Err`, and panicked the lowering ("invalid HIR: path .. was not resolved"), the same shape as E-455's scalar-index panic. Four spellings crashed -- `a.potential.access`, `a.potential.idt_nature`, `a.flow.access`, `br.potential.access` -- and which ones depended on whether the nature happened to DEFINE the attribute: electrical's Voltage defines `idt_nature` (Flux) so that crashed, while `ddt_nature` is undefined there and collected a clean "not found". The attribute that RESOLVES is the one that killed the compiler. (2) A MULTI-DIMENSIONAL TABLE FILE WHOSE AXIS WAS NOT ASCENDING WAS INTERPOLATED TO GARBAGE, SILENTLY. `interp_1d_values` states its precondition one function below the reader ("`grid` is ascending") and every 1-D form establishes it -- inline pairs and the two-column file both `sort_by` and `dedup_by` -- while the multi-dimensional reader was the one path that did not. With f(x,y)=x^2+y on x=[0,1,2], writing

that axis `2 1 0` returned 0.5 4.5 4.5, which matches NO reading of the file: taking it at its word (row k belongs to axis[k]) gives 4.5 3.0 1.5, and the ascending function gives 0.5 1.0 1.5 -- the interpolation simply clamped. Out-of-order and repeated coordinates were equally wrong, on any axis and in 3-D. Each axis is now sorted and de-duplicated with the value tensor PERMUTED to match, keeping the first of a repeat exactly as 1-D's `dedup_by` does. (3)(4) EVENT CONTROL STATEMENTS WERE ACCEPTED IN THE TWO PLACES THE LRM FORBIDS THEM -- `analog initial` (5.2.1) and analog functions (4.7.1) -- where the other two prohibitions in the same lists ( contributions, access functions/analog operators) were already enforced. No diagnostic fired even under `-E all`, and the guarded statement was silently DROPPED: `analog initial begin @( timer(1e-6)) q = 5.0; end` left q at 0.0, and the same inside a function returned x instead of 5.0. `@(initial_step)` happened to run, so the construct was not even consistently dead -- E-456's defect one level down. Checked on the STATEMENT rather than on what it guards, because the guarded body is exactly what used to disappear. (5) **`-D =1`** NAMED NO MACRO and was accepted then dropped, so the build failed against the SOURCE ("macro `GAIN` has not been declared") rather than the command line that was wrong. WRITTEN AND THEN WITHDRAWN (LRM 3.6.1.2): requiring `abstol`/`access` of every base nature, and forbidding a derived nature from changing `access`. Both were implemented and both removed when the sweep failed TWO SUITES IN ONE RUN -- E-39 supports a derived nature declaring its own access ON PURPOSE (`derivednature_demo.va` derives `Current2` that way) and E-422 pins "a nature with NO abstol attribute at all stays legal". E-422's stated reason ("the LRM makes it optional") does not survive reading 3.6.1.2, but the DECISION does, neither omission produces a wrong answer, and a bug hunt does not overturn a documented project decision as a side effect. The corpus passed with both rules in place -- it was the project's own suites that caught them, which is the whole argument for keeping them. Deliberately unchanged and pinned: `5.` as a real literal (malformed per LRM A.8.7, but always correct in value, unused in the corpus, and tightening a LEXER rule is precisely what broke eight shipping models earlier the same day in E-458), and named blocks inside analog functions (LRM 4.7.1 forbids them; they work, so refusing them would break working models to gain nothing). Four hunt candidates WITHDRAWN: a gain/GAIN collision (ngspice does warn, after the results); `-8>>1` (correct -- Verilog >> is logical and >>> gives -4); analog-function locals persisting (a known deferred item, now sharper: they persist across EVALUATIONS, so a single call reads a value written later); and `0 && (1/0)` rejected at compile time (literals only). Verified ctxguard 33/33 both solvers; corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences against the E-459 binary -- the check that matters most, since four of these fixes turn previously-accepted input into errors; `cargo test` 44 binaries; regression 374/374.

E-461 *openvaf/syntax/src/ast/node_ext.rs, openvaf/hir_def/src/body.rs, examples/strparam_examples/ (new)*

A STRING PARAMETER'S **`from '{...}`** SET SELECTION DID NOT WORK -- reported from the field: a legal member was rejected at setup with "Parameter ty is out of bounds!". ONLY THE FIRST MEMBER OF THE SET WAS EVER ENFORCED. LRM 3.4.2 gives string parameters their own range form (a `from`/`exclude` list built as an assignment pattern); the parser reads the WHOLE list -- `expr(p)` then `while p.eat(T![,])` -- so every member lands in the syntax tree, and the AST accessor then threw the rest away with `support::child`, which returns the FIRST. Measured on `from '{"aaa","bbb","ccc"}`: "aaa" accepted, "bbb" and "ccc" -- its own members -- rejected as out of bounds; writing the same set in a different ORDER changed which single value was legal. `exclude` failed the DANGEROUS way round: with `exclude '{"aaa","bbb"}` the value "bbb", explicitly forbidden by the model, was silently ACCEPTED. Nothing warned, because as far as the compiler was concerned the set had one member -- E-429's shape, elements parsed, attached and dropped by the accessor, with the errors hiding in what was dropped. A set now becomes one ParamConstraint PER MEMBER, which is exactly what `check_param` already wanted: `From` branches to the ok-exit on any match and calls `invalid` only on fallthrough, `Exclude` calls `invalid` on any match, so correct set semantics fall out with NO change to the checking logic. Numeric sets (**`from '{1,2,3}`**) had the identical defect and are fixed by the same change. Verified

strparam 33/33 both solvers; corpus 107 compiled by both, 17 rejected by both, 0 rc differences, 0 byte differences against the E-460 binary; `cargo test` 44 binaries; regression 375/375. The other half of this report was an ngspice defect -- see the companion row in ngspice_changes_full-report.md.

E-476 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_ty/src/validation.rs, examples/reportguard_examples/ (new)*

THE COMPILER WARNED ABOUT A NAME THE SIMULATOR SERVES. `$simparam("temp")` produced *warning[L025]: names the simulator parameter "temp", which this simulator does not provide*, with a note adding that *an unresolvable name is FATAL at run time*. Both claims were false: E-434 added `temp` to ngspice's `sim_params[]` precisely so a model ported from Spectre could ask for the simulation temperature, and the call returns the ambient -- 27 degC, or 40 under `.option temp=40`. The lint fired on the exact call that enhancement exists to serve. `SIMPARAM_NAMES` had not been updated with it, and the note printed beside the warning held a THIRD hand-written copy of the list that had drifted the same way -- so fixing only the array would have left the message still naming a set it no longer checked. The list whose own doc-comment says it is *taken from* ***`src/osdi/osdiload.c`*** is now a module-level **`pub(crate) const`** that the diagnostic is BUILT from, so the message cannot disagree with the check again; `temp` is added and nothing else changes. The suite checks the INVARIANT rather than the name: it reads the Rust array and the C `sim_params[]` array out of the two sources, requires them to be equal, then generates, compiles and RUNS a model that reads every served name with no default argument -- so a future divergence fails there rather than in a user's build log. NOT CHANGED: `$simparam` matching stays case-SENSITIVE, `$simparam("TNOM")` is fatal; round 45 reported that as an inconsistency and was wrong -- macOS has a case-insensitive filesystem, so two probe models written to `sr_TNOM.osdi` and `sr_tnom.osdi` were the same file by inode. Suite `reportguard_examples` 31/31 both solvers; full regression 390/390.

E-479 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_lower/src/expr.rs, openvaf/mir_opt/src/simplify.rs, openvaf/openvaf/src/lib.rs, examples/constguard_examples/ (new)*

EVERY VALUE GUARD ONLY EVER SAW A LITERAL. `const_num` had arms for literals, unary minus and the four binary operators -- and none for a path -- so naming a value made every check skip. `white_noise(-1e-12)` was rejected and `white_noise(-1e-12*1.0)` was too (folding IS applied), but `localparam real q = -1e-12; white_noise(q)` was accepted in silence, though a localparam is a compile-time constant the LRM forbids from being overridden and whose value the compiler therefore knows exactly. ONE MISSING ARM, ELEVEN GUARD SITES: `$bound_step`, `@(timer)`, `@(cross)`, `transition`, `absdelay`, `zi_nd`, `last_crossing`, `white_noise`, `flicker_noise`, the `analysis`/`$simparam` name checks and the parameter-range emptiness check. Not academic -- models name their constants, and a negative `transition` time supplied that way drove a 0-to-1 signal to -2.5 V and made it respond BEFORE the input edge, rc=0, no diagnostic from compiler or simulator. THE SAME ASSUMPTION BUILT THE COMPILE-TIME TABLES: `eval_const_real` folded only literals and unary minus and its caller turned *cannot fold* into `0.0`, so a NAMED table entry became a ZERO entry -- one table, one value written two ways, `$table_model` answering 15 and 5, a smooth plausible wrong curve; `noise_table` lost its noise entirely and the analysis reported a confident total. Only the ORDINATE column was affected, which is why it read as bad interpolation rather than a dropped value. **`abs(-0.0)`** DISAGREED WITH THE COMPILER'S OWN FOLDING -- lowered as `x < 0 ? -x : x`, and `-0.0 < 0.0` is false, so `1.0/abs(-0.0)` gave -inf from generated code and +inf from constant folding; fixed by normalising with `+0.0`, which required gating `x + 0 -> x` on the existing `EXACT_ALGEBRA` flag (exact for integers, and for every float except negative zero). GUARDS THAT DID NOT EXIST: six `$rdist_*` distributions were unchecked while `$rdist_uniform` was, so `$rdist_exponential(s, -1.0)` returned -1.735, a negative sample from a distribution supported on [0, inf); `$vt(-300)` returned a sign-flipped thermal voltage and `$vt(0)` put a NaN in the solution; `ddt`/`idt` took a negative `abstol`; and a `laplace_nd`/`laplace_zd` denominator with a zero highest-order coefficient made the

filter produce no output at all (`'{1.0}` gain 1, `'{1.0, 0.0}` gain 0) because the order came from the list LENGTH -- the `zi_*` twins already handled that padding correctly and still do. A BUILD THAT DEFINED NOTHING REPORTED SUCCESS: an unterminated `/*` swallows the module, and the compiler printed *Finished building*, exited 0 and wrote a 35 KB module-less `.osdi`, so the failure surfaced later as the simulator's *Unable to find definition of model*. NOT CHANGED, each reported by the hunt and withdrawn on reading the code: `@(timer)` period <= 0 still fires ONCE (LRM 5.10.3.3 -- that is how a computed one-shot is written), `noise_table_log` still refuses a negative power (its input is the same LINEAR Hz/power pair, interpolated log-log, so a negative has no logarithm -- only its diagnostic, which hardcoded the name `noise_table`, changed), `0.0 * NaN` still folds to 0 (E-337 keeps it: removing it moved HiSIM2's DC drain current by 10x), a literal that underflows still gives 0, and a `parameter` DEFAULT is still not policed. Suite `constguard_examples` 57/57 both solvers (28/57 against the shipped pre-fix compiler); all 40 `integration_tests/` models compile with identical exit codes; full regression 392/392.

E-489 *openvaf/hir_ty/src/validation/body.rs, openvaf/hir_lower/src/expr.rs (comment), examples/powguard_examples/ (new)*

**`pow`** AND `**` JOIN THE CONSTANT-DOMAIN GUARD. E-455 refuses a CONSTANT argument outside a function's domain -- `sqrt(-1.0)`, `ln(0.0)`, `asin(2.0)` and four more -- because the fold produces a NaN that reaches the user not as a compiler message but as *"Transient op failed, timestep too small"*, a convergence complaint for a NaN written literally in the source. **`pow`** was not in that list, and **`pow(-2.0, 0.5)`** IS **`sqrt(-2.0)`** -- a negative base with a fractional exponent has no real root. Measured before the fix it compiled clean and produced exactly that message. A zero base with a negative exponent is a division by zero and is likewise refused. TWO SPELLINGS, TWO CODE PATHS: `pow(x,y)` resolves to `BuiltIn::pow` and is judged in the call handler, while `x ** y` is `BinaryOp::Power` and never reaches it -- the first version of this fix guarded only the call form and `(-2.0)**0.5` still compiled, so a second arm judges the operator on the identical rule. Both inherit E-479's `const_num`, so a localparam base or exponent is seen exactly as a literal is. THE TRAP: `**` on two INTEGERS is a DIFFERENT operation -- IEEE 1364-2005 Table 5-6, implemented by E-420 -- where a negative exponent is fully defined (`2**-1` is 0, `-1**-3` is -1, a base of 0 is `'x` = 0 in integer context). Those are correct answers, not NaNs; the first version judged them by the REAL rule and rejected valid models, taking `vafdegen_examples` from 91/91 to ten failures, which is how the mistake was found. The arm now returns when the inferred type is Integer, and checks [7]-[11] hold that line from the other side. DELIBERATELY UNCHANGED: a run-time base or exponent and an overridable `parameter` are untouched, per E-455's stated convention that a run-time value out of domain is the model's own business. TWO FINDINGS FROM THE SAME HUNT WERE WITHDRAWN RATHER THAN FIXED, both on source evidence: real `/0` is not the integer case (E-333's guard exists because integer `sdiv x,0` is LLVM UB lowered to poison and a SIGTRAP -- `fdiv` is well-defined IEEE), and a dynamic array index out of range is left to the model, with the `elems[0]` fallback now documented at the lowering site including why NaN was rejected (the same lowering serves VARIABLE indices, where a transient out-of-range value would poison the iteration). Verified `powguard` 18/18 (12/18 pre-fix, 6 discriminating), `vafdegen` 91/91, and the whole 754-file .va corpus recompiled byte-identically with ZERO occurrences of the new diagnostic.